Project 2 Data 624

Samantha Barbaro, Catherine Dube, Luis Hernandez

2026-05-06

Contents

1 Project 2

1.1 Importing the data

Recap located at the end of the section. Skip there to jump past the code/workflow.

```
library(tidyverse)
library(VIM)

test_data <- read.csv(file.path(projPath, "testdataproj2.csv"), na.strings = c("", "NA", "null", "NULL"

training_data <- read.csv(file.path(projPath, "trainingdataproj2.csv"), na.strings = c("", "NA", "null"
```

```
glimpse(test_data)
```

```
## Rows: 267
## Columns: 33
## $ Brand.Code        <chr> "D", "A", "B", "B", "B", "B", "A", "B", "A", "D", "B~
## $ Carb.Volume       <dbl> 5.480000, 5.393333, 5.293333, 5.266667, 5.406667, 5.~
```

```
## $ Fill.Ounces       <dbl> 24.03333, 23.95333, 23.92000, 23.94000, 24.20000, 24~
## $ PC.Volume         <dbl> 0.2700000, 0.2266667, 0.3033333, 0.1860000, 0.160000~
## $ Carb.Pressure     <dbl> 65.4, 63.2, 66.4, 64.8, 69.4, 73.4, 65.2, 67.4, 66.8~
## $ Carb.Temp         <dbl> 134.6, 135.0, 140.4, 139.0, 142.2, 147.2, 134.6, 139~
## $ PSC               <dbl> 0.236, 0.042, 0.068, 0.004, 0.040, 0.078, 0.088, 0.0~
## $ PSC.Fill          <dbl> 0.40, 0.22, 0.10, 0.20, 0.30, 0.22, 0.14, 0.10, 0.48~
## $ PSC.CO2           <dbl> 0.04, 0.08, 0.02, 0.02, 0.06, NA, 0.00, 0.04, 0.04, ~
## $ Mnf.Flow          <dbl> -100, -100, -100, -100, -100, -100, -100, -100, -100~
## $ Carb.Pressure1    <dbl> 116.6, 118.8, 120.2, 124.8, 115.0, 118.6, 117.6, 121~
## $ Fill.Pressure     <dbl> 46.0, 46.2, 45.8, 40.0, 51.4, 46.4, 46.2, 40.0, 43.8~
## $ Hyd.Pressure1     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ Hyd.Pressure2     <dbl> NA, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ Hyd.Pressure3     <dbl> NA, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ Hyd.Pressure4     <int> 96, 112, 98, 132, 94, 94, 108, 108, 110, 106, 98, 96~
## $ Filler.Level      <dbl> 129.4, 120.0, 119.4, 120.2, 116.0, 120.4, 119.6, 131~
## $ Filler.Speed      <int> 3986, 4012, 4010, NA, 4018, 4010, 4010, NA, 4010, 10~
## $ Temperature       <dbl> 66.0, 65.6, 65.6, 74.4, 66.4, 66.6, 66.8, NA, 65.8, ~
## $ Usage.cont        <dbl> 21.66, 17.60, 24.18, 18.12, 21.32, 18.00, 17.68, 12.~
## $ Carb.Flow         <int> 2950, 2916, 3056, 28, 3214, 3064, 3042, 1972, 2502, ~
## $ Density           <dbl> 0.88, 1.50, 0.90, 0.74, 0.88, 0.84, 1.48, 1.60, 1.52~
## $ MFR               <dbl> 727.6, 735.8, 734.8, NA, 752.0, 732.0, 729.8, NA, 74~
## $ Balling           <dbl> 1.398, 2.942, 1.448, 1.056, 1.398, 1.298, 2.894, 3.3~
## $ Pressure.Vacuum   <dbl> -3.8, -4.4, -4.2, -4.0, -4.0, -3.8, -4.2, -4.4, -4.4~
## $ PH                <lgl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, ~
## $ Oxygen.Filler     <dbl> 0.022, 0.030, 0.046, NA, 0.082, 0.064, 0.042, 0.096,~
## $ Bowl.Setpoint     <int> 130, 120, 120, 120, 120, 120, 120, 120, 120, 120, 12~
## $ Pressure.Setpoint <dbl> 45.2, 46.0, 46.0, 46.0, 50.0, 46.0, 46.0, 46.0, 46.0~
## $ Air.Pressurer     <dbl> 142.6, 147.2, 146.6, 146.4, 145.8, 146.0, 145.0, 146~
## $ Alch.Rel          <dbl> 6.56, 7.14, 6.52, 6.48, 6.50, 6.50, 7.18, 7.16, 7.14~
## $ Carb.Rel          <dbl> 5.34, 5.58, 5.34, 5.50, 5.38, 5.42, 5.46, 5.42, 5.44~
## $ Balling.Lvl       <dbl> 1.48, 3.04, 1.46, 1.48, 1.46, 1.44, 3.02, 3.00, 3.10~
```

```
glimpse(training_data)
```

```
## Rows: 2,571
## Columns: 33
## $ Brand.Code        <chr> "B", "A", "B", "A", "A", "A", "A", "B", "B", "B", "B~
## $ Carb.Volume       <dbl> 5.340000, 5.426667, 5.286667, 5.440000, 5.486667, 5.~
## $ Fill.Ounces       <dbl> 23.96667, 24.00667, 24.06000, 24.00667, 24.31333, 23~
## $ PC.Volume         <dbl> 0.2633333, 0.2386667, 0.2633333, 0.2933333, 0.111333~
## $ Carb.Pressure     <dbl> 68.2, 68.4, 70.8, 63.0, 67.2, 66.6, 64.2, 67.6, 64.2~
## $ Carb.Temp         <dbl> 141.2, 139.6, 144.8, 132.6, 136.8, 138.4, 136.8, 141~
## $ PSC               <dbl> 0.104, 0.124, 0.090, NA, 0.026, 0.090, 0.128, 0.154,~
## $ PSC.Fill          <dbl> 0.26, 0.22, 0.34, 0.42, 0.16, 0.24, 0.40, 0.34, 0.12~
## $ PSC.CO2           <dbl> 0.04, 0.04, 0.16, 0.04, 0.12, 0.04, 0.04, 0.04, 0.14~
## $ Mnf.Flow          <dbl> -100, -100, -100, -100, -100, -100, -100, -100, -100~
## $ Carb.Pressure1    <dbl> 118.8, 121.6, 120.2, 115.2, 118.4, 119.6, 122.2, 124~
## $ Fill.Pressure     <dbl> 46.0, 46.0, 46.0, 46.4, 45.8, 45.6, 51.8, 46.8, 46.0~
## $ Hyd.Pressure1     <dbl> 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ Hyd.Pressure2     <dbl> NA, NA, NA, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ Hyd.Pressure3     <dbl> NA, NA, NA, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0~
## $ Hyd.Pressure4     <int> 118, 106, 82, 92, 92, 116, 124, 132, 90, 108, 94, 86~
## $ Filler.Level      <dbl> 121.2, 118.6, 120.0, 117.8, 118.6, 120.2, 123.4, 118~
## $ Filler.Speed      <int> 4002, 3986, 4020, 4012, 4010, 4014, NA, 1004, 4014, ~
```

```
## $ Temperature       <dbl> 66.0, 67.6, 67.0, 65.6, 65.6, 66.2, 65.8, 65.2, 65.4~
## $ Usage.cont        <dbl> 16.18, 19.90, 17.76, 17.42, 17.68, 23.82, 20.74, 18.~
## $ Carb.Flow         <int> 2932, 3144, 2914, 3062, 3054, 2948, 30, 684, 2902, 3~
## $ Density           <dbl> 0.88, 0.92, 1.58, 1.54, 1.54, 1.52, 0.84, 0.84, 0.90~
## $ MFR               <dbl> 725.0, 726.8, 735.0, 730.6, 722.8, 738.8, NA, NA, 74~
## $ Balling           <dbl> 1.398, 1.498, 3.142, 3.042, 3.042, 2.992, 1.298, 1.2~
## $ Pressure.Vacuum   <dbl> -4.0, -4.0, -3.8, -4.4, -4.4, -4.4, -4.4, -4.4, -4.4~
## $ PH                <dbl> 8.36, 8.26, 8.94, 8.24, 8.26, 8.32, 8.40, 8.38, 8.38~
## $ Oxygen.Filler     <dbl> 0.022, 0.026, 0.024, 0.030, 0.030, 0.024, 0.066, 0.0~
## $ Bowl.Setpoint     <int> 120, 120, 120, 120, 120, 120, 120, 120, 120, 120, 12~
## $ Pressure.Setpoint <dbl> 46.4, 46.8, 46.6, 46.0, 46.0, 46.0, 46.0, 46.0, 46.0~
## $ Air.Pressurer     <dbl> 142.6, 143.0, 142.0, 146.2, 146.2, 146.6, 146.2, 146~
## $ Alch.Rel          <dbl> 6.58, 6.56, 7.66, 7.14, 7.14, 7.16, 6.54, 6.52, 6.52~
## $ Carb.Rel          <dbl> 5.32, 5.30, 5.84, 5.42, 5.44, 5.44, 5.38, 5.34, 5.34~
## $ Balling.Lvl       <dbl> 1.48, 1.56, 3.28, 3.04, 3.04, 3.02, 1.44, 1.44, 1.44~
```

```
str(training_data)
```

```
## 'data.frame':    2571 obs. of  33 variables:
##  $ Brand.Code       : chr  "B" "A" "B" "A" ...
##  $ Carb.Volume      : num  5.34 5.43 5.29 5.44 5.49 ...
##  $ Fill.Ounces      : num  24 24 24.1 24 24.3 ...
##  $ PC.Volume        : num  0.263 0.239 0.263 0.293 0.111 ...
##  $ Carb.Pressure    : num  68.2 68.4 70.8 63 67.2 66.6 64.2 67.6 64.2 72 ...
##  $ Carb.Temp        : num  141 140 145 133 137 ...
##  $ PSC              : num  0.104 0.124 0.09 NA 0.026 0.09 0.128 0.154 0.132 0.014 ...
##  $ PSC.Fill         : num  0.26 0.22 0.34 0.42 0.16 0.24 0.4 0.34 0.12 0.24 ...
##  $ PSC.CO2          : num  0.04 0.04 0.16 0.04 0.12 0.04 0.04 0.04 0.14 0.06 ...
##  $ Mnf.Flow         : num  -100 -100 -100 -100 -100 -100 -100 -100 -100 -100 ...
##  $ Carb.Pressure1   : num  119 122 120 115 118 ...
##  $ Fill.Pressure    : num  46 46 46 46.4 45.8 45.6 51.8 46.8 46 45.2 ...
##  $ Hyd.Pressure1    : num  0 0 0 0 0 0 0 0 0 0 ...
##  $ Hyd.Pressure2    : num  NA NA NA 0 0 0 0 0 0 0 ...
##  $ Hyd.Pressure3    : num  NA NA NA 0 0 0 0 0 0 0 ...
##  $ Hyd.Pressure4    : int  118 106 82 92 92 116 124 132 90 108 ...
##  $ Filler.Level     : num  121 119 120 118 119 ...
##  $ Filler.Speed     : int  4002 3986 4020 4012 4010 4014 NA 1004 4014 4028 ...
##  $ Temperature      : num  66 67.6 67 65.6 65.6 66.2 65.8 65.2 65.4 66.6 ...
##  $ Usage.cont       : num  16.2 19.9 17.8 17.4 17.7 ...
##  $ Carb.Flow        : int  2932 3144 2914 3062 3054 2948 30 684 2902 3038 ...
##  $ Density          : num  0.88 0.92 1.58 1.54 1.54 1.52 0.84 0.84 0.9 0.9 ...
##  $ MFR              : num  725 727 735 731 723 ...
##  $ Balling          : num  1.4 1.5 3.14 3.04 3.04 ...
##  $ Pressure.Vacuum  : num  -4 -4 -3.8 -4.4 -4.4 -4.4 -4.4 -4.4 -4.4 -4.4 ...
##  $ PH               : num  8.36 8.26 8.94 8.24 8.26 8.32 8.4 8.38 8.38 8.5 ...
##  $ Oxygen.Filler    : num  0.022 0.026 0.024 0.03 0.03 0.024 0.066 0.046 0.064 0.022 ...
##  $ Bowl.Setpoint    : int  120 120 120 120 120 120 120 120 120 120 ...
##  $ Pressure.Setpoint: num  46.4 46.8 46.6 46 46 46 46 46 46 46 ...
##  $ Air.Pressurer    : num  143 143 142 146 146 ...
##  $ Alch.Rel         : num  6.58 6.56 7.66 7.14 7.14 7.16 6.54 6.52 6.52 6.54 ...
##  $ Carb.Rel         : num  5.32 5.3 5.84 5.42 5.44 5.44 5.38 5.34 5.34 5.34 ...
##  $ Balling.Lvl      : num  1.48 1.56 3.28 3.04 3.04 3.02 1.44 1.44 1.44 1.38 ...
```

All the rows with numbers are formatted as numbers. All the variables are numeric except for brand code,

likely the only categorical variable.

1.1.1 Recap

- Loaded the training set (2571 rows) and the evaluation set (267 rows) from the local CSVs in the repo.
- Did a quick look at the structure with `glimpse()` and `str()` to confirm everything imported correctly.
- All variables came in as numeric except `Brand.Code`, which is the only categorical predictor.

1.2 Exploring the data

> Recap located at the end of the section. Skip there to jump past the code/workflow.

We'll look at:

- The number of cases
- Missing values
- Data types (numerical vs. categorical)

1.2.1 Number of cases

```
nrow(training_data)
```

```
## [1] 2571
```

```
nrow(test_data)
```

```
## [1] 267
```

The training data has 2571 cases and the test data has 267.

1.2.2 Variables

Counting the number of columns/variables:

```
ncol(test_data)
```

```
## [1] 33
```

```
ncol(training_data)
```

```
## [1] 33
```

Both data sets have 33 columns.

Are all the variables represented in both data sets?

```r
colnames(test_data)
```

```
##  [1] "Brand.Code"        "Carb.Volume"       "Fill.Ounces"
##  [4] "PC.Volume"         "Carb.Pressure"     "Carb.Temp"
##  [7] "PSC"               "PSC.Fill"          "PSC.CO2"
## [10] "Mnf.Flow"          "Carb.Pressure1"    "Fill.Pressure"
## [13] "Hyd.Pressure1"     "Hyd.Pressure2"     "Hyd.Pressure3"
## [16] "Hyd.Pressure4"     "Filler.Level"      "Filler.Speed"
## [19] "Temperature"       "Usage.cont"        "Carb.Flow"
## [22] "Density"           "MFR"               "Balling"
## [25] "Pressure.Vacuum"   "PH"                "Oxygen.Filler"
## [28] "Bowl.Setpoint"     "Pressure.Setpoint" "Air.Pressurer"
## [31] "Alch.Rel"          "Carb.Rel"          "Balling.Lvl"
```

```r
colnames(training_data)
```

```
##  [1] "Brand.Code"        "Carb.Volume"       "Fill.Ounces"
##  [4] "PC.Volume"         "Carb.Pressure"     "Carb.Temp"
##  [7] "PSC"               "PSC.Fill"          "PSC.CO2"
## [10] "Mnf.Flow"          "Carb.Pressure1"    "Fill.Pressure"
## [13] "Hyd.Pressure1"     "Hyd.Pressure2"     "Hyd.Pressure3"
## [16] "Hyd.Pressure4"     "Filler.Level"      "Filler.Speed"
## [19] "Temperature"       "Usage.cont"        "Carb.Flow"
## [22] "Density"           "MFR"               "Balling"
## [25] "Pressure.Vacuum"   "PH"                "Oxygen.Filler"
## [28] "Bowl.Setpoint"     "Pressure.Setpoint" "Air.Pressurer"
## [31] "Alch.Rel"          "Carb.Rel"          "Balling.Lvl"
```

Both data sets have the same columns, so we can proceed.

1.2.3 Missing data

How many missing values are there? Are there any trends? How many complete cases are there?

1.2.3.1 Evaluation set (`test_data`) Note: `test_data` here is the evaluation file used for the project deliverable. PH is intentionally blank in this file because that's what the model predicts. It's not a held-out test split from training.

Count nulls/missing values in the evaluation set:

```r
sort(colSums(is.na(test_data)), decreasing = TRUE)
```

```
##               PH              MFR     Filler.Speed       Brand.Code
##              267               31               10                8
##      Fill.Ounces              PSC          PSC.CO2       PC.Volume
##                6                5                5                4
##   Carb.Pressure1    Hyd.Pressure4         PSC.Fill    Oxygen.Filler
##                4                4                3                3
##         Alch.Rel    Fill.Pressure     Filler.Level      Temperature
##                3                2                2                2
```

```
##        Usage.cont Pressure.Setpoint          Carb.Rel       Carb.Volume
##                 2                 2                 2                 1
##         Carb.Temp     Hyd.Pressure2     Hyd.Pressure3           Density
##                 1                 1                 1                 1
##           Balling   Pressure.Vacuum     Bowl.Setpoint     Air.Pressurer
##                 1                 1                 1                 1
##     Carb.Pressure          Mnf.Flow     Hyd.Pressure1         Carb.Flow
##                 0                 0                 0                 0
##       Balling.Lvl
##                 0
```

First off, all the **pH values are missing**. This is not a test set -it's what we will predict. The training data will have to be split on its own into a training and test set.

Most columns are missing a few (under 10) values or none at all. MFR is missing the most values - 31 out of 267, or about 12%. Filler speed is next, missing only 10 values.

Next, we'll look at the data row-wise and see if there are full or close-to-full missing cases.

```
rowSums(is.na(test_data))
```

```
##   [1]  3  1  1  4  1  2  1  4  1  2  3  1  1  1  1  4  1  1  1  1  1  2  3  1  1
##  [26]  1  1  1  2  1  1  2  2  2  2  1  1  1  1  2  1  2  1  1  1  1  1  1  1  1
##  [51]  3  1  2  2  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
##  [76]  1  2  1  1  1  1  1  1  1  5  2  1  1  1  1  2 15  1  2  2  1  3  1  2  1
## [101]  2  1  1  1  1  1  1  1  1  2  1  1  1  1  3  1  1  2  1  1  2  1  1  1  1
## [126]  1  2  2  1  1  2  2  2  3  1  1  3  2  2  2  1  1  1  2  1  1  1  1  2  1
## [151]  1  1  1  1  1  1  1  1  1  1  2  2  1  1  1  2  2  1  1  1  2  2  1  1  1
## [176]  1  1  1  2  1  1  1  1  1  2  1  1  1  1  1  1  1  1  2  1  1  1  1  1  1
## [201]  1  1  2  1  1  1  1  2  2  2  1  1  1  1  1  1  1  1  1  1  1  1  1  1  1
## [226]  1  3  1  1  1  1  2  1  1  1  2  1  1  1  1  1  1  2  1  1  1  1  1  3  1
## [251]  2  1  3  1  1  9  2  1  1  1  1  1  1  1  1  1  1
```

```
#totals by row (1 does not count)
table(rowSums(is.na(test_data)))
```

```
##
##   1   2   3   4   5   9  15
## 200  50  11   3   1   1   1
```

```
#total NAs (subtracting 267 for the missing pH values)
sum(is.na(test_data)) - 267
```

```
## [1] 107
```

Overall, a data set with 267 rows and 33 columns is missing 107 values, or about 1.3%. 67 out of 267 cases are missing at least one value (about 25%).

One case has 14 missing values, so it may make sense to drop that; another is missing 8. Otherwise, most rows with missing values are missing only 1-3.

1.2.3.2 Training data

Which variables are missing data?

```
sort(colSums(is.na(training_data)), decreasing = TRUE)
```

```
##               MFR        Brand.Code      Filler.Speed         PC.Volume
##               212               120                57                39
##           PSC.CO2       Fill.Ounces               PSC    Carb.Pressure1
##                39                38                33                32
##     Hyd.Pressure4     Carb.Pressure         Carb.Temp          PSC.Fill
##                30                27                26                23
##     Fill.Pressure      Filler.Level     Hyd.Pressure2     Hyd.Pressure3
##                22                20                15                15
##       Temperature     Oxygen.Filler Pressure.Setpoint     Hyd.Pressure1
##                14                12                12                11
##       Carb.Volume          Carb.Rel          Alch.Rel        Usage.cont
##                10                10                 9                 5
##                PH         Mnf.Flow         Carb.Flow     Bowl.Setpoint
##                 4                 2                 2                 2
##           Density           Balling       Balling.Lvl   Pressure.Vacuum
##                 1                 1                 1                 0
##     Air.Pressurer
##                 0
```

MFR is missing 212 out of 2571 cases, about 8.2%. Brand code, the only categorical variable, is missing 120 times, or about 4.7% of the time. Filler speed is missing 57 times, about 2.2%, and everything else is under 2%.

Let's see if anything strange is happening across rows:

```
#nothing out of the ordinary across rows. Most rows are complete.
#Rows with NAs typically 1 or 2 NAs, wich
rowSums(is.na(training_data))
```

```
##    [1]  2  2  2  1  0  0  2  1  0  0  0  0  1  0  0  0  0  0  0  0  0  2  0  0
##   [25]  0  1  0  0  0  0  0  0  0  0  0  1  1  0  0  0  1  0  0  0  0  0  0  1
##   [49]  0  2  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0
##   [73]  0  0  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0  0  2  0  0
##   [97]  0  1  0  0  0  0  0  0  0  0  0  2  5  0  0  0  0  0  1  0  0  0  0  0
##  [121]  0  0  0  0  2  0  0  1  0  0  0  0  0  1  0  0  0  2  0  0  0  0  0  0
##  [145]  0  0  0  0  0  0  1  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
##  [169]  0  0  0  0  0  0  0  0  1  0  2  0  0  0  0  0  2  0  0  0  0  0  0  0
##  [193]  0  0  0  1  0  0  0  0  0  0  0  0  0  0  1  1  1  1  1  1  1  1  0  0  0
##  [217]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  2  1  0  0  0  0  1
##  [241]  0  1  6  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0  0  1  2
##  [265]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
##  [289]  0  0  0  0  0  0  0  0  1  1  1  1  1  0  0  0  0  3  0  0  0  0  0  2
##  [313]  0  0  0  0  0  0  0  1  0  0  0  1  1  0  1  0  0  0  1  0  0  1  2  0
##  [337]  0  1  0  2  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0  0
##  [361]  0  0  0  0  2  2  0  2  0  0  0  1  0  2  1  0  1  0  0  0  0  0  0  0
##  [385]  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  2  0  0  0  0  0  0  0
##  [409]  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  2  2  1  1  0
##  [433]  0  0  2  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0
##  [457]  0  0  1  0  0  0  1  0  0  0  0  0  0  0  2  1  0  0  4  1  1  1  1  1
##  [481]  1  1  1  0  0  0  0  0  0  0  0  1  0  1  0  3  0  1  1  0  0  0  0  0
##  [505]  0  0  0  1  0  0  0  0  0  0  0  0  1  0  1  0  0  0  0  0  0  0  0  0
```

```
##  [529] 2 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 1 0
##  [553] 0 0 0 0 0 1 0 1 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0
##  [577] 1 0 0 0 0 0 0 1 0 0 0 3 0 0 1 0 0 0 0 0 0 0 0 0 0
##  [601] 0 0 0 0 0 0 0 0 2 4 0 0 0 0 0 0 0 0 0 0 0 0 1 0
##  [625] 0 0 0 0 3 0 0 0 0 0 0 0 0 0 0 0 0 3 0 0 0 0 0 0
##  [649] 1 0 0 0 0 0 1 0 2 2 1 0 0 0 0 0 0 0 0 0 0 0 1 0
##  [673] 0 1 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 5 2 0 0 1 2 3
##  [697] 3 0 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 1 0 0 3 0
##  [721] 0 2 0 0 0 0 0 0 0 0 0 1 2 0 0 0 0 9 4 0 0 0 0 0
##  [745] 0 0 0 0 0 1 0 0 1 0 3 0 0 1 0 0 0 0 0 0 1 0 0 0
##  [769] 0 0 2 2 3 3 3 3 4 3 3 3 3 3 3 3 4 0 0 0 0 0 0 0
##  [793] 0 0 2 0 0 0 1 2 4 1 0 0 0 0 0 0 2 1 0 0 0 0 0 3
##  [817] 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 2 1 0 0 0 1 0 0 0
##  [841] 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
##  [865] 2 0 0 0 0 0 0 0 0 0 0 0 0 0 2 2 0 0 0 0 1 0 0 0
##  [889] 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 6 0 0 0
##  [913] 2 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 0 0 0 0 0 1 0
##  [937] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 1 1 1 1 1 1
##  [961] 2 1 1 2 2 0 0 0 1 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0
##  [985] 0 0 0 0 0 0 0 0 0 0 2 1 1 1 0 3 0 0 0 0 0 0 0 0 0
## [1009] 0 0 0 0 0 2 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 2 0 0
## [1033] 0 0 2 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 3 0 0
## [1057] 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
## [1081] 0 0 0 0 0 0 0 1 0 0 0 0 0 8 2 0 1 0 0 0 0 1 0 0
## [1105] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1
## [1129] 0 0 0 0 0 0 0 0 3 1 0 0 0 2 0 0 0 0 0 0 0 0 0 0
## [1153] 0 0 0 0 1 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
## [1177] 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0
## [1201] 1 0 1 0 0 0 0 0 1 1 1 1 3 2 3 2 4 1 1 1 1 1 0 0
## [1225] 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 0 4 2 0 0 1 0 0
## [1249] 0 0 0 0 3 0 0 0 0 0 0 2 0 0 0 0 1 0 0 1 1 1 0 0
## [1273] 1 0 0 0 0 0 1 3 0 0 0 0 1 0 0 0 0 0 0 0 2 0 2 1
## [1297] 0 1 0 4 0 1 0 1 0 0 1 1 1 0 0 0 0 0 1 1 1 2 1 1
## [1321] 1 1 1 2 1 1 0 0 0 0 1 1 1 1 1 1 2 1 1 1 2 1 1 1
## [1345] 1 1 2 1 1 1 1 1 1 1 2 1 2 0 1 1 1 1 1 1 3 1 1 2
## [1369] 2 1 0 0 0 0 0 2 0 0 0 0 0 0 0 1 1 1 1 0 0 0 0 0
## [1393] 0 0 0 2 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [1417] 0 0 0 0 3 2 0 0 0 0 0 1 0 0 3 0 0 0 0 0 0 0 0 0
## [1441] 0 0 0 0 0 0 0 0 3 0 0 0 0 0 0 0 0 0 0 0 3 0 0 0
## [1465] 0 0 1 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0
## [1489] 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0
## [1513] 0 0 0 0 0 0 0 2 0 0 0 0 0 0 0 0 4 1 0 0 0 0 0 6 0
## [1537] 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1
## [1561] 2 3 0 0 0 0 0 0 0 0 0 0 0 0 1 3 0 0 0 0 0 0 0 0 0
## [1585] 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [1609] 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0
## [1633] 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
## [1657] 0 0 0 0 0 0 0 0 1 1 0 0 0 0 1 0 1 0 0 0 0 0 1 0
## [1681] 0 0 1 0 0 1 1 0 1 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0
## [1705] 0 0 1 0 0 0 0 0 0 0 0 0 0 0 12 1 0 0 1 0 3 1 2 0
## [1729] 0 0 0 1 0 0 0 2 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0
## [1753] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 2 0 0 0 0 0 1 1 0 1 0
## [1777] 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 2 1 1
## [1801] 4 1 1 1 1 1 3 1 0 0 0 0 0 0 0 2 2 0 0 0 0 0 0 0
```

```
## [1825]  2  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  5  0  0  0
## [1849]  0  0  1  0  0  2  0  0  0  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0
## [1873]  0  0  0  0  0  0  3  0  0  0  0  0  0  0  0  0  0  0  0  0  0  2  0  0  0
## [1897]  0  0  0  2  0  2  0  0  0  0  0  1  0  0  0  0  0  1  0  0  0  0  1  0
## [1921]  1  0  0  0  0  2  0  1  0  0  1  0  0  0  0  0  0  0  0  0  1  0  1  0  0
## [1945]  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  2  1  1  1  1  1  0  0
## [1969]  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  1  0  0  0
## [1993]  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  1
## [2017]  3  1  1  1  3  1  1  3  1  1  0  0  0  0  0  1  0  0  0  0  0  0  0  0
## [2041]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0
## [2065]  0  2  0  1  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0  2  0  0
## [2089]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0  1  0
## [2113]  0  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0  1  0  0  0  0  0  0  0
## [2137]  0  0  0  1  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  2  0  0  0
## [2161]  0  0  0  0  0  0  0  0  0  0  0  0  0  1  1  0  0  0  0  0  0  0  0  0
## [2185]  1  1  1  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0
## [2209]  0  0  0  0  0  5  0  0  0  1  0  0  0  0  0  0  2  0  0  0  0  0  0  0
## [2233]  0  0  1  0  0  0  0  1  0  0  2  0  0  0  0  0  0  0  0  0  0  0  0  0
## [2257]  2  1  1  1  0  0  0  0  0  0  0  0  0  0  0  0  4  0  0  0  0  0  0  2  0
## [2281]  0  1  0  0  0  0  1  0  0  0  0  2  0  0  0  0  0  1  0  0  0  2  0  0
## [2305]  0  0  1  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  2  0  0  1  2
## [2329]  0  1  1  0  1  0  0  0  0  0  1  2  0  0  0  0  0  0  0  0  0  0  0  0  0
## [2353]  0  0  0  0  2  0  3  0  0  0  0  0  0  0  0  0  0  0  1  0  0  3  0  0  0
## [2377]  0  0  0  2  0  0  0  0  2  3  0  0  1  0  0  0  0  0  0  1  0  0  0  0
## [2401]  0  0  0  0  0  0  0  0  1  0  0  0  2  2  0  0  0  0  0  0  0  0  0  0  0
## [2425]  0  0  1  1  1  0  0  0  0  0  0  0  2  0  1  0  0  0  0  0  0  0  0  1  0
## [2449]  0  0  0  0  0  0  0  0  1  0  0  0  1  0  0  0  0  0  0  0  0  0  1  1
## [2473]  0  0  0  0  2  0  2  0  0  0  0  0  0  0  0  1  0  0  0  0  0  0  0  0
## [2497]  0  0  0  0  0  0  0  1  0  0  0  7  1  0  0  0  0  0  2  0  0  0  0  0
## [2521]  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  0  1  0  0  0  0  0
## [2545]  0  0  0  0  0  1  0  0  0  0  0  0  0  0  2  0  0  0  0  1  0  0  1  1
## [2569]  0  0  3
```

```
table(rowSums(is.na(training_data)))
```

```
##
##    0    1    2    3    4    5    6    7    8    9   12
## 2038  345  119   45   13    4    3    1    1    1    1
```

```
sum(is.na(test_data))
```

```
## [1] 374
```

Overall, 844 values are missing across the training set, about 1% of all cells. 2038/2571 cases are complete. That's about 79% complete, 21% incomplete. Let's also check to see if the data is MCAR.

```
library(naniar)
numeric_only <- training_data %>%
  select(-Brand.Code)

numeric_only <- numeric_only %>%
  dplyr::select(where(is.numeric)) %>%
```

```
  mutate(across(everything(), as.double))

mcar_test(numeric_only)
```

```
## # A tibble: 1 x 4
##   statistic    df p.value missing.patterns
##       <dbl> <dbl>   <dbl>            <int>
## 1    10791.  2871       0               99
```

The p-value is 0, which means the data most likely isn't missing completely at random (there are patterns in the missing data).

1.2.3.3 A closer look at Brand.Code missingness

The training set has 120 rows where Brand.Code is blank (about 4.7% of cases). Before deciding how to handle them in modeling, we check whether those rows look different from the rest. If the missingness carries signal about PH, we should preserve it rather than impute it away.

```
# How does PH look in rows where Brand.Code is missing vs known?
training_data %>%
  mutate(brand_missing = is.na(Brand.Code) | Brand.Code == "") %>%
  filter(!is.na(PH)) %>%
  group_by(brand_missing) %>%
  summarise(
    n = n(),
    ph_mean = mean(PH),
    ph_median = median(PH),
    ph_sd = sd(PH),
    ph_min = min(PH),
    ph_max = max(PH)
  )
```

```
## # A tibble: 2 x 7
##   brand_missing     n ph_mean ph_median ph_sd ph_min ph_max
##   <lgl>         <int>   <dbl>     <dbl> <dbl>  <dbl>  <dbl>
## 1 FALSE          2447    8.55      8.54 0.172   7.88   9.36
## 2 TRUE            120    8.49      8.51 0.177   8.12   8.88
```

Two visible differences. Missing-Brand rows have a mean PH about 0.06 units below known-Brand rows (8.49 vs 8.55), and a noticeably narrower PH range (8.12 to 8.88 vs 7.88 to 9.36). They cluster toward the middle of the distribution rather than the tails.

That looks like what Kuhn and Johnson calls informative missingness: the absence of a Brand.Code value is itself associated with a specific PH pattern. Imputing the missing values into one of A/B/C/D would roll these rows into whichever brand the imputer picks, distorting that brand's signal and erasing the mean shift. The cleaner approach is to keep the missingness as its own level (`"Unknown"`) when we model.

1.2.4 Recap

- 2571 training cases, 267 evaluation cases, 33 columns in both
- All `PH` values in the evaluation set are missing. This is the file we are predicting on, not a test set
- The missingness is concentrated in:

  - `MFR` (~8% training, ~12% eval)
  - `Brand.Code` (~5% training)
  - `Filler.Speed` (~2% training)
  - everything else was under 2%

- ~79% of training cases are complete. We ran the MCAR test (`naniar::mcar_test`) which returned p ~ 0, so the missingness is not completely at random
- Did some extra digging on Brand.Code's missingness (since it is categorical). And it shows the 120 missing-Brand rows have a slightly lower mean PH and a narrower PH range than known-Brand rows, so the missingness itself looks informative. Right now we're opting to preserve it rather than impute it when modeling.

1.3 Data cleaning

Recap located at the end of the section. Skip there to jump past the code/workflow.

In this section:

- Variables with zero or near-zero variance
- Highly correlated variables
- Outliers/range of variables/scaling (if needed)

1.3.1 Checking for variables with near-zero or zero variance

This data set has 33 variables. Let's check to see if they're all (potentially) providing useful predictive information by testing for near-zero or zero variance.

```
library(caret)
nzv_results <- nearZeroVar(training_data, saveMetrics = TRUE)

nzv_results
```

```
##                   freqRatio percentUnique zeroVar   nzv
## Brand.Code         2.014634     0.1555815   FALSE FALSE
## Carb.Volume        1.055556     3.9284325   FALSE FALSE
## Fill.Ounces        1.163043     3.5783742   FALSE FALSE
## PC.Volume          1.100000    17.6584986   FALSE FALSE
## Carb.Pressure      1.016393     4.1229094   FALSE FALSE
## Carb.Temp          1.016667     4.7841307   FALSE FALSE
## PSC                1.211538     5.0175029   FALSE FALSE
## PSC.Fill           1.091892     1.2446519   FALSE FALSE
## PSC.CO2            1.078303     0.5056398   FALSE FALSE
## Mnf.Flow           1.048443    18.9420459   FALSE FALSE
## Carb.Pressure1     1.031746     5.4453520   FALSE FALSE
## Fill.Pressure      1.762931     4.2007001   FALSE FALSE
## Hyd.Pressure1     31.111111     9.5293660   FALSE  TRUE
## Hyd.Pressure2      7.271028     8.0513419   FALSE FALSE
## Hyd.Pressure3     11.450704     7.4679113   FALSE FALSE
## Hyd.Pressure4      1.008264     1.5558149   FALSE FALSE
## Filler.Level       1.086957    11.2018670   FALSE FALSE
## Filler.Speed       1.045161     9.4904706   FALSE FALSE
```

```
## Temperature        1.040000     2.1781408   FALSE FALSE
## Usage.cont         1.105263    18.7086737   FALSE FALSE
## Carb.Flow          1.586207    20.7312330   FALSE FALSE
## Density            1.108374     3.0338390   FALSE FALSE
## MFR                1.222222    22.8315830   FALSE FALSE
## Balling            1.193182     8.4402956   FALSE FALSE
## Pressure.Vacuum    1.389728     0.6223259   FALSE FALSE
## PH                 1.078125     2.0225593   FALSE FALSE
## Oxygen.Filler      1.266667    13.1466356   FALSE FALSE
## Bowl.Setpoint      2.990847     0.4278491   FALSE FALSE
## Pressure.Setpoint  1.319361     0.3111630   FALSE FALSE
## Air.Pressurer      1.093407     1.2446519   FALSE FALSE
## Alch.Rel           1.098315     2.0614547   FALSE FALSE
## Carb.Rel           1.011583     1.6336056   FALSE FALSE
## Balling.Lvl        1.294872     3.1894205   FALSE FALSE
```

`Hyd.Pressure1` has a near-zero variance, so we could eliminate that one.

1.3.2 Looking for highly correlated variables

We'll create a correlation grid, so we can see actual correlation between variables. We'll also create a correlation plot and use `findCorrelation` from the `caret` package to find variables that have 90% or higher correlation.

```
correlation_grid <- training_data %>%
  drop_na() %>%
  select(-Brand.Code) %>%
  cor()

correlation_grid <- as.data.frame(correlation_grid)

correlation_grid
```

```
##                   Carb.Volume  Fill.Ounces    PC.Volume Carb.Pressure
## Carb.Volume        1.000000000  0.022648036 -0.243234517  0.4278586099
## Fill.Ounces        0.022648036  1.000000000 -0.172701966  0.0044669914
## PC.Volume         -0.243234517 -0.172701966  1.000000000 -0.1300941804
## Carb.Pressure      0.427858610  0.004466991 -0.130094180  1.0000000000
## Carb.Temp         -0.135080260 -0.020073097  0.008599427  0.8243350333
## PSC               -0.020625187  0.043056041  0.190961266 -0.0520086907
## PSC.Fill          -0.012364141  0.069748653 -0.055914847 -0.0179275381
## PSC.CO2           -0.051977547  0.014705610 -0.055356933 -0.0174011136
## Mnf.Flow           0.100525364 -0.005787251 -0.203032882  0.0189865815
## Carb.Pressure1     0.081146212 -0.004186326 -0.238496181  0.0221844474
## Fill.Pressure     -0.096123314  0.068377098 -0.095744421 -0.0850458606
## Hyd.Pressure1     -0.051894268 -0.146359721  0.271171741 -0.0704043564
## Hyd.Pressure2      0.033567314 -0.143047986  0.028768135 -0.0267152675
## Hyd.Pressure3      0.057610071 -0.107816452 -0.024741466 -0.0082140513
## Hyd.Pressure4     -0.567908627  0.056086557  0.132413786 -0.3229128556
## Filler.Level      -0.018377226  0.010856355  0.220434988  0.0097134297
## Filler.Speed       0.003233452  0.044277046 -0.073766021  0.0320230919
## Temperature       -0.189760851 -0.011208233  0.142065435 -0.0490025241
## Usage.cont         0.073297402  0.085289046 -0.269204705  0.0004415267
```

```
## Carb.Flow         -0.088782919 -0.116991410  0.254535534 -0.0111938869
## Density            0.801808803 -0.080414770 -0.167310345  0.4591743002
## MFR                0.010144634  0.043445999 -0.066678296  0.0292986178
## Balling            0.818778791 -0.068439337 -0.204785622  0.4539307252
## Pressure.Vacuum   -0.079190180  0.052165821 -0.051605143  0.0051279147
## PH                 0.073251595 -0.118207504  0.097746859  0.0788617427
## Oxygen.Filler     -0.112682435 -0.032064258  0.212285069 -0.0506609705
## Bowl.Setpoint      0.001993607  0.011188522  0.210972310  0.0187581570
## Pressure.Setpoint -0.160640146  0.065078231 -0.022210124 -0.1171228371
## Air.Pressurer     -0.024854172  0.058101103 -0.036605483  0.0219237613
## Alch.Rel           0.824787575 -0.110432476 -0.177865067  0.4590380468
## Carb.Rel           0.841072250 -0.118666586 -0.167433257  0.4736224720
## Balling.Lvl        0.817993347 -0.063820071 -0.210081156  0.4589466196
##                      Carb.Temp          PSC     PSC.Fill      PSC.CO2
## Carb.Volume       -0.135080260 -0.020625187 -0.012364141 -0.051977547
## Fill.Ounces       -0.020073097  0.043056041  0.069748653  0.014705610
## PC.Volume          0.008599427  0.190961266 -0.055914847 -0.055356933
## Carb.Pressure      0.824335033 -0.052008691 -0.017927538 -0.017401114
## Carb.Temp          1.000000000 -0.044380293 -0.010968792  0.023660514
## PSC               -0.044380293  1.000000000  0.193242059  0.053922116
## PSC.Fill          -0.010968792  0.193242059  1.000000000  0.201967429
## PSC.CO2            0.023660514  0.053922116  0.201967429  1.000000000
## Mnf.Flow          -0.030110321  0.042982180 -0.033801564  0.049931986
## Carb.Pressure1    -0.014312694 -0.011071066 -0.029772433  0.038316144
## Fill.Pressure     -0.028433189  0.030908100 -0.010528126  0.078519840
## Hyd.Pressure1     -0.045431217 -0.019758828 -0.053220514 -0.049037869
## Hyd.Pressure2     -0.039543090 -0.033813839 -0.080467321 -0.021838338
## Hyd.Pressure3     -0.035120030 -0.020448188 -0.068958062  0.005757630
## Hyd.Pressure4     -0.002573081  0.036485173  0.019810466  0.054940092
## Filler.Level       0.011038984 -0.016061814  0.048227359 -0.042306839
## Filler.Speed       0.029355933 -0.002395360 -0.017295907 -0.008421406
## Temperature        0.061914927  0.014217909  0.030080929  0.037007308
## Usage.cont        -0.043584359  0.049860720 -0.025727952  0.032502663
## Carb.Flow          0.045050619 -0.037361416  0.017579095 -0.013058433
## Density            0.021593836 -0.067168725 -0.024708871 -0.045382139
## MFR                0.023442657  0.007659111 -0.008663604 -0.004263878
## Balling            0.006005105 -0.064450826 -0.019108545 -0.043036807
## Pressure.Vacuum    0.037327836  0.055246194  0.033093793 -0.018438819
## PH                 0.034594375 -0.069906559 -0.031374041 -0.088759922
## Oxygen.Filler      0.012010201  0.002765929 -0.005769535 -0.016971816
## Bowl.Setpoint      0.007550616 -0.020808347  0.049191096 -0.044820225
## Pressure.Setpoint -0.027029545  0.042979144 -0.005800041  0.082772777
## Air.Pressurer      0.038177076  0.061433424 -0.017995701 -0.006984004
## Alch.Rel           0.004966509 -0.060809035 -0.017045886 -0.050859393
## Carb.Rel           0.009020802 -0.065354969 -0.015365172 -0.066475871
## Balling.Lvl        0.010458674 -0.061576860 -0.016064731 -0.047296997
##                      Mnf.Flow Carb.Pressure1 Fill.Pressure Hyd.Pressure1
## Carb.Volume        0.100525364    0.081146212   -0.09612331  -0.051894268
## Fill.Ounces       -0.005787251   -0.004186326    0.06837710  -0.146359721
## PC.Volume         -0.203032882   -0.238496181   -0.09574442   0.271171741
## Carb.Pressure      0.018986582    0.022184447   -0.08504586  -0.070404356
## Carb.Temp         -0.030110321   -0.014312694   -0.02843319  -0.045431217
## PSC                0.042982180   -0.011071066    0.03090810  -0.019758828
## PSC.Fill          -0.033801564   -0.029772433   -0.01052813  -0.053220514
```

```
## PSC.CO2             0.049931986    0.038316144    0.07851984  -0.049037869
## Mnf.Flow            1.000000000    0.465355315    0.49111329   0.298997690
## Carb.Pressure1      0.465355315    1.000000000    0.22002705  -0.012711716
## Fill.Pressure       0.491113286    0.220027054    1.00000000   0.137594249
## Hyd.Pressure1       0.298997690   -0.012711716    0.13759425   1.000000000
## Hyd.Pressure2       0.648337926    0.284710255    0.31836940   0.711851345
## Hyd.Pressure3       0.764142833    0.375658047    0.43637028   0.605589019
## Hyd.Pressure4       0.073124200    0.038936494    0.29144885   0.051658157
## Filler.Level       -0.608380668   -0.425646963   -0.45892889  -0.029684260
## Filler.Speed        0.024363769    0.012389362   -0.21347499  -0.022850088
## Temperature        -0.063477350   -0.070067643    0.06397894  -0.073341745
## Usage.cont          0.560192617    0.354875344    0.30257079   0.118691986
## Carb.Flow          -0.497211539   -0.273632871   -0.18715299  -0.169136801
## Density             0.021343082    0.040888366   -0.20887500  -0.003601856
## MFR                 0.019593355    0.013931715   -0.21290076  -0.035646540
## Balling             0.112393982    0.073508154   -0.15676227   0.025863408
## Pressure.Vacuum    -0.559943315   -0.284881843   -0.30810832  -0.324133005
## PH                 -0.457218313   -0.112108310   -0.31311751  -0.048631533
## Oxygen.Filler      -0.550066399   -0.278795209   -0.24769592  -0.118317983
## Bowl.Setpoint      -0.611580420   -0.432671459   -0.42609290  -0.029844050
## Pressure.Setpoint   0.482530739    0.234201877    0.81528556   0.190079720
## Air.Pressurer      -0.041525175    0.061506659    0.03280945  -0.209227461
## Alch.Rel            0.027999719    0.009047055   -0.20467008   0.006667887
## Carb.Rel           -0.027880871    0.004804776   -0.20272201   0.034387945
## Balling.Lvl         0.042267589    0.033601804   -0.17885789  -0.007754220
##                    Hyd.Pressure2 Hyd.Pressure3 Hyd.Pressure4 Filler.Level
## Carb.Volume           0.03356731    0.057610071  -0.567908627  -0.01837723
## Fill.Ounces          -0.14304799   -0.107816452   0.056086557   0.01085635
## PC.Volume             0.02876814   -0.024741466   0.132413786   0.22043499
## Carb.Pressure        -0.02671527   -0.008214051  -0.322912856   0.00971343
## Carb.Temp            -0.03954309   -0.035120030  -0.002573081   0.01103898
## PSC                  -0.03381384   -0.020448188   0.036485173  -0.01606181
## PSC.Fill             -0.08046732   -0.068958062   0.019810466   0.04822736
## PSC.CO2              -0.02183834    0.005757630   0.054940092  -0.04230684
## Mnf.Flow              0.64833793    0.764142833   0.073124200  -0.60838067
## Carb.Pressure1        0.28471026    0.375658047   0.038936494  -0.42564696
## Fill.Pressure         0.31836940    0.436370279   0.291448848  -0.45892889
## Hyd.Pressure1         0.71185135    0.605589019   0.051658157  -0.02968426
## Hyd.Pressure2         1.00000000    0.917661875   0.022980025  -0.41103521
## Hyd.Pressure3         0.91766187    1.000000000   0.046649319  -0.52796590
## Hyd.Pressure4         0.02298003    0.046649319   1.000000000  -0.10010344
## Filler.Level         -0.41103521   -0.527965898  -0.100103442   1.00000000
## Filler.Speed          0.07028157    0.031692930  -0.259685780   0.06319357
## Temperature          -0.13725053   -0.120288087   0.283933984   0.05040323
## Usage.cont            0.37224053    0.419761856   0.070916924  -0.35740823
## Carb.Flow            -0.32187768   -0.363691033  -0.009527157   0.01743320
## Density               0.05842429    0.040354370  -0.584381995   0.00772950
## MFR                   0.04937645    0.012841039  -0.280014344   0.06935680
## Balling               0.10480326    0.125869294  -0.594996647   0.01188558
## Pressure.Vacuum      -0.62816754   -0.672258326  -0.054652623   0.35866815
## PH                   -0.21571713   -0.260535183  -0.184518087   0.34879540
## Oxygen.Filler        -0.29438355   -0.363660021   0.009692254   0.22540536
## Bowl.Setpoint        -0.41457787   -0.529215855  -0.117019292   0.97724166
## Pressure.Setpoint     0.34183180    0.459918732   0.288770453  -0.44595292
```

```
## Air.Pressurer       -0.17237591  -0.067700529   0.021731381  -0.12573419
## Alch.Rel             0.03916606   0.049948952  -0.685576013   0.04355856
## Carb.Rel             0.03150549   0.013543248  -0.581827500   0.13107856
## Balling.Lvl          0.02885940   0.041595480  -0.575044299   0.05317923
##                   Filler.Speed Temperature    Usage.cont    Carb.Flow
## Carb.Volume        0.003233452 -0.18976085  0.0732974017 -0.088782919
## Fill.Ounces        0.044277046 -0.01120823  0.0852890456 -0.116991410
## PC.Volume         -0.073766021  0.14206544 -0.2692047046  0.254535534
## Carb.Pressure      0.032023092 -0.04900252  0.0004415267 -0.011193887
## Carb.Temp          0.029355933  0.06191493 -0.0435843590  0.045050619
## PSC               -0.002395360  0.01421791  0.0498607203 -0.037361416
## PSC.Fill          -0.017295907  0.03008093 -0.0257279520  0.017579095
## PSC.CO2           -0.008421406  0.03700731  0.0325026634 -0.013058433
## Mnf.Flow           0.024363769 -0.06347735  0.5601926171 -0.497211539
## Carb.Pressure1     0.012389362 -0.07006764  0.3548753438 -0.273632871
## Fill.Pressure     -0.213474995  0.06397894  0.3025707885 -0.187152994
## Hyd.Pressure1     -0.022850088 -0.07334175  0.1186919862 -0.169136801
## Hyd.Pressure2      0.070281568 -0.13725053  0.3722405332 -0.321877677
## Hyd.Pressure3      0.031692930 -0.12028809  0.4197618556 -0.363691033
## Hyd.Pressure4     -0.259685780  0.28393398  0.0709169237 -0.009527157
## Filler.Level       0.063193573  0.05040323 -0.3574082260  0.017433196
## Filler.Speed       1.000000000 -0.06794451  0.0466749382 -0.062641899
## Temperature       -0.067944506  1.00000000 -0.0950428265  0.140119431
## Usage.cont         0.046674938 -0.09504283  1.0000000000 -0.488854072
## Carb.Flow         -0.062641899  0.14011943 -0.4888540721  1.000000000
## Density            0.025454377 -0.19369890 -0.0045531798  0.023058872
## MFR                0.951264445 -0.08156825  0.0397443802 -0.074148067
## Balling            0.037938064 -0.23844781  0.0692694347 -0.102299824
## Pressure.Vacuum    0.045187804  0.04313148 -0.3196915871  0.290026894
## PH                -0.047596639 -0.18152910 -0.3577042377  0.240757744
## Oxygen.Filler     -0.044126285  0.09338308 -0.3164959408  0.385692082
## Bowl.Setpoint      0.027688288  0.05010937 -0.3584563273  0.012221618
## Pressure.Setpoint -0.068525700  0.05904194  0.2626733751 -0.166362628
## Air.Pressurer     -0.006199674  0.06692883 -0.1025056258  0.137849013
## Alch.Rel          -0.005598672 -0.24776232 -0.0237408078  0.007527013
## Carb.Rel           0.005300760 -0.13595194 -0.0390798757 -0.006940815
## Balling.Lvl        0.003823258 -0.22497504  0.0232069975 -0.055365859
##                        Density          MFR      Balling Pressure.Vacuum
## Carb.Volume        0.801808803  0.010144634  0.818778791    -0.079190180
## Fill.Ounces       -0.080414770  0.043445999 -0.068439337     0.052165821
## PC.Volume         -0.167310345 -0.066678296 -0.204785622    -0.051605143
## Carb.Pressure      0.459174300  0.029298618  0.453930725     0.005127915
## Carb.Temp          0.021593836  0.023442657  0.006005105     0.037327836
## PSC               -0.067168725  0.007659111 -0.064450826     0.055246194
## PSC.Fill          -0.024708871 -0.008663604 -0.019108545     0.033093793
## PSC.CO2           -0.045382139 -0.004263878 -0.043036807    -0.018438819
## Mnf.Flow           0.021343082  0.019593355  0.112393982    -0.559943315
## Carb.Pressure1     0.040888366  0.013931715  0.073508154    -0.284881843
## Fill.Pressure     -0.208875002 -0.212900763 -0.156762274    -0.308108315
## Hyd.Pressure1     -0.003601856 -0.035646540  0.025863408    -0.324133005
## Hyd.Pressure2      0.058424287  0.049376448  0.104803262    -0.628167539
## Hyd.Pressure3      0.040354370  0.012841039  0.125869294    -0.672258326
## Hyd.Pressure4     -0.584381995 -0.280014344 -0.594996647    -0.054652623
## Filler.Level       0.007729500  0.069356797  0.011885578     0.358668150
```

```
## Filler.Speed       0.025454377  0.951264445  0.037938064     0.045187804
## Temperature       -0.193698895 -0.081568252 -0.238447810     0.043131483
## Usage.cont        -0.004553180  0.039744380  0.069269435    -0.319691587
## Carb.Flow          0.023058872 -0.074148067 -0.102299824     0.290026894
## Density            1.000000000  0.039383665  0.953127632    -0.083762802
## MFR                0.039383665  1.000000000  0.051171065     0.043987724
## Balling            0.953127632  0.051171065  1.000000000    -0.167052546
## Pressure.Vacuum   -0.083762802  0.043987724 -0.167052546     1.000000000
## PH                 0.095340535 -0.052377217  0.072297357     0.213173584
## Oxygen.Filler     -0.046621720 -0.048876320 -0.113757056     0.278478283
## Bowl.Setpoint      0.019792536  0.042312091  0.027415802     0.358696671
## Pressure.Setpoint -0.250884909 -0.064390559 -0.210098742    -0.298520301
## Air.Pressurer     -0.082976446 -0.008182360 -0.102307854     0.175090738
## Alch.Rel           0.922993173 -0.001003949  0.945725964    -0.054788103
## Carb.Rel           0.860322233  0.010884909  0.858990647    -0.008434181
## Balling.Lvl        0.956417297  0.016039036  0.988412082    -0.051626926
##                             PH Oxygen.Filler Bowl.Setpoint Pressure.Setpoint
## Carb.Volume        0.073251595  -0.112682435   0.001993607      -0.160640146
## Fill.Ounces       -0.118207504  -0.032064258   0.011188522       0.065078231
## PC.Volume          0.097746859   0.212285069   0.210972310      -0.022210124
## Carb.Pressure      0.078861743  -0.050660970   0.018758157      -0.117122837
## Carb.Temp          0.034594375   0.012010201   0.007550616      -0.027029545
## PSC               -0.069906559   0.002765929  -0.020808347       0.042979144
## PSC.Fill          -0.031374041  -0.005769535   0.049191096      -0.005800041
## PSC.CO2           -0.088759922  -0.016971816  -0.044820225       0.082772777
## Mnf.Flow          -0.457218313  -0.550066399  -0.611580420       0.482530739
## Carb.Pressure1    -0.112108310  -0.278795209  -0.432671459       0.234201877
## Fill.Pressure     -0.313117513  -0.247695921  -0.426092896       0.815285564
## Hyd.Pressure1     -0.048631533  -0.118317983  -0.029844050       0.190079720
## Hyd.Pressure2     -0.215717133  -0.294383550  -0.414577866       0.341831797
## Hyd.Pressure3     -0.260535183  -0.363660021  -0.529215855       0.459918732
## Hyd.Pressure4     -0.184518087   0.009692254  -0.117019292       0.288770453
## Filler.Level       0.348795399   0.225405359   0.977241656      -0.445952917
## Filler.Speed      -0.047596639  -0.044126285   0.027688288      -0.068525700
## Temperature       -0.181529097   0.093383081   0.050109366       0.059041937
## Usage.cont        -0.357704238  -0.316495941  -0.358456327       0.262673375
## Carb.Flow          0.240757744   0.385692082   0.012221618      -0.166362628
## Density            0.095340535  -0.046621720   0.019792536      -0.250884909
## MFR               -0.052377217  -0.048876320   0.042312091      -0.064390559
## Balling            0.072297357  -0.113757056   0.027415802      -0.210098742
## Pressure.Vacuum    0.213173584   0.278478283   0.358696671      -0.298520301
## PH                 1.000000000   0.162016479   0.358504933      -0.316950863
## Oxygen.Filler      0.162016479   1.000000000   0.224734144      -0.241221158
## Bowl.Setpoint      0.358504933   0.224734144   1.000000000      -0.440555036
## Pressure.Setpoint -0.316950863  -0.241221158  -0.440555036       1.000000000
## Air.Pressurer     -0.008276544   0.116495267  -0.130939728       0.073805550
## Alch.Rel           0.156053375  -0.050111016   0.063332637      -0.268800293
## Carb.Rel           0.189575611  -0.039206163   0.152117160      -0.261257572
## Balling.Lvl        0.103279163  -0.074544984   0.070442468      -0.242343282
##                   Air.Pressurer     Alch.Rel     Carb.Rel  Balling.Lvl
## Carb.Volume        -0.024854172  0.824787575  0.841072250  0.817993347
## Fill.Ounces         0.058101103 -0.110432476 -0.118666586 -0.063820071
## PC.Volume          -0.036605483 -0.177865067 -0.167433257 -0.210081156
## Carb.Pressure       0.021923761  0.459038047  0.473622472  0.458946620
```

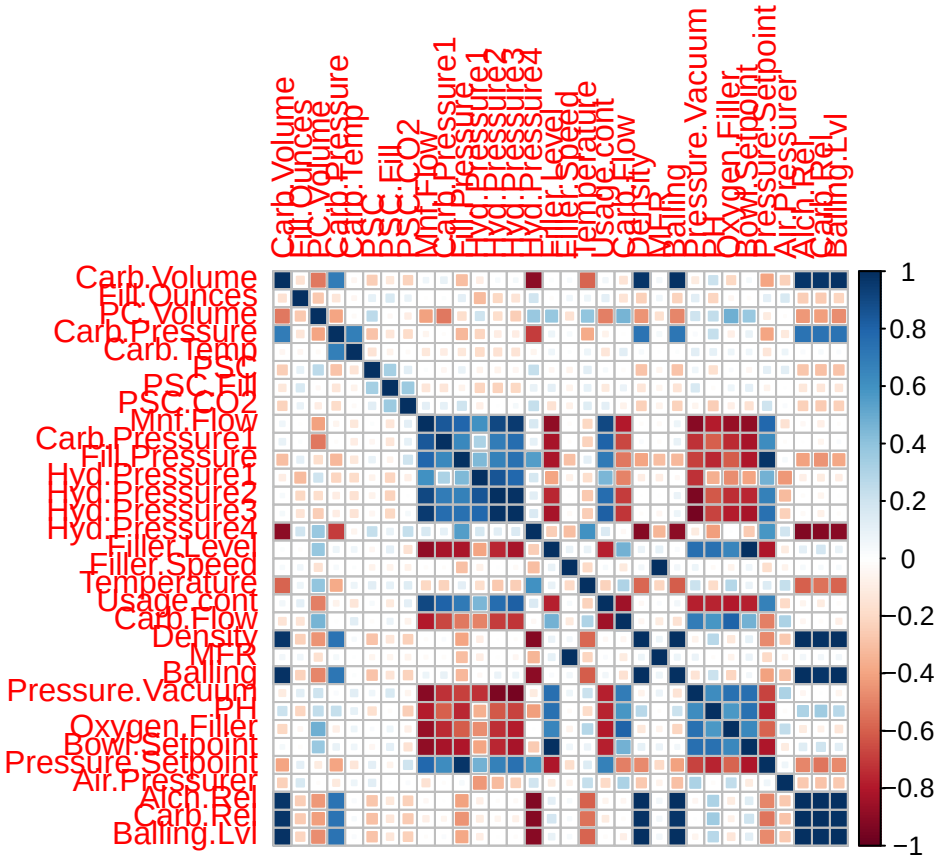
```
## Carb.Temp           0.038177076  0.004966509  0.009020802  0.010458674
## PSC                 0.061433424 -0.060809035 -0.065354969 -0.061576860
## PSC.Fill           -0.017995701 -0.017045886 -0.015365172 -0.016064731
## PSC.CO2            -0.006984004 -0.050859393 -0.066475871 -0.047296997
## Mnf.Flow           -0.041525175  0.027999719 -0.027880871  0.042267589
## Carb.Pressure1      0.061506659  0.009047055  0.004804776  0.033601804
## Fill.Pressure       0.032809452 -0.204670077 -0.202722006 -0.178857893
## Hyd.Pressure1      -0.209227461  0.006667887  0.034387945 -0.007754220
## Hyd.Pressure2      -0.172375910  0.039166055  0.031505491  0.028859401
## Hyd.Pressure3      -0.067700529  0.049948952  0.013543248  0.041595480
## Hyd.Pressure4       0.021731381 -0.685576013 -0.581827500 -0.575044299
## Filler.Level       -0.125734192  0.043558564  0.131078561  0.053179225
## Filler.Speed       -0.006199674 -0.005598672  0.005300760  0.003823258
## Temperature         0.066928829 -0.247762315 -0.135951942 -0.224975043
## Usage.cont         -0.102505626 -0.023740808 -0.039079876  0.023206997
## Carb.Flow           0.137849013  0.007527013 -0.006940815 -0.055365859
## Density            -0.082976446  0.922993173  0.860322233  0.956417297
## MFR                -0.008182360 -0.001003949  0.010884909  0.016039036
## Balling            -0.102307854  0.945725964  0.858990647  0.988412082
## Pressure.Vacuum     0.175090738 -0.054788103 -0.008434181 -0.051626926
## PH                 -0.008276544  0.156053375  0.189575611  0.103279163
## Oxygen.Filler       0.116495267 -0.050111016 -0.039206163 -0.074544984
## Bowl.Setpoint      -0.130939728  0.063332637  0.152117160  0.070442468
## Pressure.Setpoint   0.073805550 -0.268800293 -0.261257572 -0.242343282
## Air.Pressurer       1.000000000 -0.082451451 -0.107386102 -0.085434755
## Alch.Rel           -0.082451451  1.000000000  0.882791450  0.947343341
## Carb.Rel           -0.107386102  0.882791450  1.000000000  0.872148303
## Balling.Lvl        -0.085434755  0.947343341  0.872148303  1.000000000
```

There are many highly correlated variables. A quick scan shows:

- `hyd.pressure` 2 and 3 have a .91 correlation
- `filler.level` and bowl.set.point have a .98 correlation
- `fillers.speed` and MFR have a .95 correlation
- `Density` and `Balling.Lvl` have a .95 correlation
- `Balling` and `Balling.Lvl` have a .98 correlation; both are strongly correlated with `Density` and `Alch.Rel`
- `Alch.Rel` and `Balling.Lvl` have a .95 correlation; it also has a .95 correlation with `Balling`, `Balling.Lvl`, and .93 with `Density`

Here's a correlation plot so we can see all the positively and negatively correlated variables:

```
library(corrplot)

grid <- training_data %>%
  drop_na() %>%
  select(-Brand.Code) %>%
  cor()

corrplot(cor(grid),method = "square")
```



There are many negatively correlated variables here that do not need to be removed, but may be helpful for analysis. We can see that the response variable, pH, has a strong negative correlation with some variables, including `Fill.Pressure`, `Usage.cont`, and `Air.Pressurer`.

The `findCorrelation` feature will decide which variables to drop (it leaves the variable that best represents the group):

```
library(caret)

#if we remove colinear variables at 95% correlation, we would drop 10 variables
indexestodrop95 <- findCorrelation(cor(grid), cutoff = 0.95)
colnames(grid)[indexestodrop95]
```

```
##  [1] "Pressure.Setpoint" "Mnf.Flow"          "Hyd.Pressure3"
##  [4] "Bowl.Setpoint"     "Carb.Rel"          "Balling"
##  [7] "Alch.Rel"          "Balling.Lvl"       "Density"
## [10] "Filler.Speed"
```

```
#this will leave us with 22 predictor variables and the response variable.

#if we remove colinear variables at 90% correlation, we would drop 12 variables
indexestodrop90 <- findCorrelation(cor(grid), cutoff = 0.90)
colnames(grid)[indexestodrop90]
```

```
##  [1] "Pressure.Setpoint" "Mnf.Flow"          "Hyd.Pressure3"
```

```
##  [4] "Pressure.Vacuum"   "Hyd.Pressure4"     "Bowl.Setpoint"
##  [7] "Carb.Rel"          "Balling"           "Alch.Rel"
## [10] "Balling.Lvl"       "Density"           "Filler.Speed"
```

```
#this will leave us with 20 predictor variables and the response variable.
```

1.3.3 Removing variables

Removing some of the 33 variables will help simplify this data set. We will remove the variable with NZV and highly colinear variables (greater than or about .90, as defined above).

the `findcorrelation` function says to drop `Filler.Speed` instead of `MFR`. However, we will opt to drop `MFR` instead, given that it has the highest number of missing values (`Filler.Speed` has 57 as opposed to `MFR`'s 212). Having to impute all these values will makes the variable less reliable, and dropping it would mean having to drop more cases.

```
#removing the variable with NZV
training_data_new <- training_data %>%
  select(-Hyd.Pressure1)

#removing correlated variables
training_data_new <- training_data_new %>%
  select(-Pressure.Setpoint, -MFR, -Hyd.Pressure3,
         -Pressure.Vacuum, -Hyd.Pressure4, -Bowl.Setpoint, -Carb.Rel,
         -Balling, -Alch.Rel, - Balling.Lvl, - Density)

#donig the same for the test set
test_data_new <- test_data %>%
  select(-Hyd.Pressure1, -Pressure.Setpoint, -MFR, -Hyd.Pressure3,
         -Pressure.Vacuum, -Hyd.Pressure4, -Bowl.Setpoint, -Carb.Rel,
         -Balling, -Alch.Rel, - Balling.Lvl, - Density)
```

1.3.4 Checking for outliers

Before we deal with missing values, let's see if there are any glaring outliers or data entry errors.

```
#may be an outlier in oxygen filler - the max calues is much higher than the

summary(test_data_new)
```

```
##   Brand.Code          Carb.Volume     Fill.Ounces      PC.Volume
##  Length:267         Min.   :5.147   Min.   :23.75   Min.   :0.09867
##  Class :character   1st Qu.:5.287   1st Qu.:23.92   1st Qu.:0.23333
##  Mode  :character   Median :5.340   Median :23.97   Median :0.27533
##                     Mean   :5.369   Mean   :23.97   Mean   :0.27769
##                     3rd Qu.:5.465   3rd Qu.:24.01   3rd Qu.:0.32200
##                     Max.   :5.667   Max.   :24.20   Max.   :0.46400
##                     NA's   :1       NA's   :6       NA's   :4
##  Carb.Pressure     Carb.Temp          PSC             PSC.Fill
##  Min.   :60.20   Min.   :130.0   Min.   :0.00400   Min.   :0.0200
##  1st Qu.:65.30   1st Qu.:138.4   1st Qu.:0.04450   1st Qu.:0.1000
##  Median :68.00   Median :140.8   Median :0.07600   Median :0.1800
```

```
##  Mean   :68.25   Mean   :141.2   Mean   :0.08545   Mean   :0.1903
##  3rd Qu.:70.60   3rd Qu.:143.8   3rd Qu.:0.11200   3rd Qu.:0.2600
##  Max.   :77.60   Max.   :154.0   Max.   :0.24600   Max.   :0.6200
##                  NA's   :1       NA's   :5         NA's   :3
##     PSC.CO2           Mnf.Flow       Carb.Pressure1  Fill.Pressure
##  Min.   :0.00000   Min.   :-100.20   Min.   :113.0   Min.   :37.80
##  1st Qu.:0.02000   1st Qu.:-100.00   1st Qu.:120.2   1st Qu.:46.00
##  Median :0.04000   Median :   0.20   Median :123.4   Median :47.80
##  Mean   :0.05107   Mean   :  21.03   Mean   :123.0   Mean   :48.14
##  3rd Qu.:0.06000   3rd Qu.: 141.30   3rd Qu.:125.5   3rd Qu.:50.20
##  Max.   :0.24000   Max.   : 220.40   Max.   :136.0   Max.   :60.20
##  NA's   :5                           NA's   :4       NA's   :2
##  Hyd.Pressure2     Filler.Level    Filler.Speed   Temperature
##  Min.   :-50.00   Min.   : 69.2   Min.   :1006   Min.   :63.80
##  1st Qu.:  0.00   1st Qu.:100.6   1st Qu.:3812   1st Qu.:65.40
##  Median : 26.80   Median :118.6   Median :3978   Median :65.80
##  Mean   : 20.11   Mean   :110.3   Mean   :3581   Mean   :66.23
##  3rd Qu.: 34.80   3rd Qu.:120.2   3rd Qu.:3996   3rd Qu.:66.60
##  Max.   : 61.40   Max.   :153.2   Max.   :4020   Max.   :75.40
##  NA's   :1        NA's   :2       NA's   :10     NA's   :2
##    Usage.cont      Carb.Flow       PH          Oxygen.Filler
##  Min.   :12.90   Min.   :   0   Mode:logical   Min.   :0.00240
##  1st Qu.:18.12   1st Qu.:1083   NA's:267       1st Qu.:0.01960
##  Median :21.44   Median :3038                  Median :0.03370
##  Mean   :20.90   Mean   :2409                  Mean   :0.04666
##  3rd Qu.:23.74   3rd Qu.:3215                  3rd Qu.:0.05440
##  Max.   :24.60   Max.   :3858                  Max.   :0.39800
##  NA's   :2                                     NA's   :3
##  Air.Pressurer
##  Min.   :141.2
##  1st Qu.:142.2
##  Median :142.6
##  Mean   :142.8
##  3rd Qu.:142.8
##  Max.   :147.2
##  NA's   :1
```

```
#there's also at least one really large value on oxygen filler here
summary(training_data_new)
```

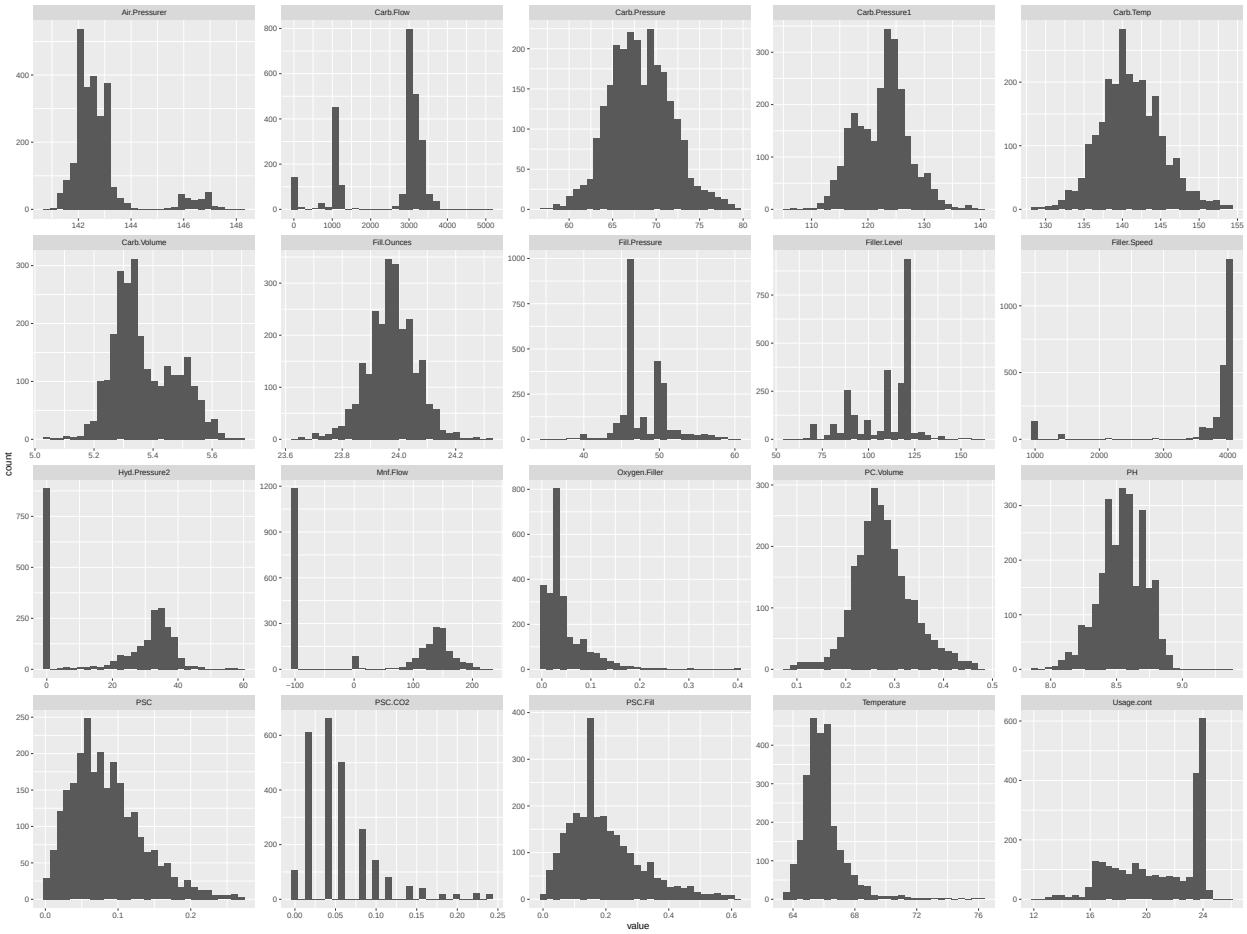
```
##   Brand.Code          Carb.Volume     Fill.Ounces      PC.Volume
##  Length:2571        Min.   :5.040   Min.   :23.63   Min.   :0.07933
##  Class :character   1st Qu.:5.293   1st Qu.:23.92   1st Qu.:0.23917
##  Mode  :character   Median :5.347   Median :23.97   Median :0.27133
##                     Mean   :5.370   Mean   :23.97   Mean   :0.27712
##                     3rd Qu.:5.453   3rd Qu.:24.03   3rd Qu.:0.31200
##                     Max.   :5.700   Max.   :24.32   Max.   :0.47800
##                     NA's   :10      NA's   :38      NA's   :39
##  Carb.Pressure     Carb.Temp          PSC            PSC.Fill
##  Min.   :57.00   Min.   :128.6   Min.   :0.00200   Min.   :0.0000
##  1st Qu.:65.60   1st Qu.:138.4   1st Qu.:0.04800   1st Qu.:0.1000
##  Median :68.20   Median :140.8   Median :0.07600   Median :0.1800
##  Mean   :68.19   Mean   :141.1   Mean   :0.08457   Mean   :0.1954
##  3rd Qu.:70.60   3rd Qu.:143.8   3rd Qu.:0.11200   3rd Qu.:0.2600
```

```
##  Max.   :79.40   Max.   :154.0   Max.   :0.27000   Max.   :0.6200
##  NA's   :27      NA's   :26      NA's   :33        NA's   :23
##     PSC.CO2           Mnf.Flow       Carb.Pressure1  Fill.Pressure
##  Min.   :0.00000   Min.   :-100.20   Min.   :105.6   Min.   :34.60
##  1st Qu.:0.02000   1st Qu.:-100.00   1st Qu.:119.0   1st Qu.:46.00
##  Median :0.04000   Median :  65.20   Median :123.2   Median :46.40
##  Mean   :0.05641   Mean   :  24.57   Mean   :122.6   Mean   :47.92
##  3rd Qu.:0.08000   3rd Qu.: 140.80   3rd Qu.:125.4   3rd Qu.:50.00
##  Max.   :0.24000   Max.   : 229.40   Max.   :140.2   Max.   :60.40
##  NA's   :39        NA's   :2         NA's   :32      NA's   :22
##  Hyd.Pressure2    Filler.Level    Filler.Speed   Temperature      Usage.cont
##  Min.   : 0.00   Min.   : 55.8   Min.   : 998   Min.   :63.60   Min.   :12.08
##  1st Qu.: 0.00   1st Qu.: 98.3   1st Qu.:3888   1st Qu.:65.20   1st Qu.:18.36
##  Median :28.60   Median :118.4   Median :3982   Median :65.60   Median :21.79
##  Mean   :20.96   Mean   :109.3   Mean   :3687   Mean   :65.97   Mean   :20.99
##  3rd Qu.:34.60   3rd Qu.:120.0   3rd Qu.:3998   3rd Qu.:66.40   3rd Qu.:23.75
##  Max.   :59.40   Max.   :161.2   Max.   :4030   Max.   :76.20   Max.   :25.90
##  NA's   :15      NA's   :20      NA's   :57     NA's   :14      NA's   :5
##    Carb.Flow          PH        Oxygen.Filler     Air.Pressurer
##  Min.   :  26   Min.   :7.880   Min.   :0.00240   Min.   :140.8
##  1st Qu.:1144   1st Qu.:8.440   1st Qu.:0.02200   1st Qu.:142.2
##  Median :3028   Median :8.540   Median :0.03340   Median :142.6
##  Mean   :2468   Mean   :8.546   Mean   :0.04684   Mean   :142.8
##  3rd Qu.:3186   3rd Qu.:8.680   3rd Qu.:0.06000   3rd Qu.:143.0
##  Max.   :5104   Max.   :9.360   Max.   :0.40000   Max.   :148.2
##  NA's   :2      NA's   :4       NA's   :12
```

After quickly browsing the remaining variables, while some of the fields have a wide range of values, none of the values look like errors (there are clusters of similar low/high values for each variable). While they may count as outliers (we'll calculate later) they are likely legitimate data ant not input errors.

Let's check histograms to see if anything stands out:

```
## `stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



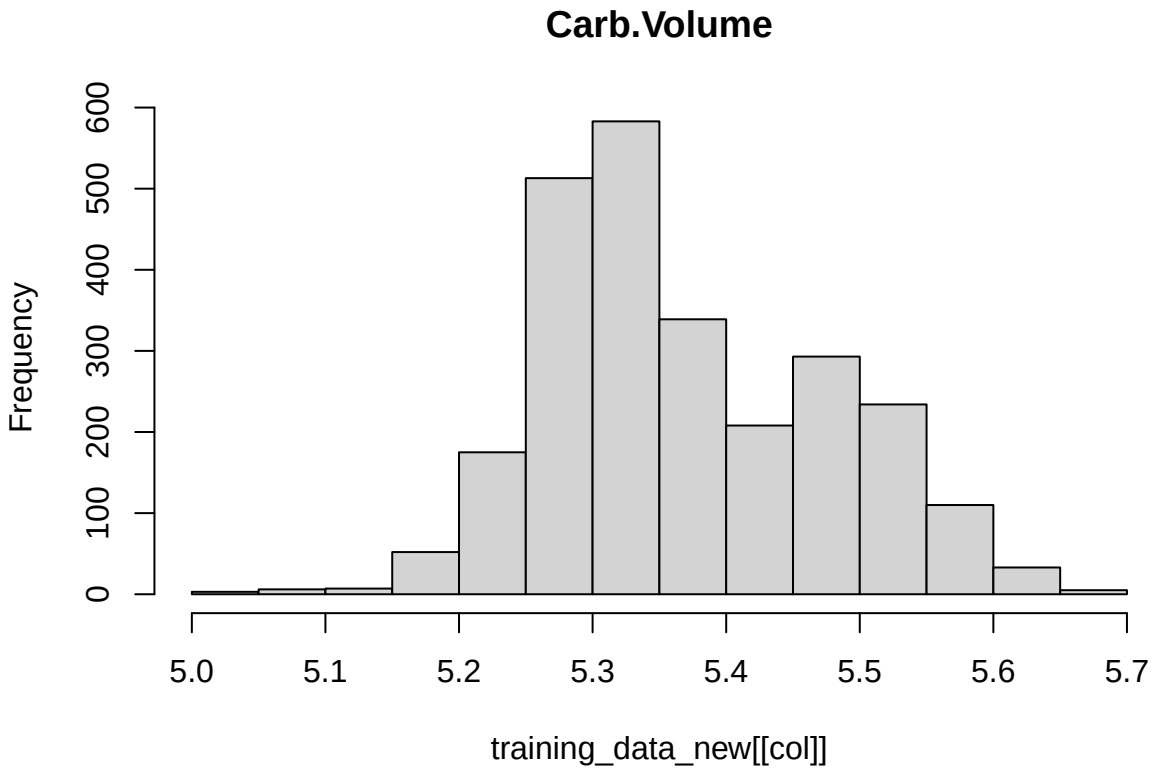

```
##   Brand.Code    n
## 1          A  293
## 2          B 1239
## 3          C  304
## 4          D  615
## 5       <NA>  120
```

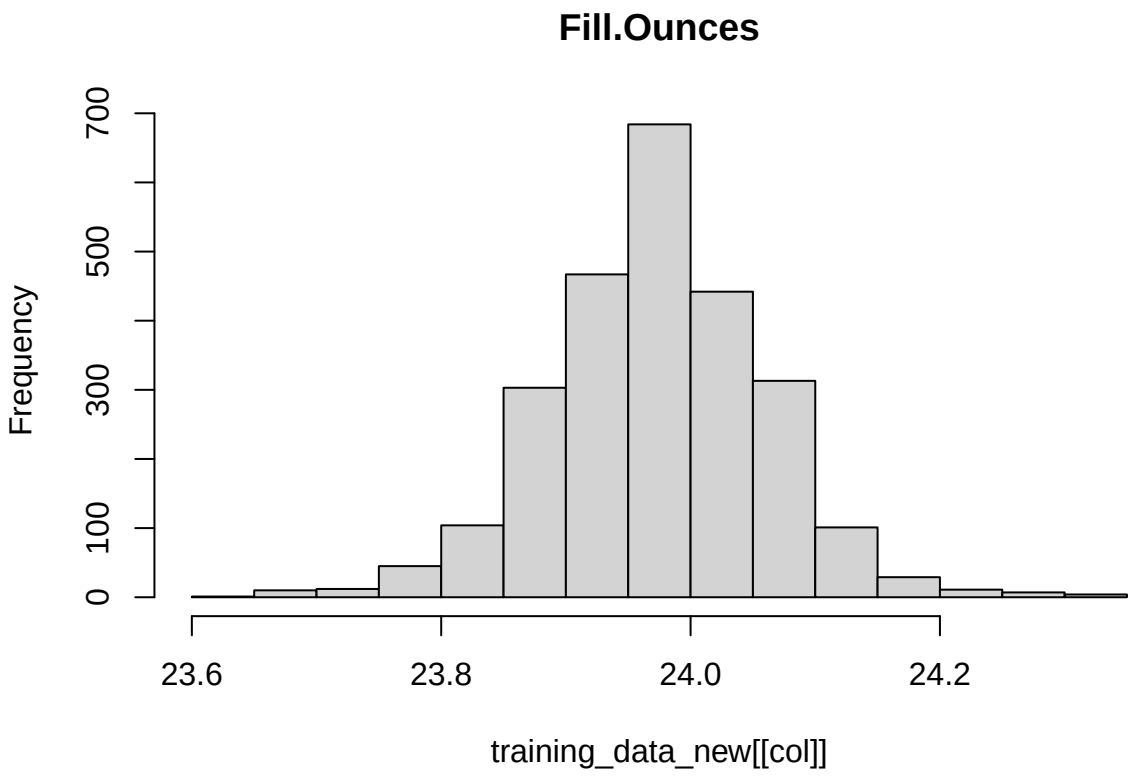
Not all the data is normally distributed. pH, the response variable is normally distributed. Other variables close to a normal distribution include `Carb.Pressure`, `Carb.Pressure1` (slightly bimodal), `Carb.Temp`, `Fill.Ounces`, `Carb.Volume` (also slightly bimodal), `PSC` (skewed slightly right), `PC.Volume`, `Temperature` (skewed slightly right), and `PSC.Fill` (skewed right with a large peak).

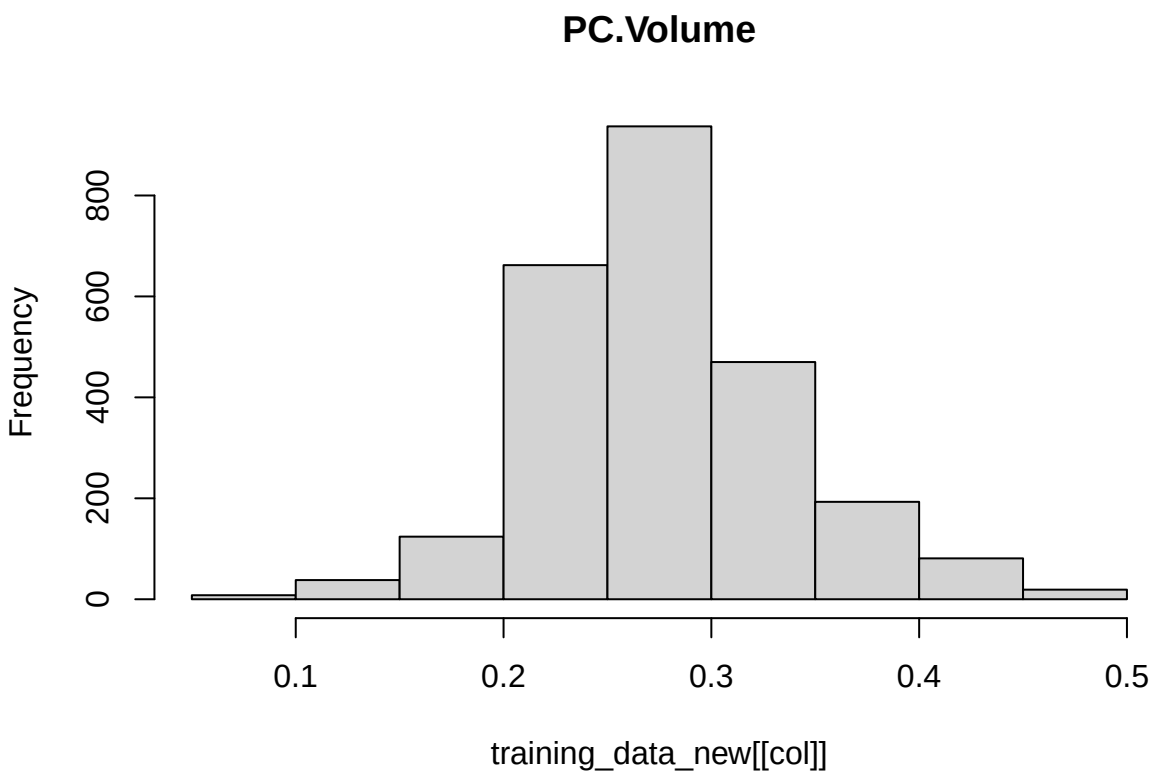
A few are more clearly bimodal/have very different peaks on either side of the graph: `Air.Pressurer`, `Carb.Flow`, `Fill.Pressure`, `Filler.Level`, `Filler.Speed`, `Hyd.Pressure2`, `Mnf.Flow`, and `Usage.cont`.

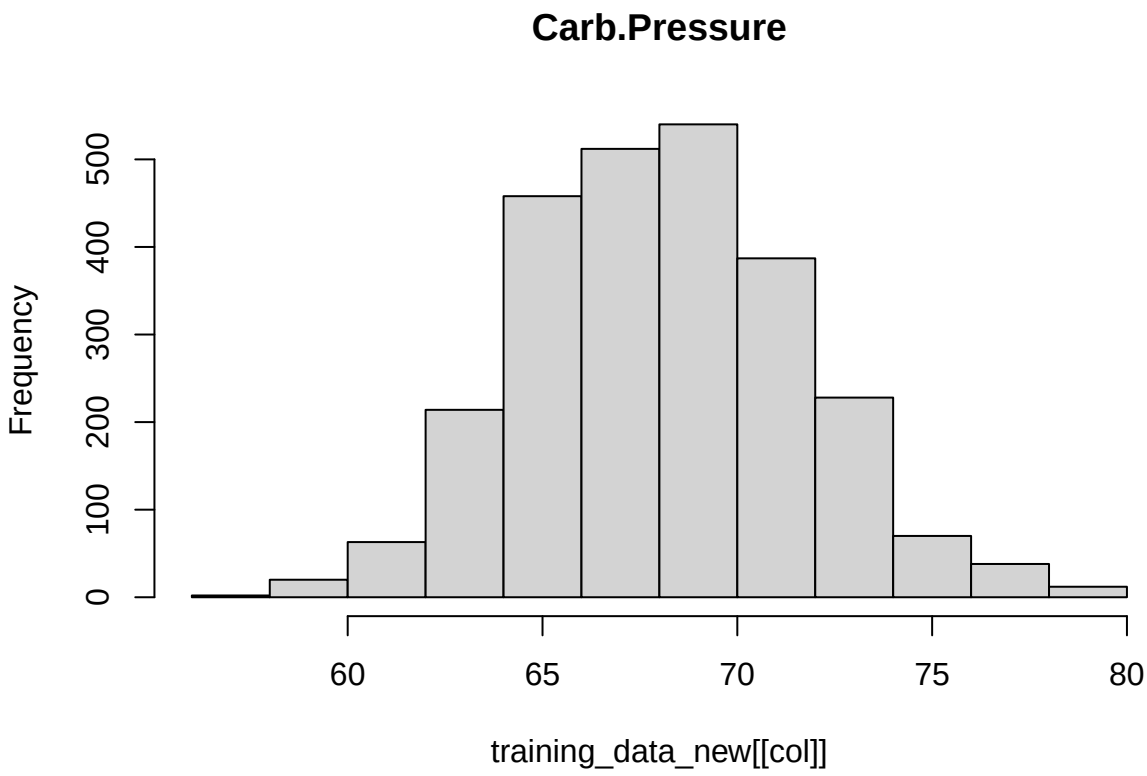
Finally, some are skewed right: in addition to the varaibles mentioned above, `Oxygen.Filler` looks like it may have some outliers, and `PSC.CO2` has a strange distribution, where the variable clusters around certain values, almost like a discrete variable.

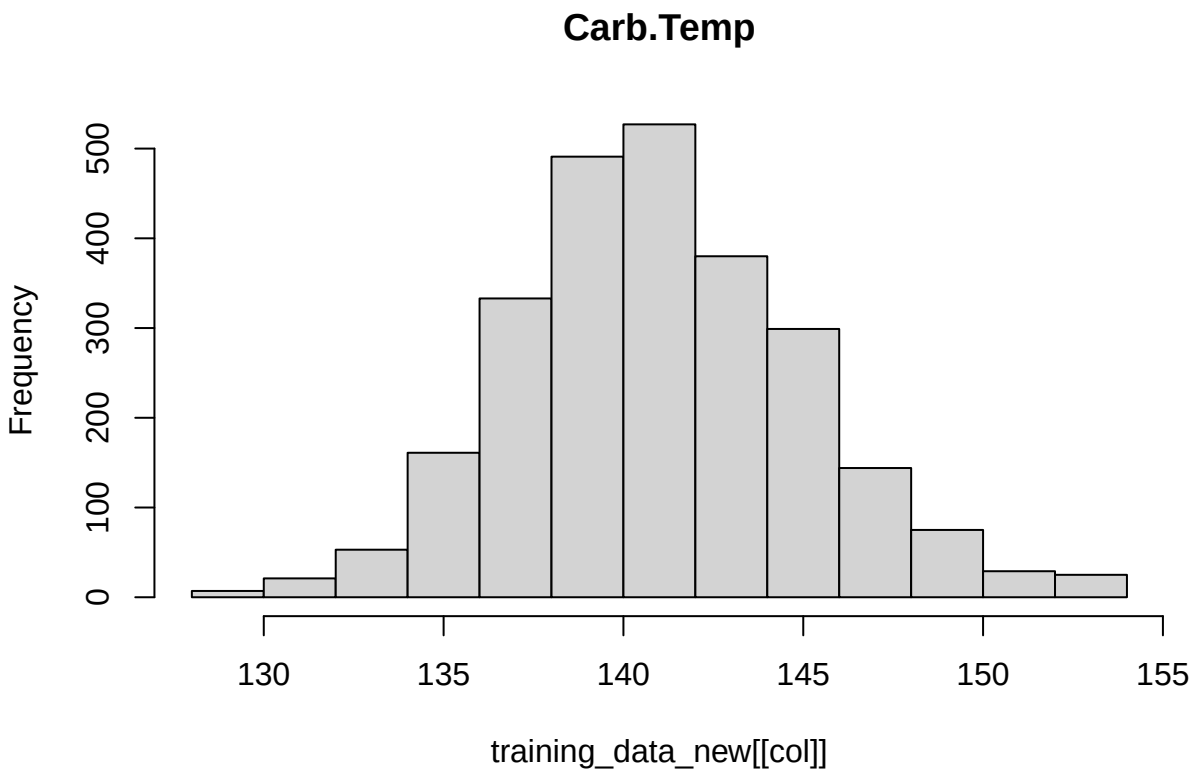
```
#histograms

for (col in names(training_data_new %>% select(-Brand.Code))) {
  hist(training_data_new[[col]], main=col)
}
```

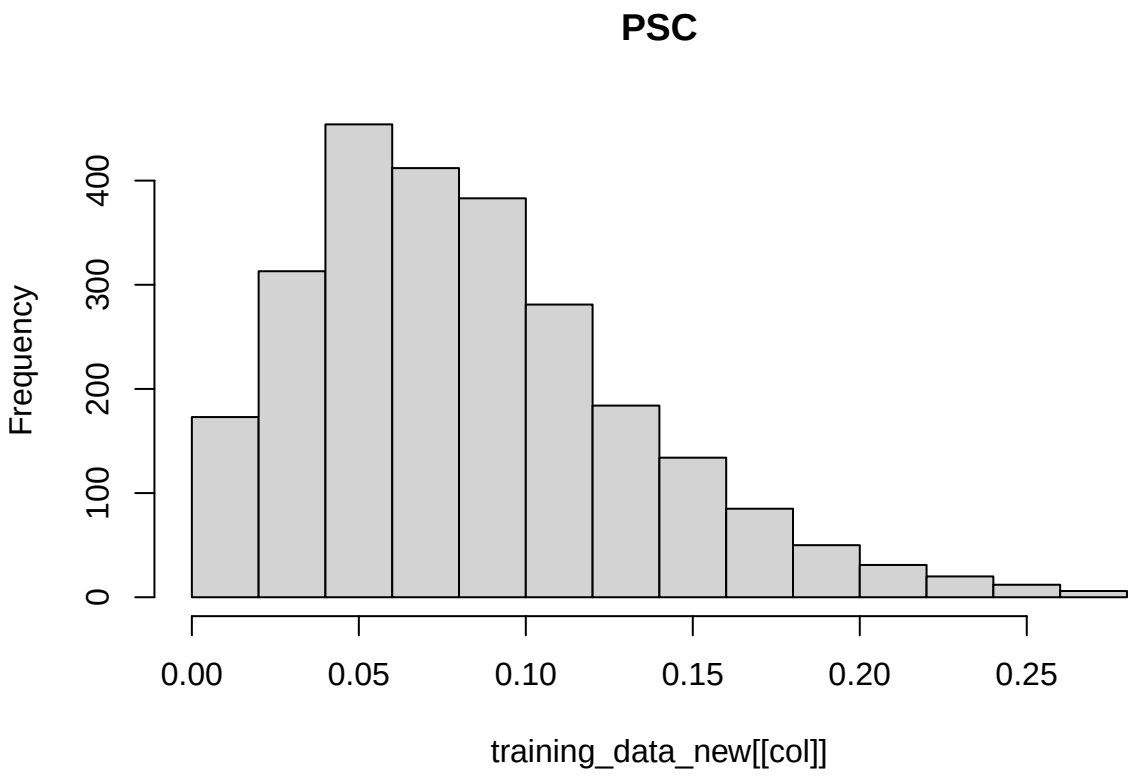

Carb.Volume
Frequency
0
100
200
300
400
500
600
5.0
5.1
5.2
5.3
5.4
5.5
5.6
5.7
training_data_new[[col]]

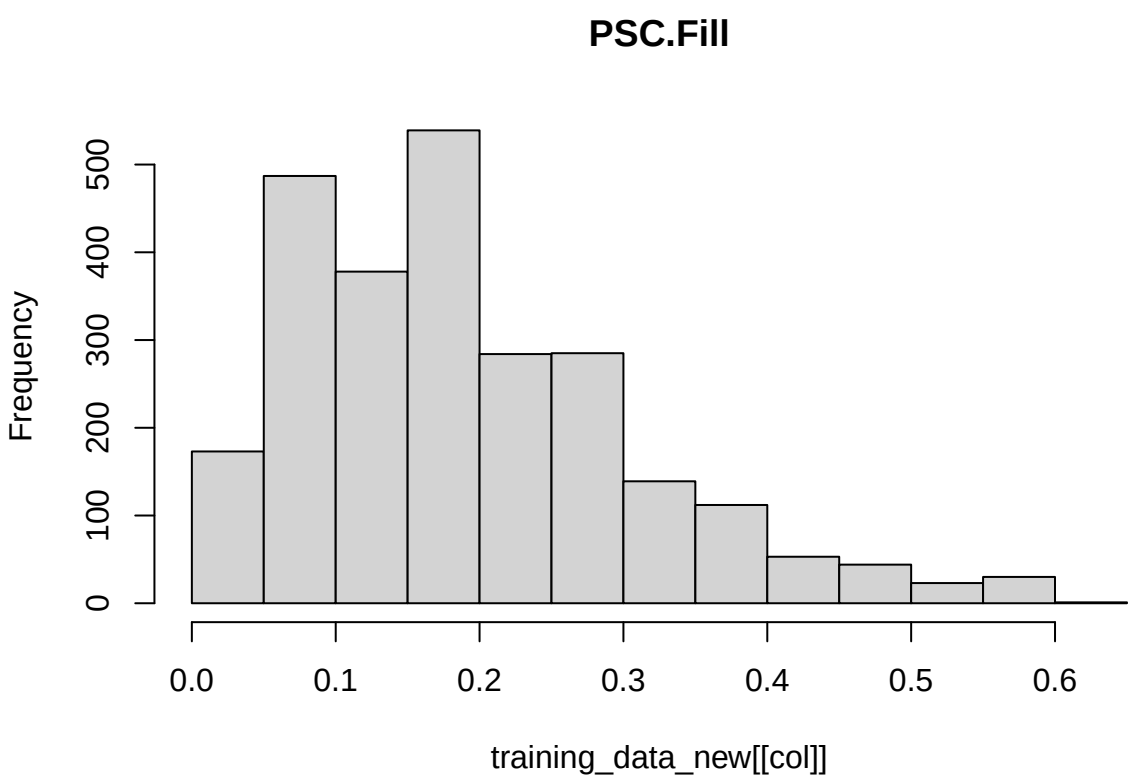

Fill.Ounces
Frequency
0
100
300
500
700
23.6
23.8
24.0
24.2
training_data_new[[col]]

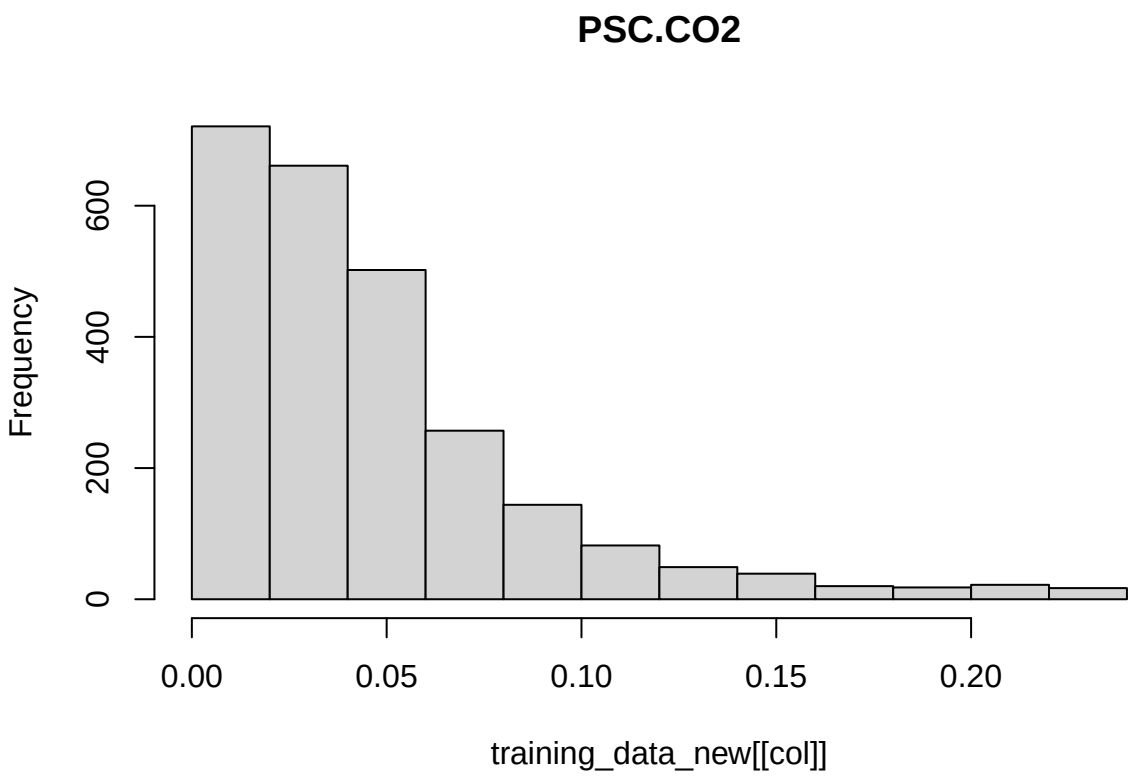

PC.Volume
Frequency
0
200
400
600
800
0.1
0.2
0.3
0.4
0.5
training_data_new[[col]]

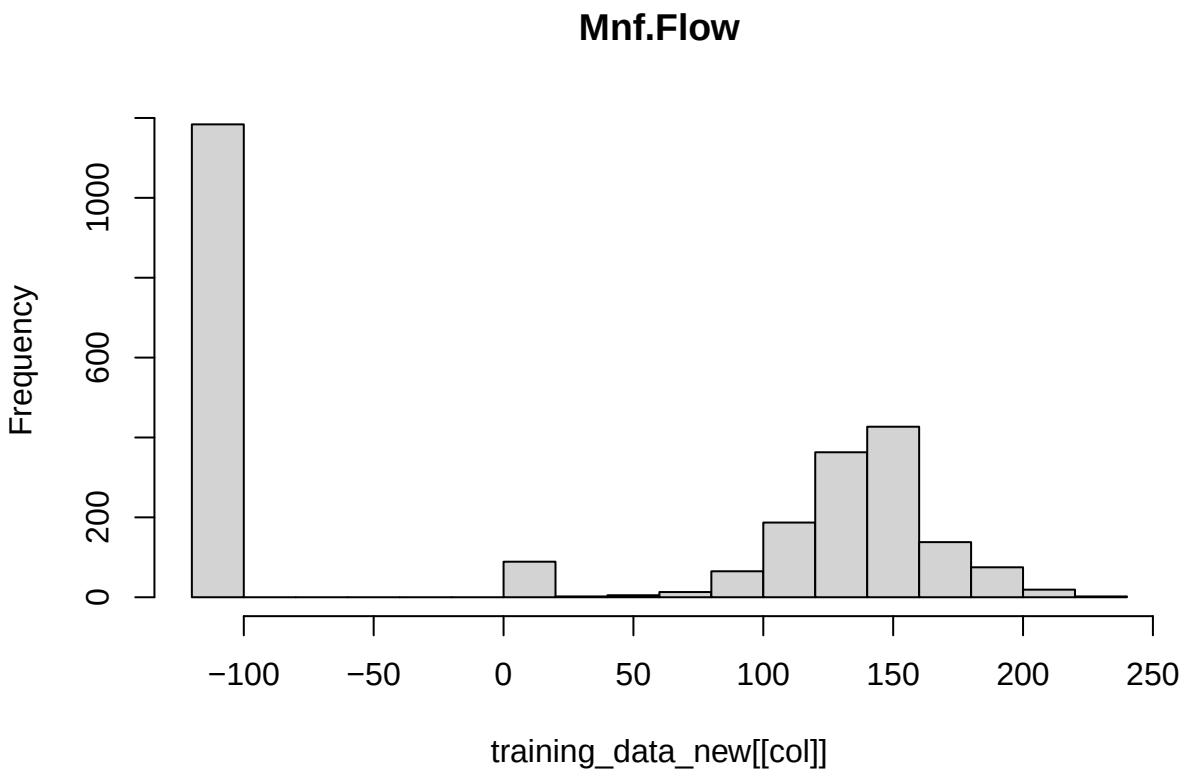

Carb.Pressure
Frequency
0
100
200
300
400
500
60
65
70
75
80
training_data_new[[col]]

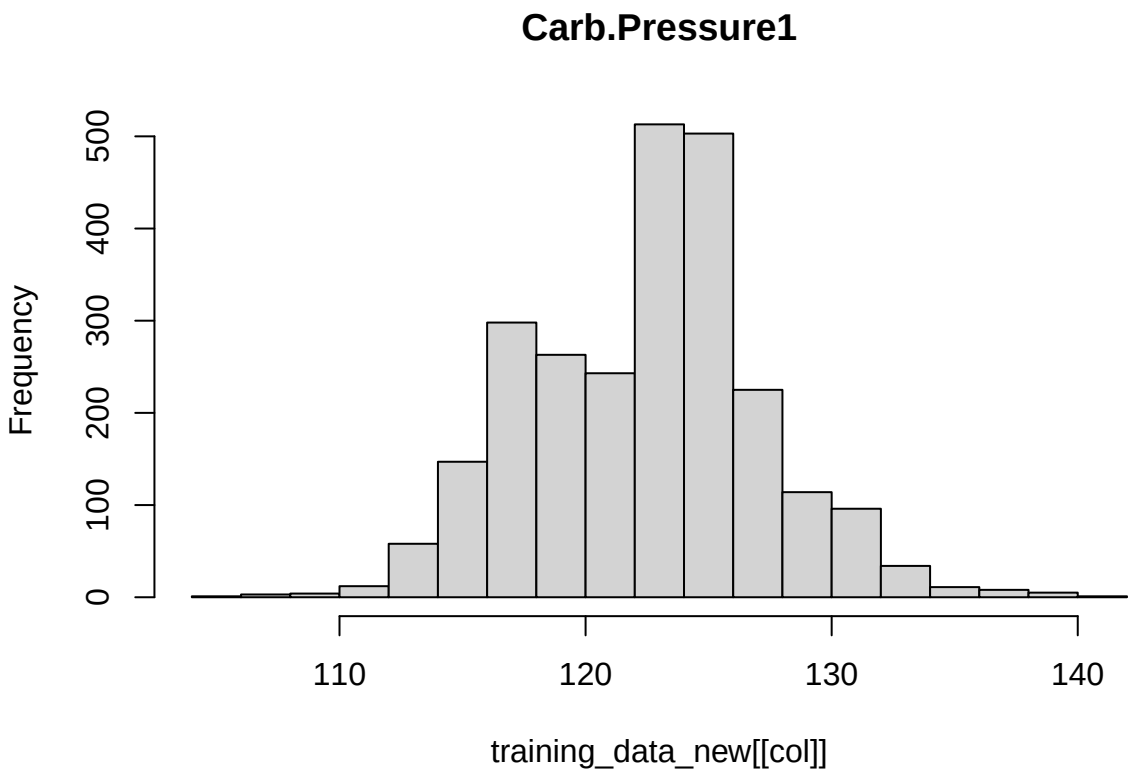

Carb.Temp
Frequency
0
100
200
300
400
500
130
135
140
145
150
155
training_data_new[[col]]

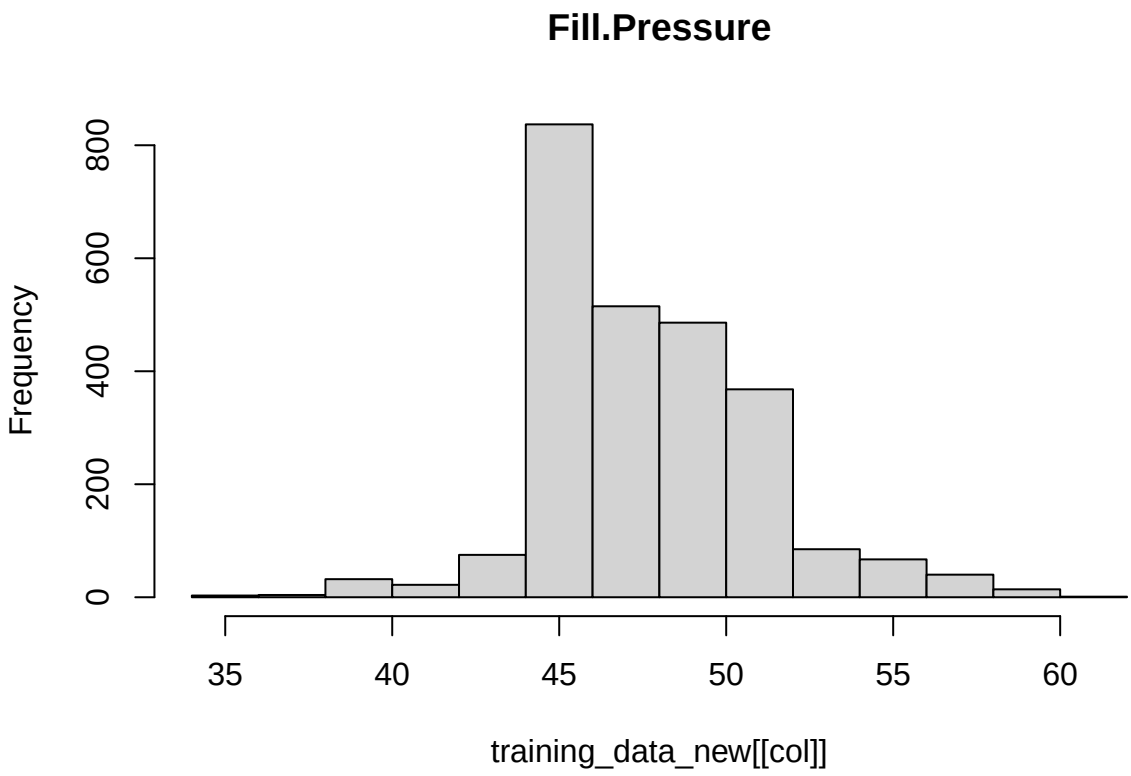

PSC
Frequency
0
100
200
300
400
0.00
0.05
0.10
0.15
0.20
0.25
training_data_new[[col]]

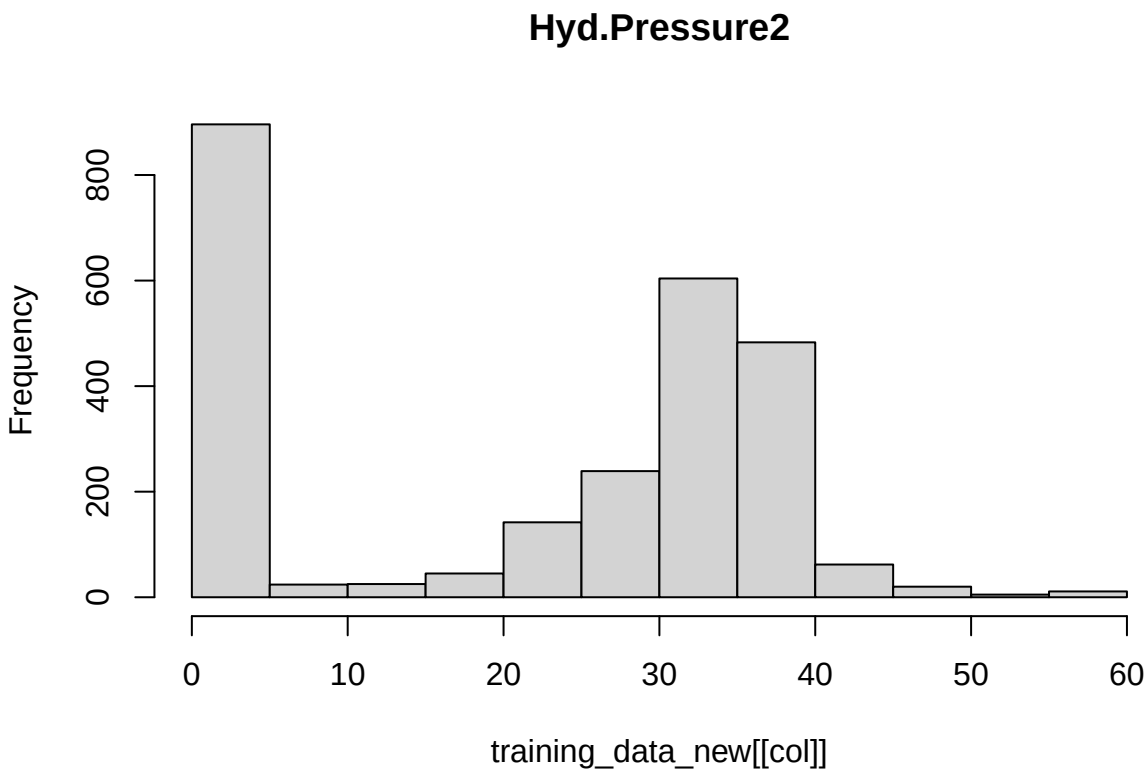

PSC.Fill
Frequency
0
100
200
300
400
500
0.0
0.1
0.2
0.3
0.4
0.5
0.6
training_data_new[[col]]

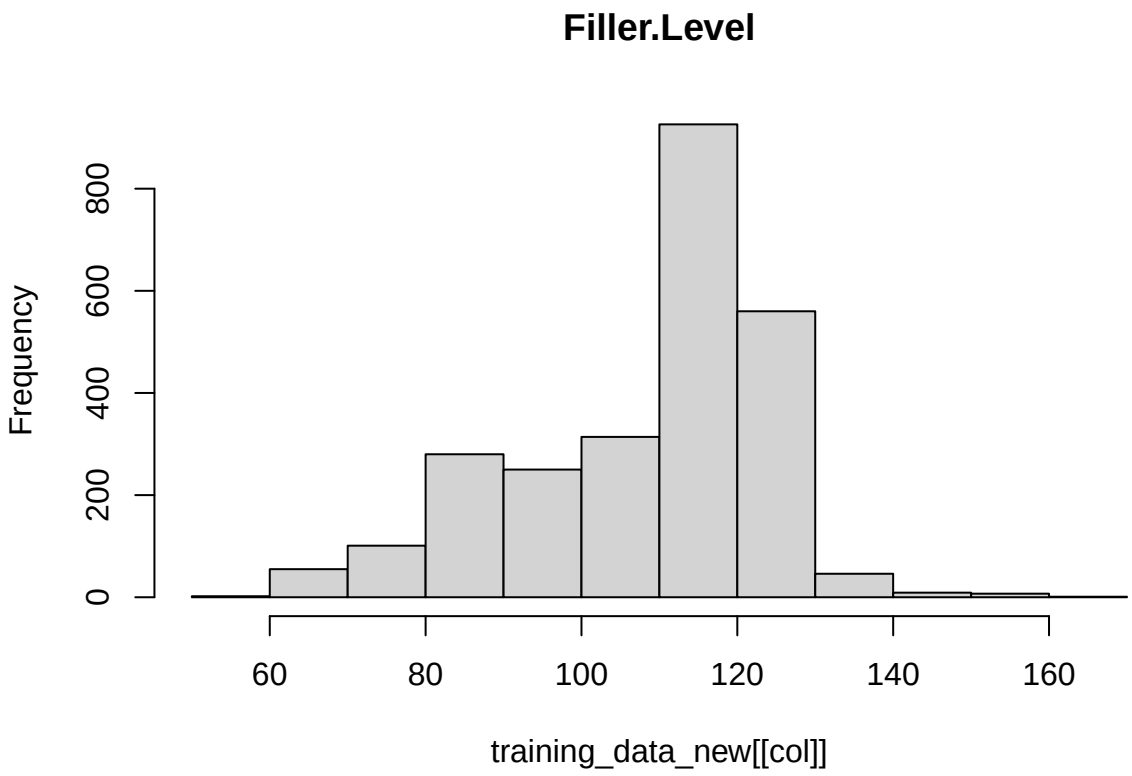

PSC.CO2
Frequency
0
200
400
600
0.00
0.05
0.10
0.15
0.20
training_data_new[[col]]

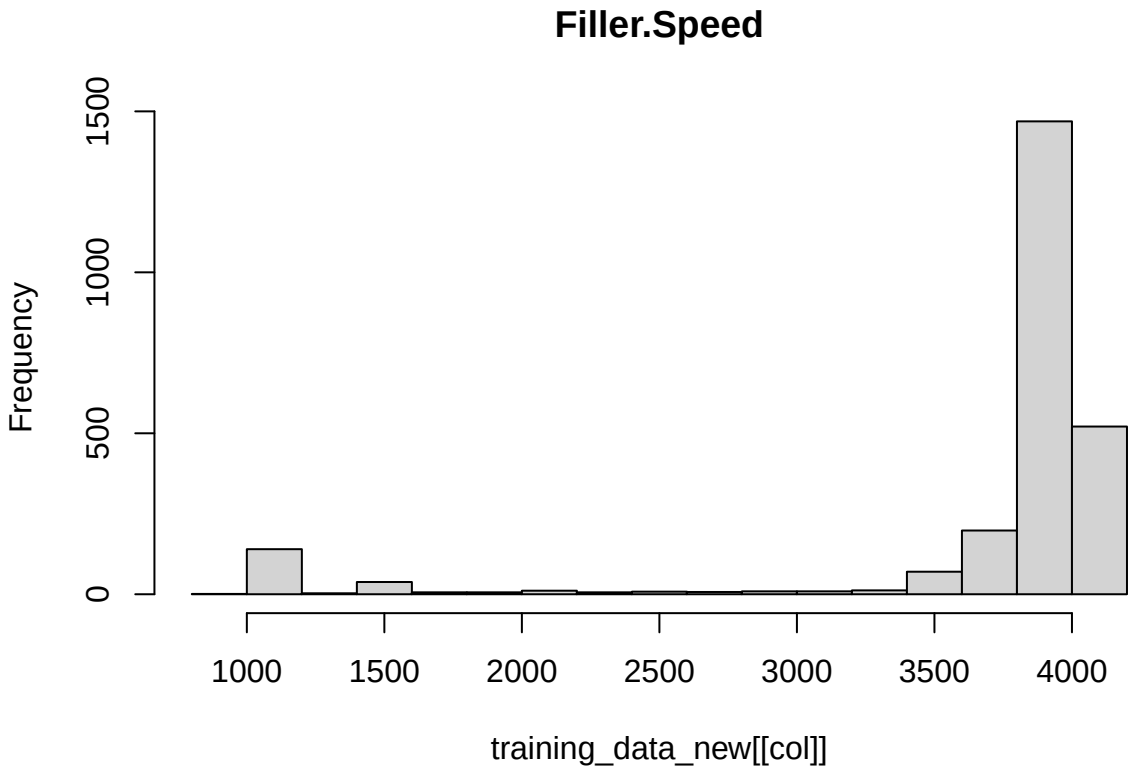

Mnf.Flow
Frequency
0
200
600
1000
−100
−50
0
50
100
150
200
250
training_data_new[[col]]

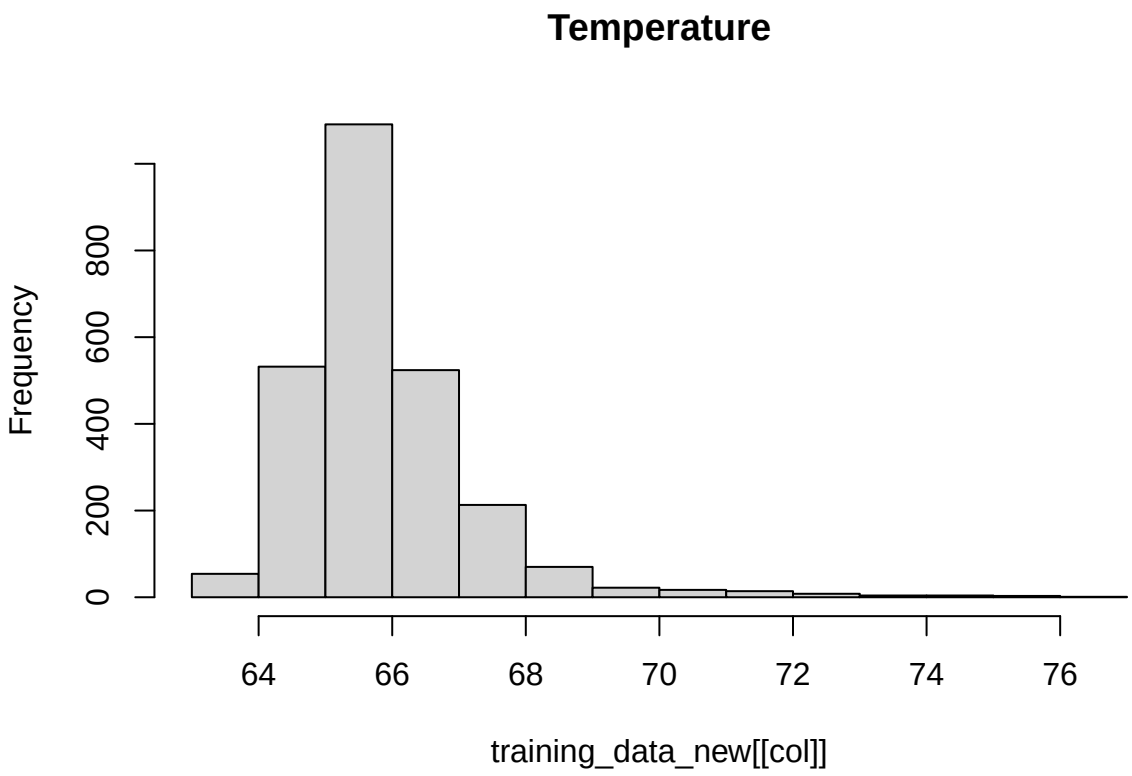

Carb.Pressure1
Frequency
0
100
200
300
400
500
110
120
130
140
training_data_new[[col]]

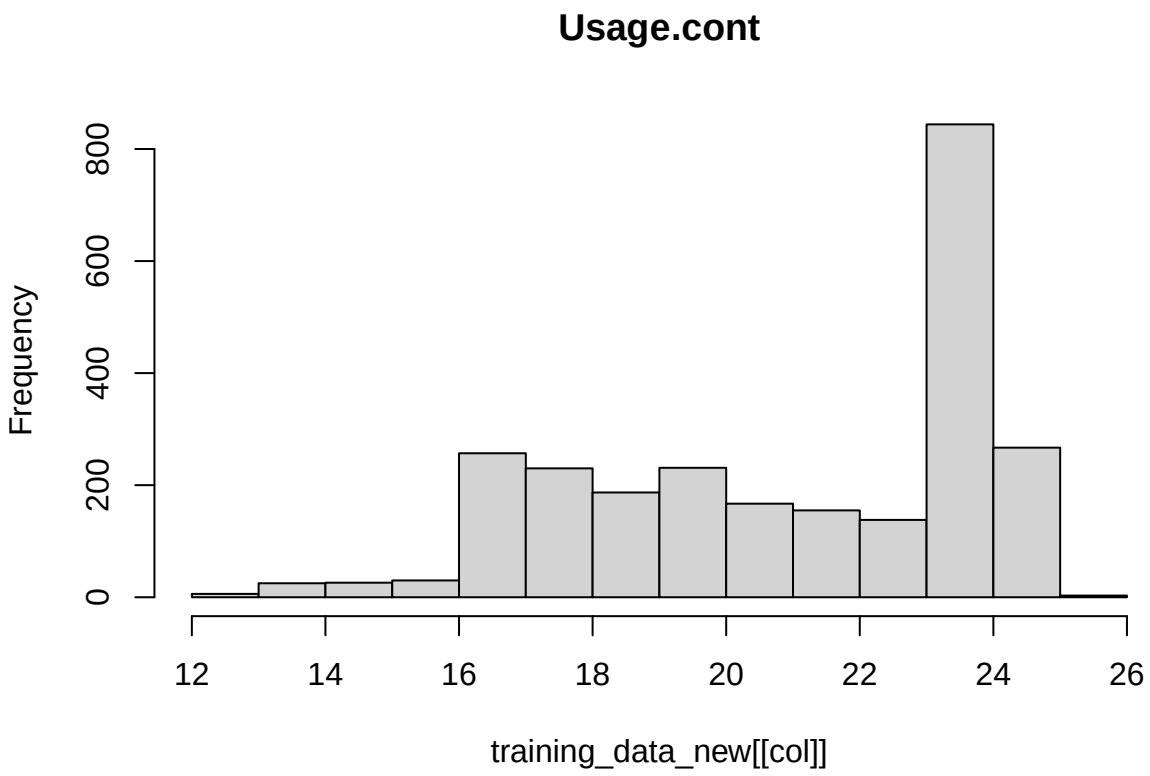

Fill.Pressure
Frequency
0
200
400
600
800
35
40
45
50
55
60
training_data_new[[col]]

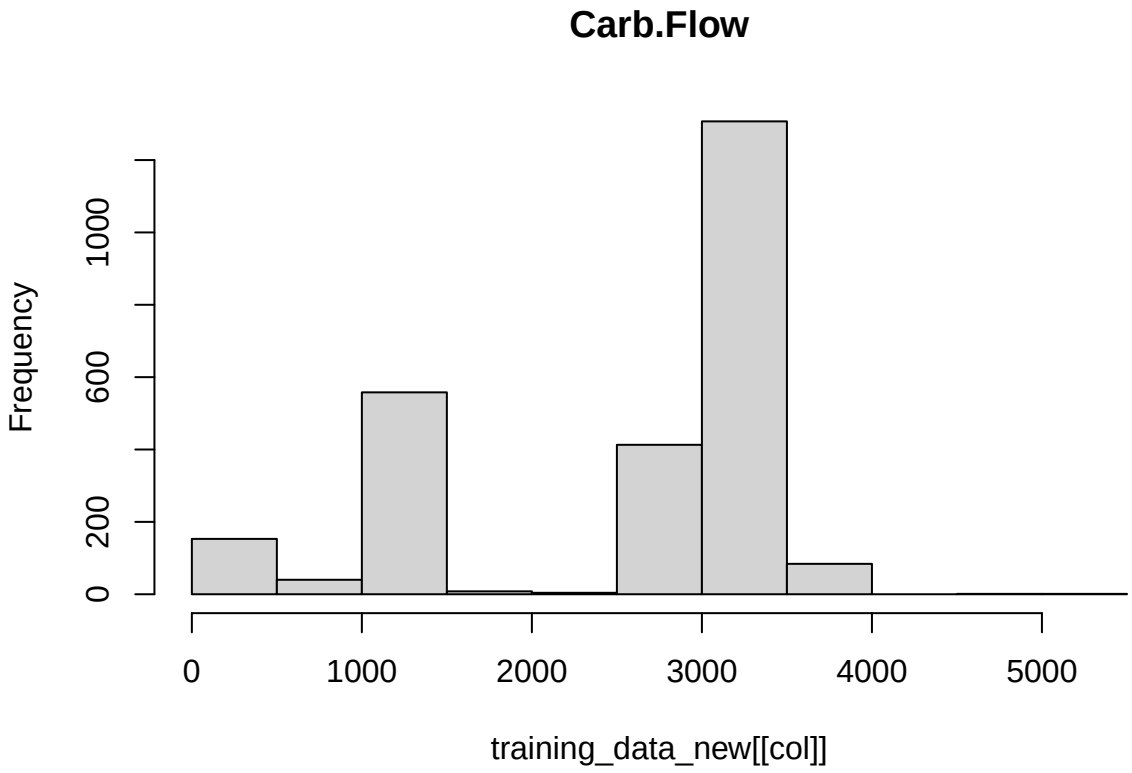

Hyd.Pressure2
Frequency
0
200
400
600
800
0
10
20
30
40
50
60
training_data_new[[col]]

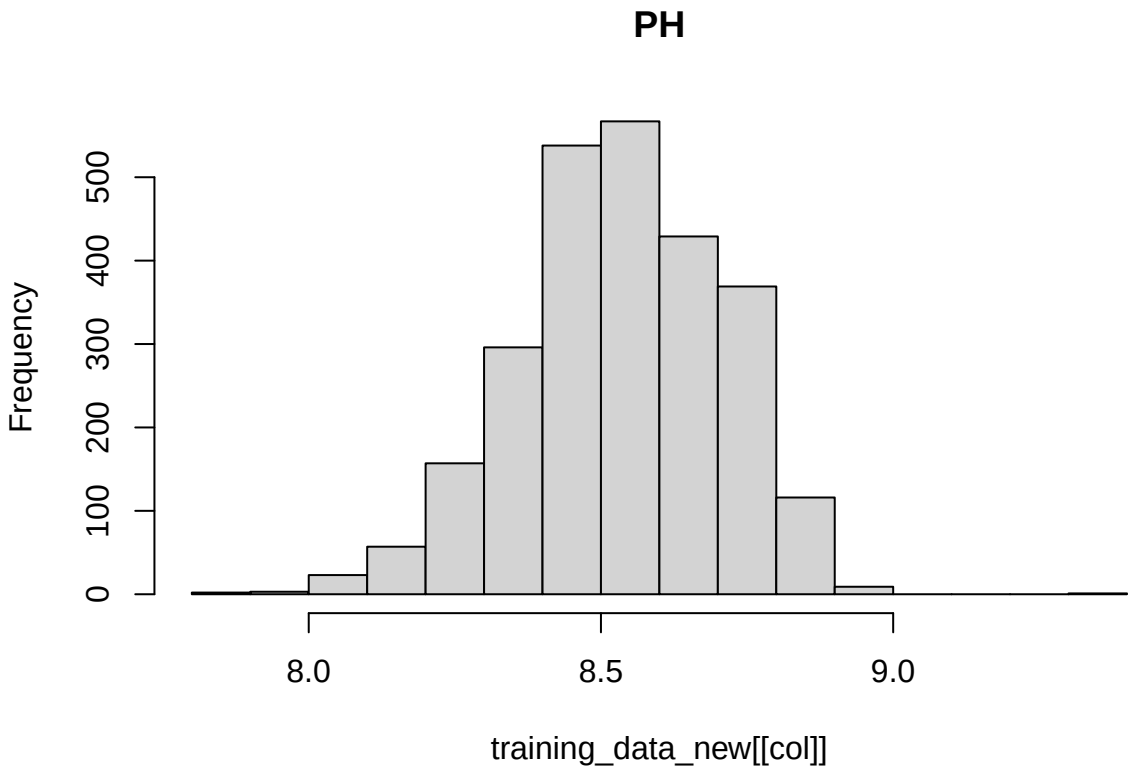

Filler.Level
Frequency
0
200
400
600
800
60
80
100
120
140
160
training_data_new[[col]]

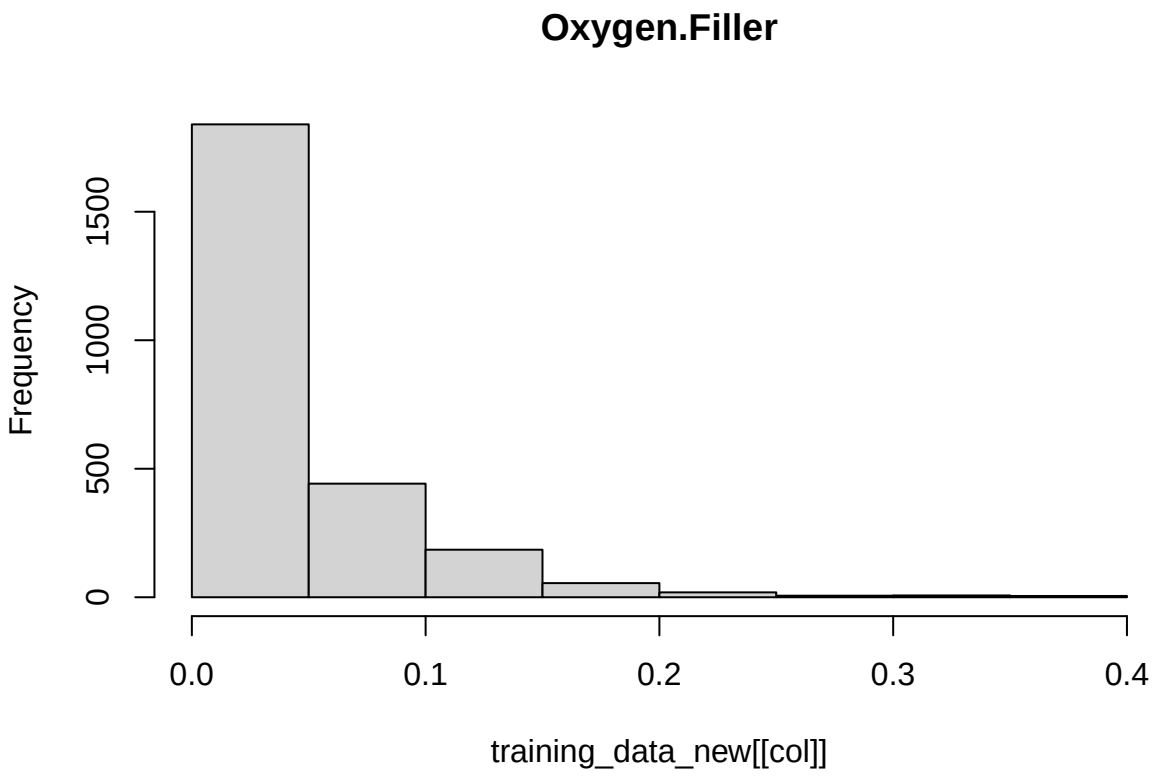

Filler.Speed
Frequency
0
500
1000
1500
1000
1500
2000
2500
3000
3500
4000
training_data_new[[col]]

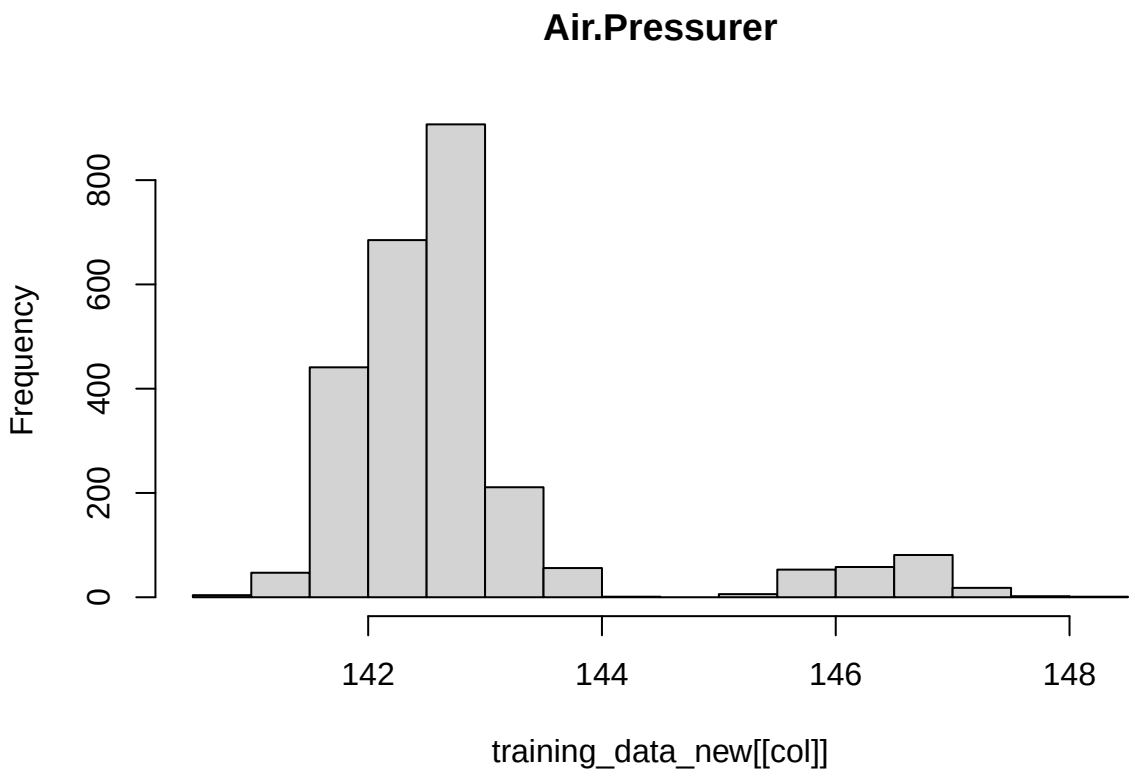

Temperature
Frequency
0
200
400
600
800
64
66
68
70
72
74
76
training_data_new[[col]]


Usage.cont
Frequency
0
200
400
600
800
12
14
16
18
20
22
24
26
training_data_new[[col]]

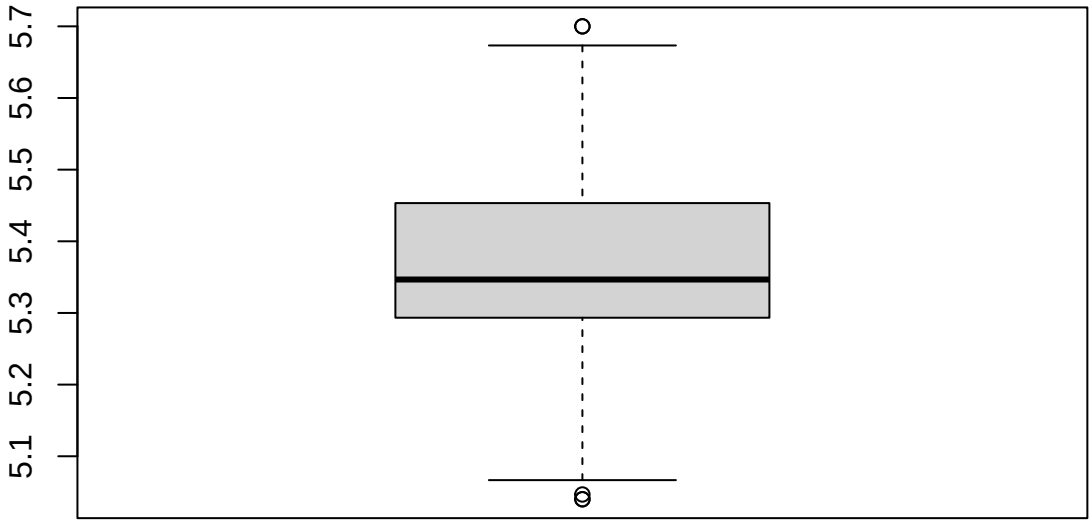

Carb.Flow
Frequency
0
200
600
1000
0
1000
2000
3000
4000
5000
training_data_new[[col]]

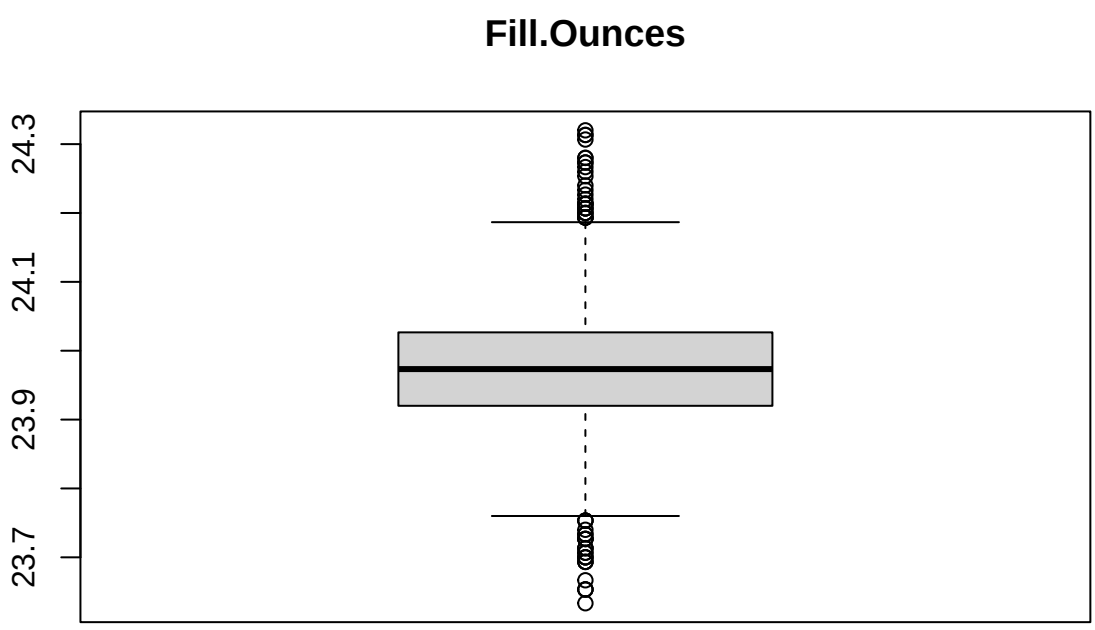

PH
Frequency
0
100
200
300
400
500
8.0
8.5
9.0
training_data_new[[col]]

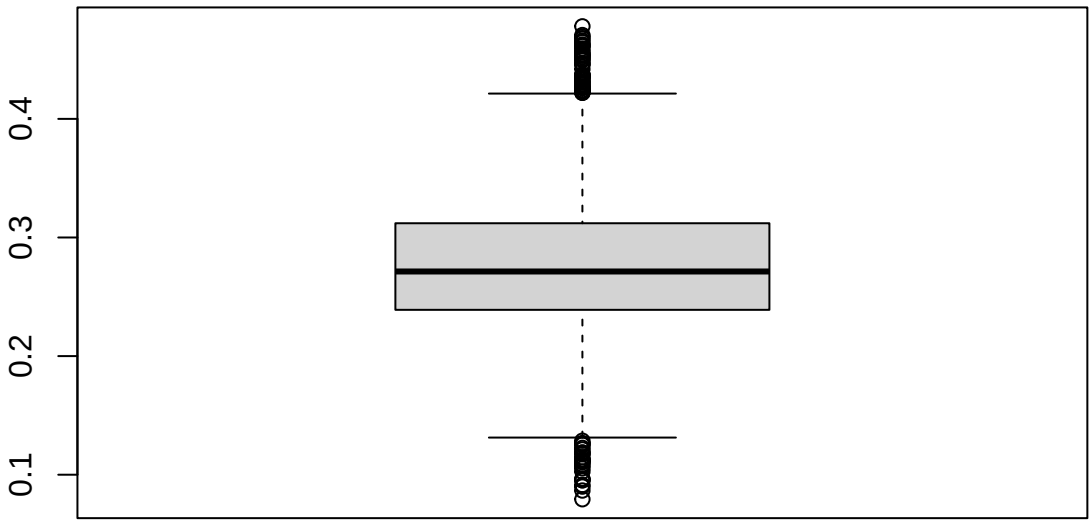

Oxygen.Filler
Frequency
0
500
1000
1500
0.0
0.1
0.2
0.3
0.4
training_data_new[[col]]

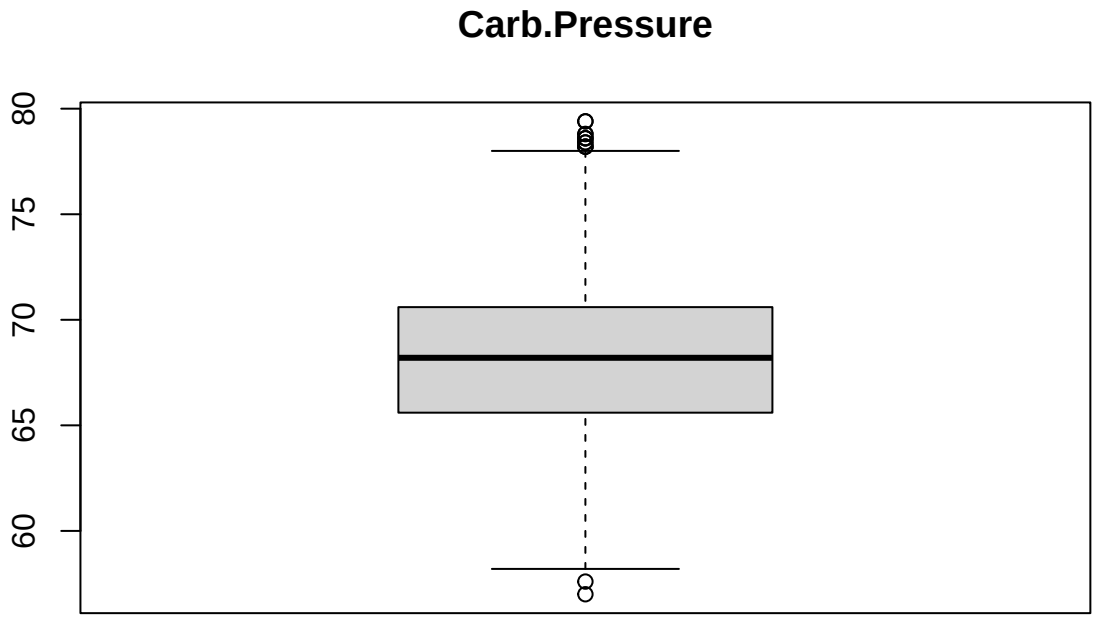


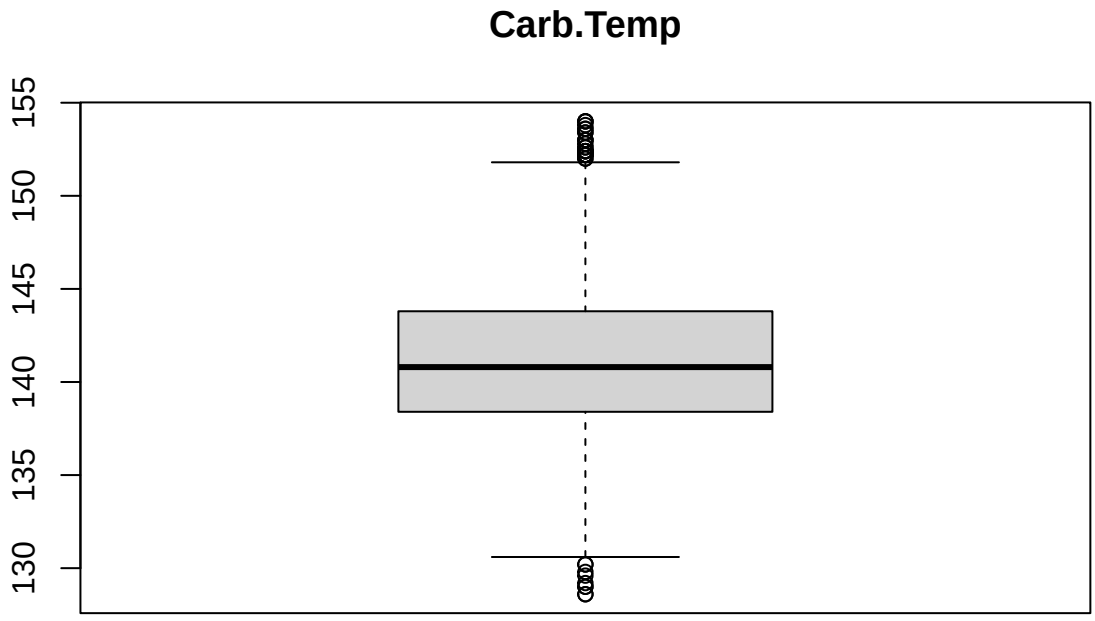

```
#box plots
for (col in names(training_data_new %>% select(-Brand.Code))) {
  boxplot(training_data_new[[col]], main=col)
}
```

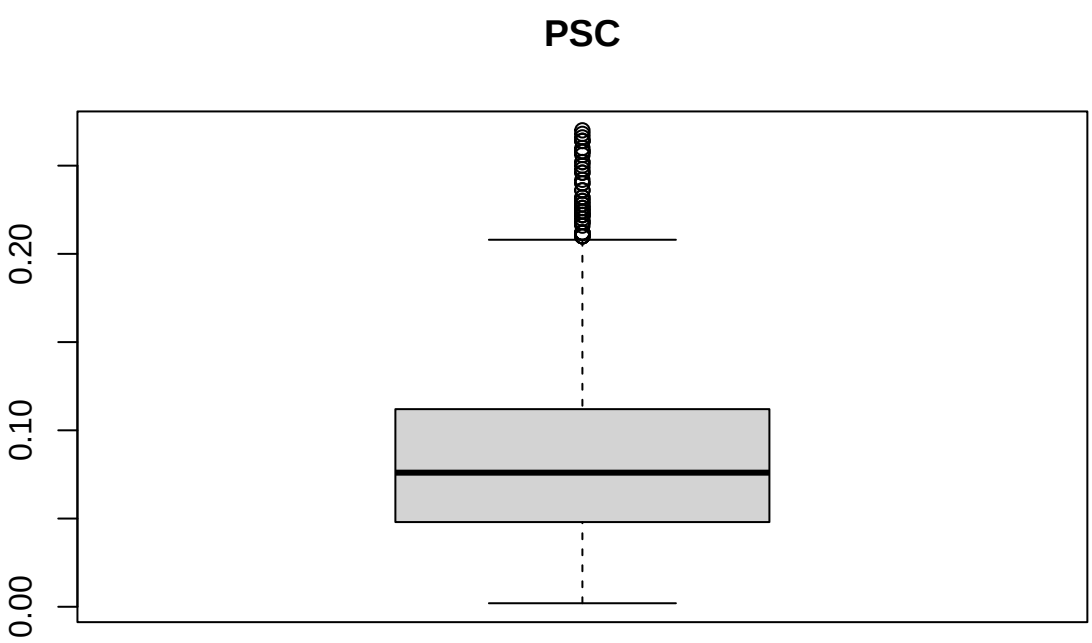

Carb.Volume
5.7
5.6
5.5
5.4
5.3
5.2
5.1

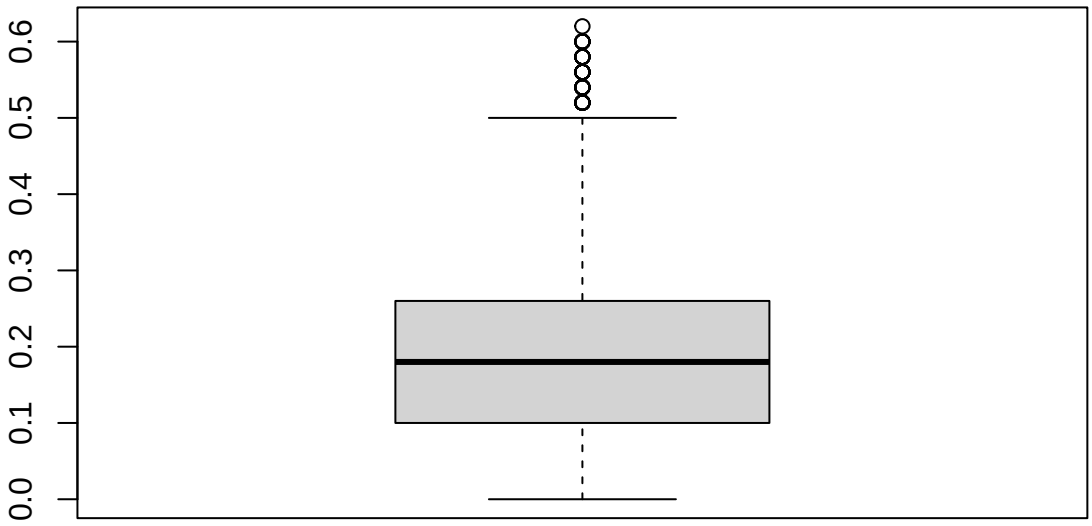

Fill.Ounces
24.3
24.1
23.9
23.7

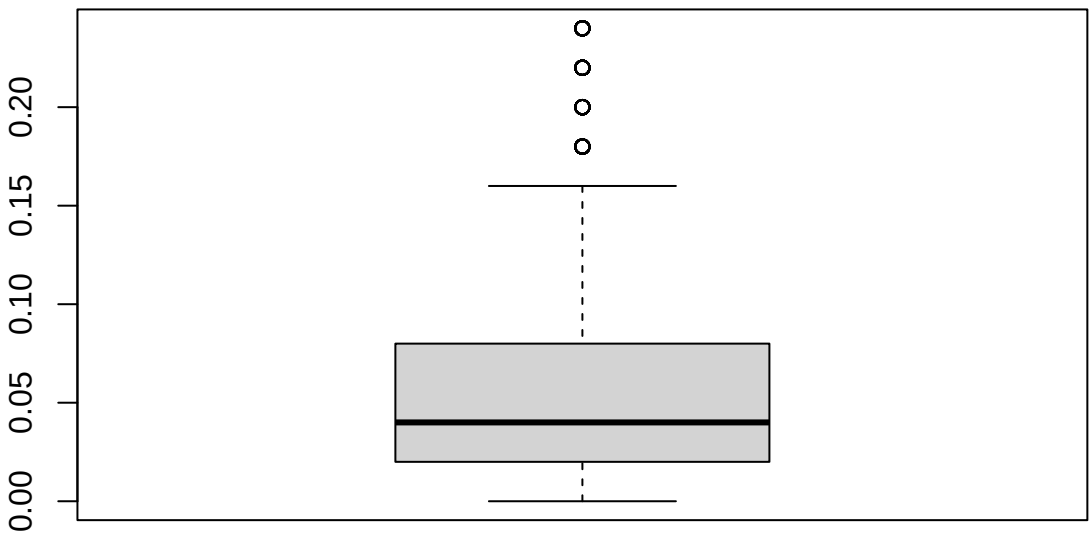

PC.Volume
0.1
0.2
0.3
0.4

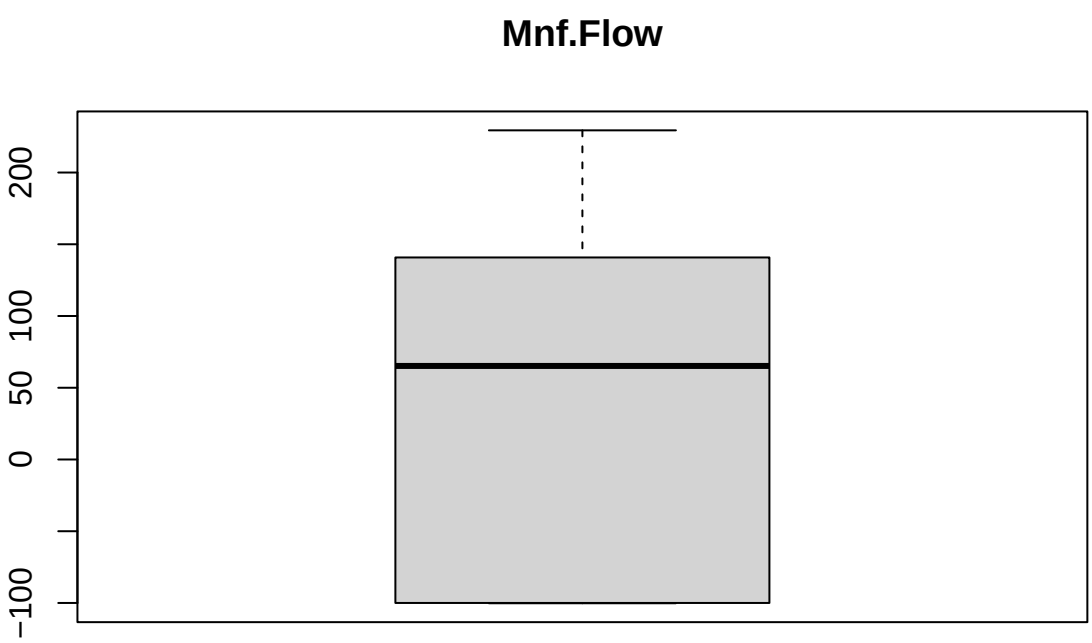

Carb.Pressure
80
75
70
65
60

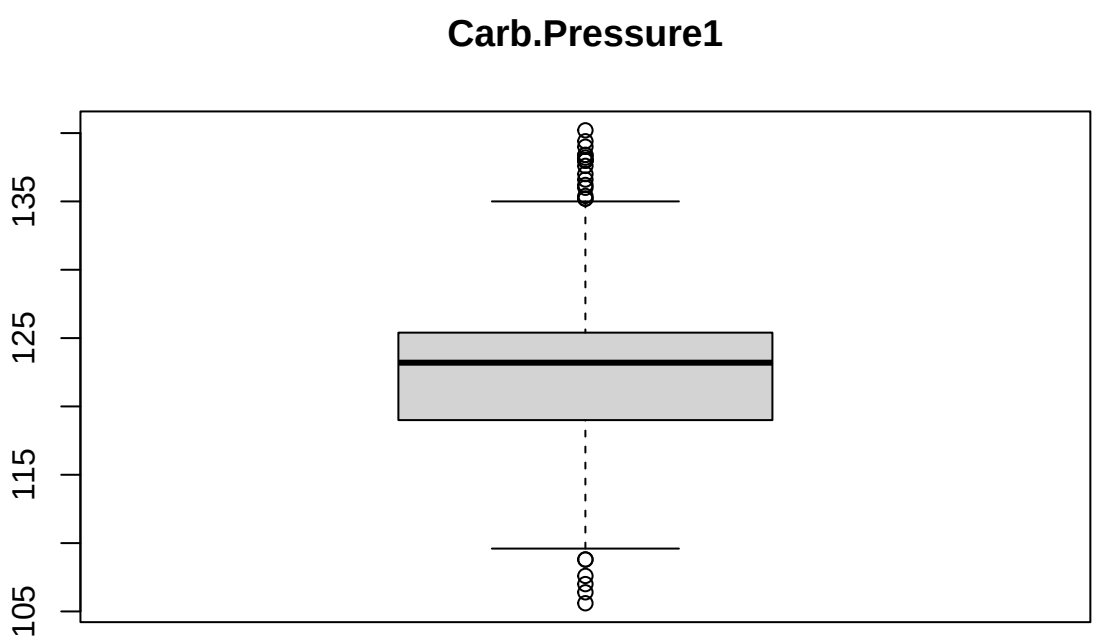

Carb.Temp
155
150
145
140
135
130

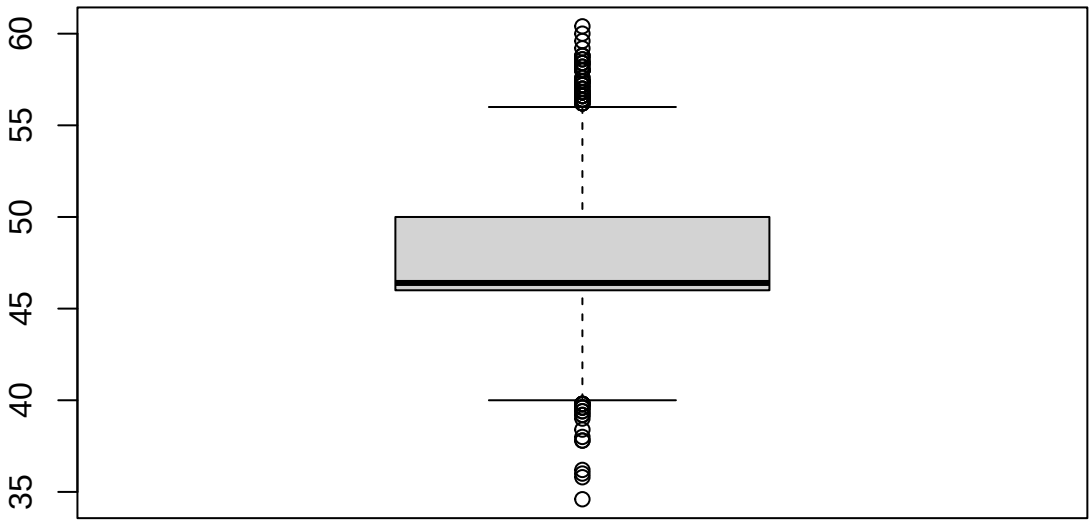

PSC
0.00
0.10
0.20

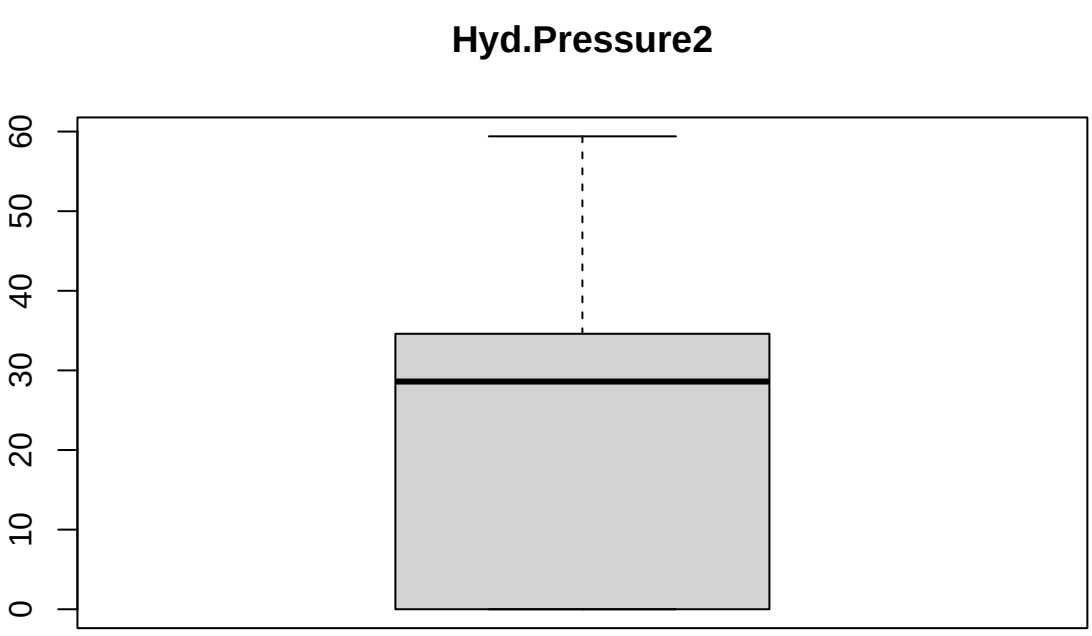

PSC.Fill
0.0
0.1
0.2
0.3
0.4
0.5
0.6

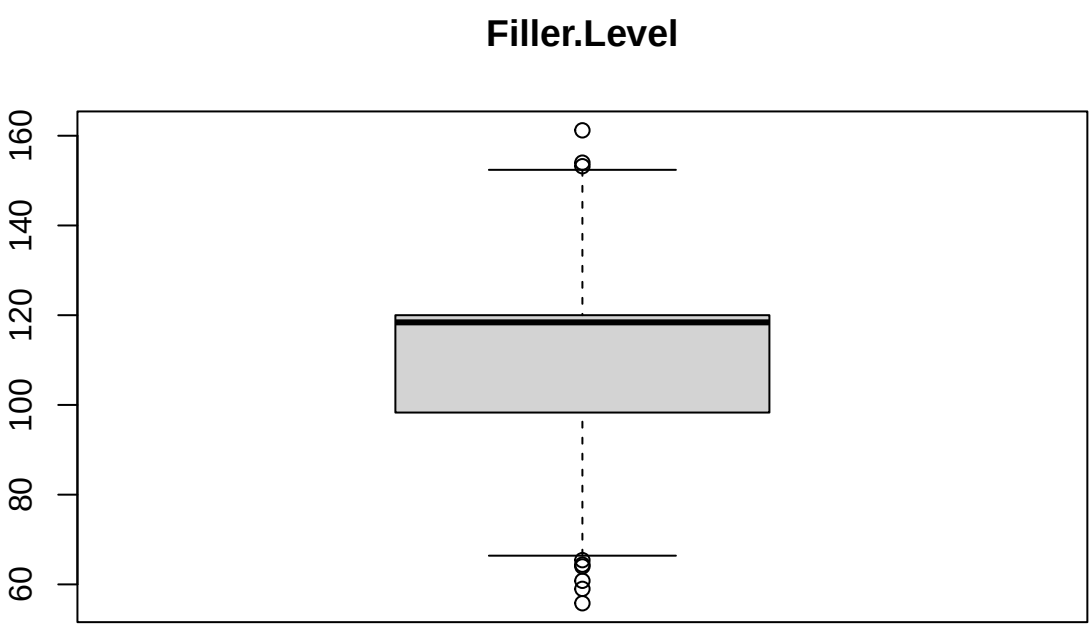

PSC.CO2
0.00
0.05
0.10
0.15
0.20

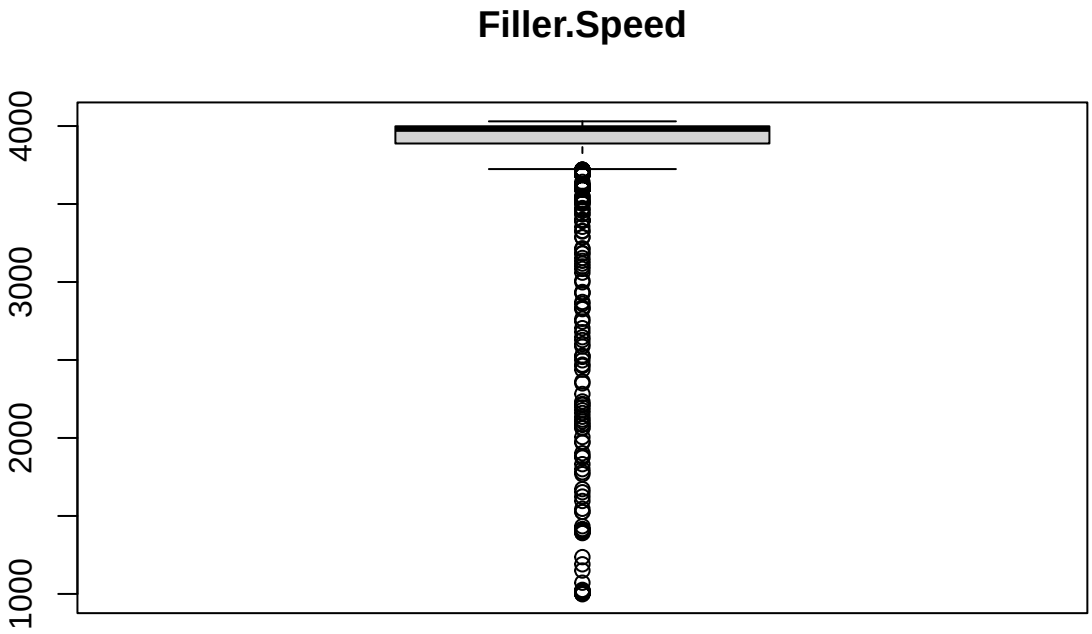

Mnf.Flow
200
100
50
0
−100

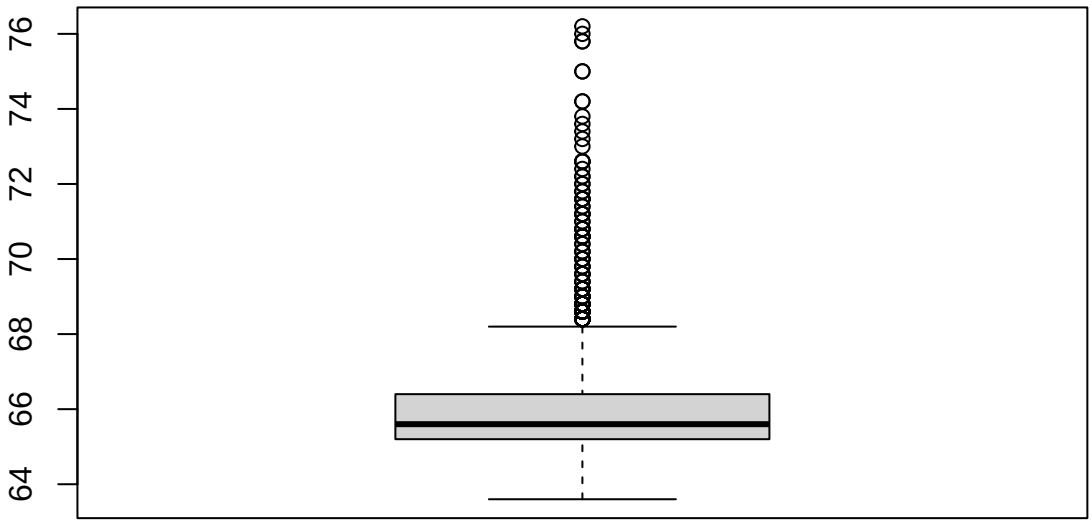

Carb.Pressure1
135
125
115
105

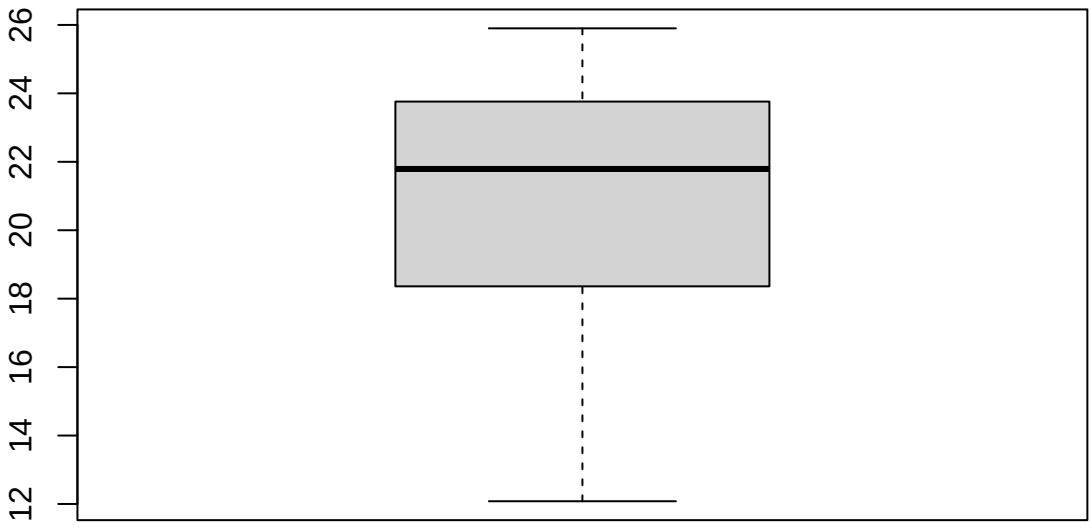

Fill.Pressure
60
55
50
45
40
35

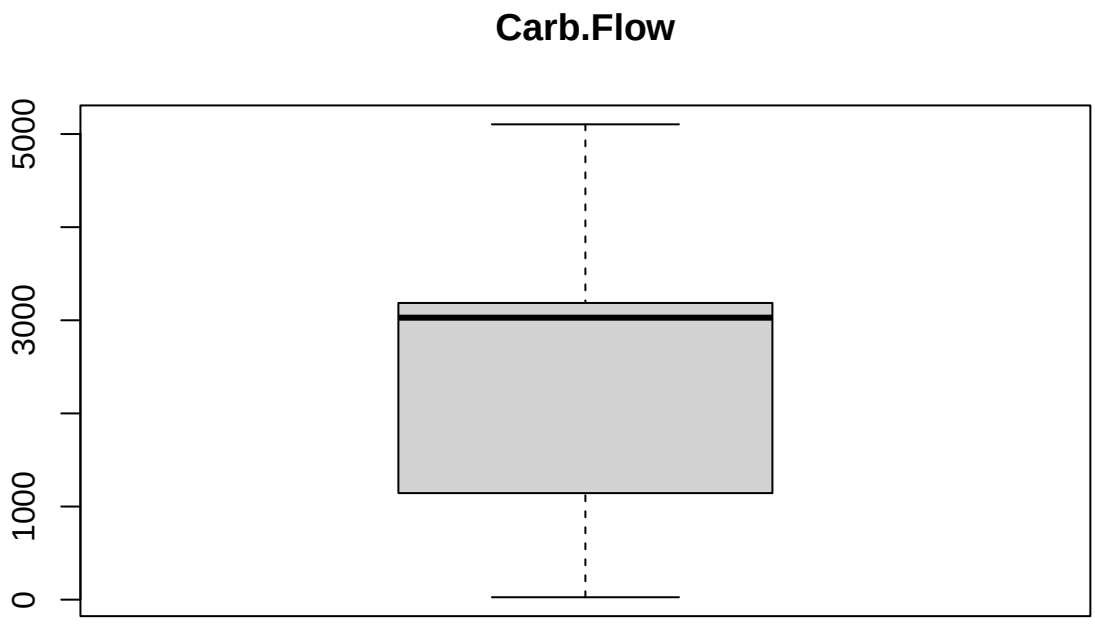

Hyd.Pressure2
0
10
20
30
40
50
60

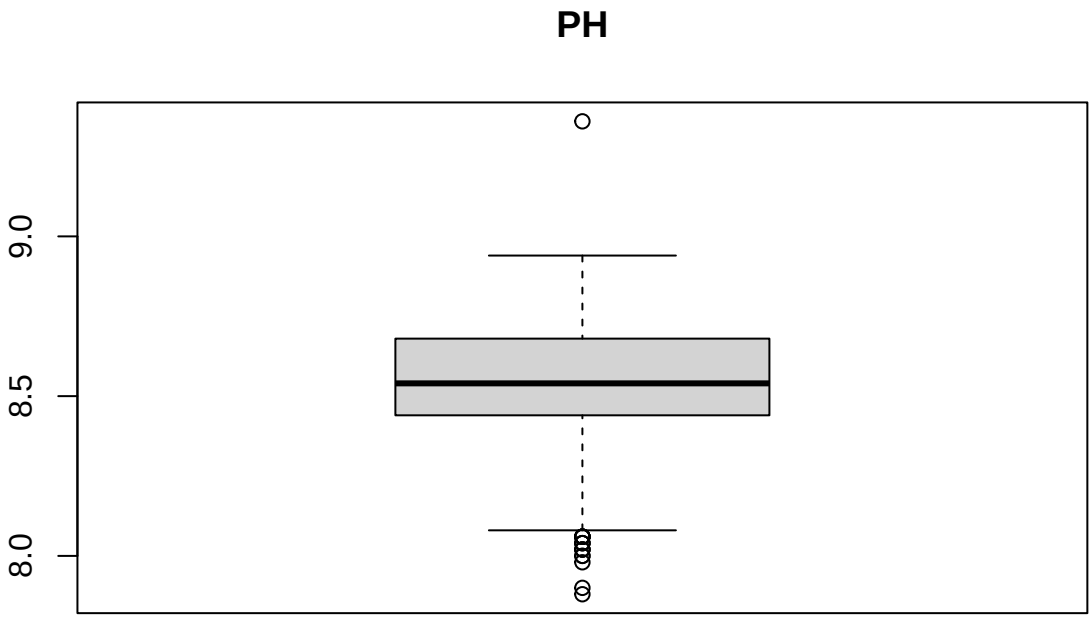

Filler.Level
160
140
120
100
80
60

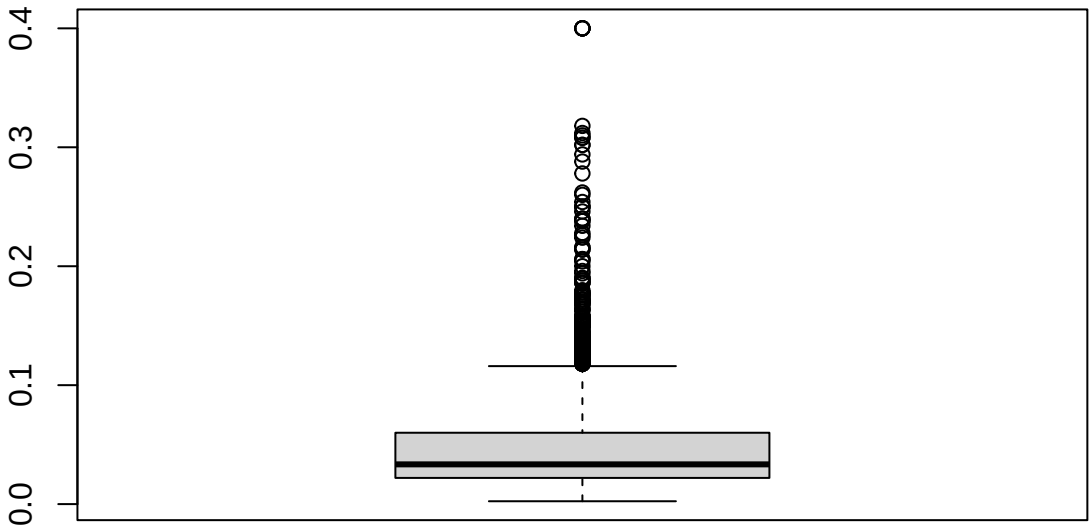

Filler.Speed
4000
3000
2000
1000

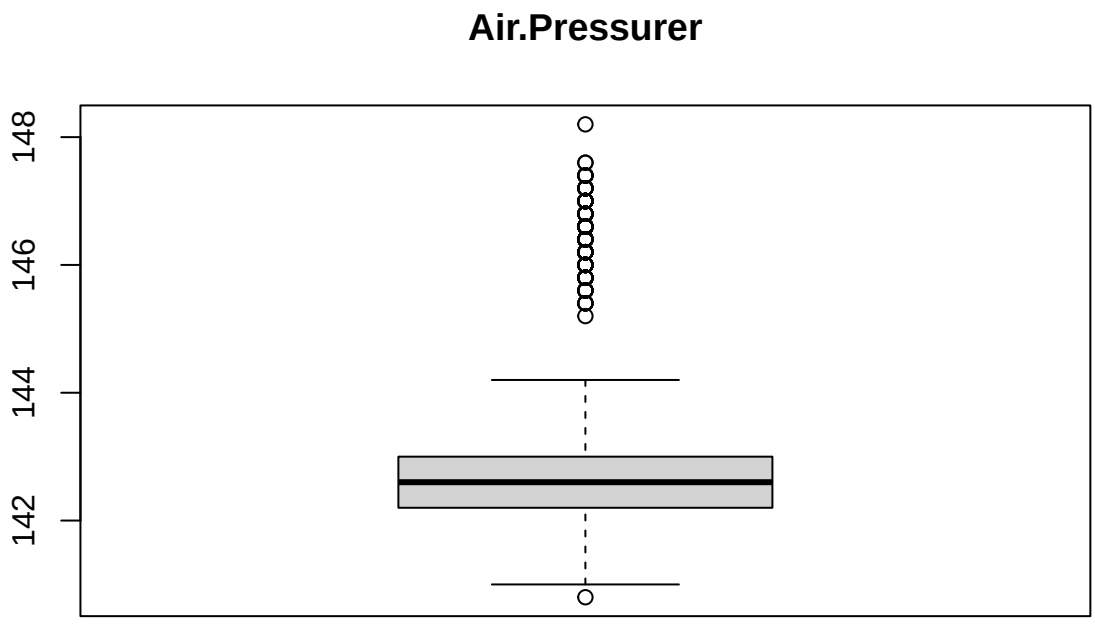

Temperature
64
66
68
70
72
74
76


Usage.cont
26
24
22
20
18
16
14
12


Carb.Flow
5000
3000
1000
0


PH
9.0
8.5
8.0


Oxygen.Filler
0.0
0.1
0.2
0.3
0.4



Calculating the number of outliers in each column based on the IQR method

```
outliers <- sapply(training_data_new %>% select(-Brand.Code), function(x) {
  Q1 <- quantile(x, 0.25, na.rm=TRUE)
  Q3 <- quantile(x, 0.75, na.rm=TRUE)
  IQR <- Q3 - Q1
  sum(x < Q1 - 1.5*IQR | x > Q3 + 1.5*IQR, na.rm=TRUE)
})
outliers
```

```
##    Carb.Volume    Fill.Ounces      PC.Volume  Carb.Pressure      Carb.Temp
##              5             55             86             14             36
##            PSC       PSC.Fill        PSC.CO2      Mnf.Flow Carb.Pressure1
##             52             54             77              0             24
##  Fill.Pressure  Hyd.Pressure2   Filler.Level   Filler.Speed    Temperature
##             79              0              9            439            123
##     Usage.cont      Carb.Flow             PH  Oxygen.Filler  Air.Pressurer
##              0              0             18            189            220
```

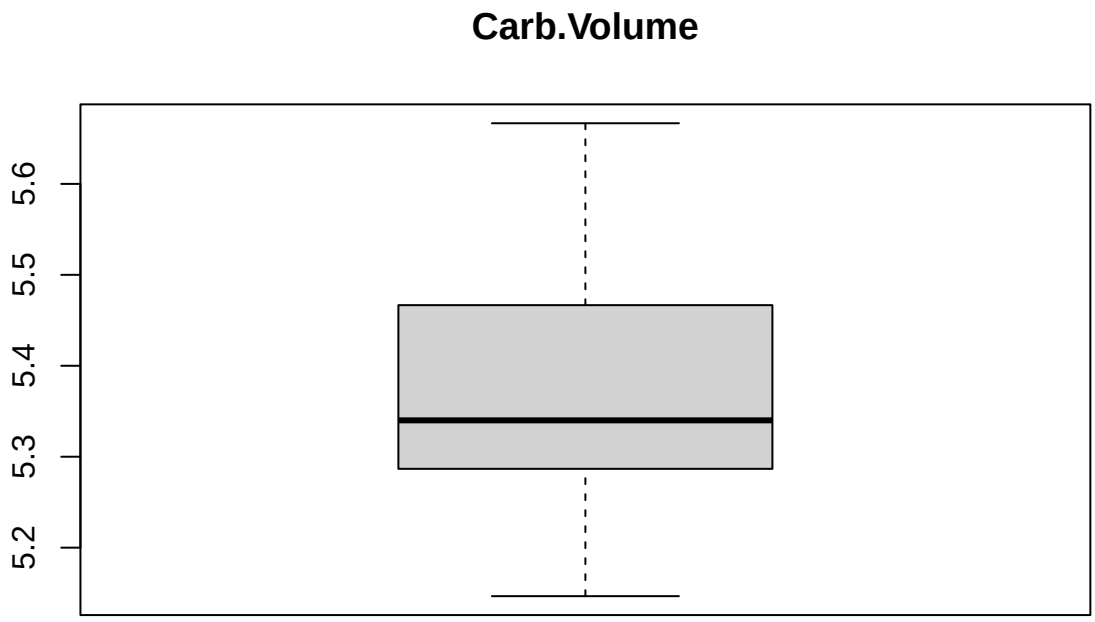
Cross-checking the graphs with the data set, we could remove the one `Filler.Level` value that appears on the top of the box plot, 161.2 when the next-highest is 154. pH has one comparatively high value (9.36) that's nowhere close to the next-highest value. We can remove that row since it's missing 4 other variables. PH at that level isn't "impossible" here, but with 4 missing predictor values in the same row and the value isolated far from the rest of the distribution, it seems more like a data entry issue than a real measurement we want to be using. Other outlier values are not one-offs, but instead there are a handful of data points at

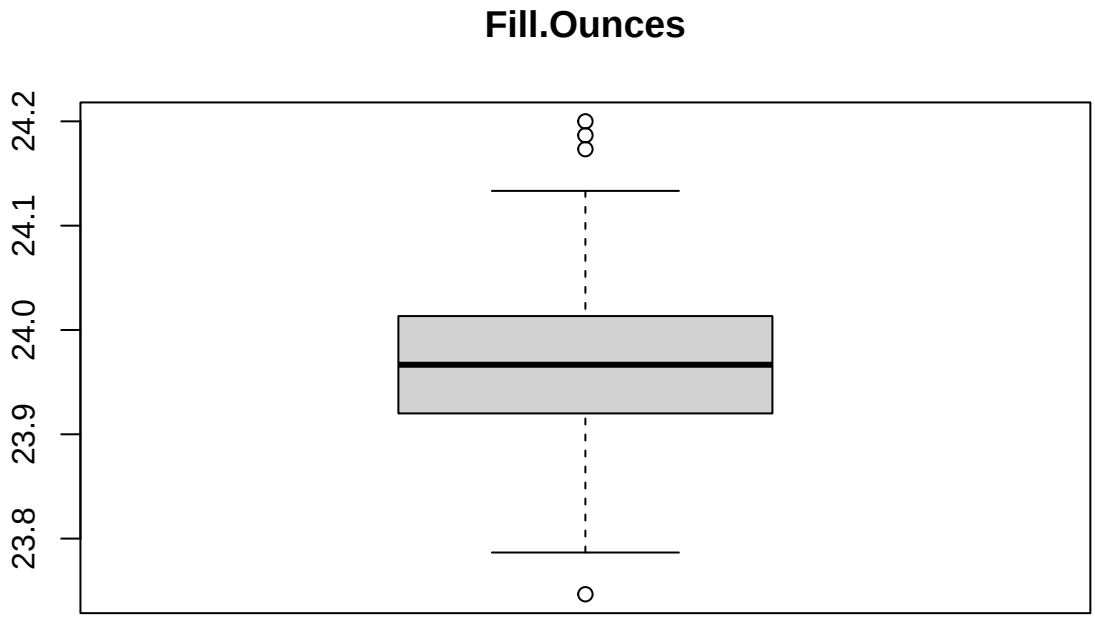
the high/low potential outlier values, so we're leaving the other alone. (For example, it looks like there's one particularly high value for `Oxygen.Filler`, but ther when we double checked, there 5 cases with the same value. Some light notes on that are written below.)

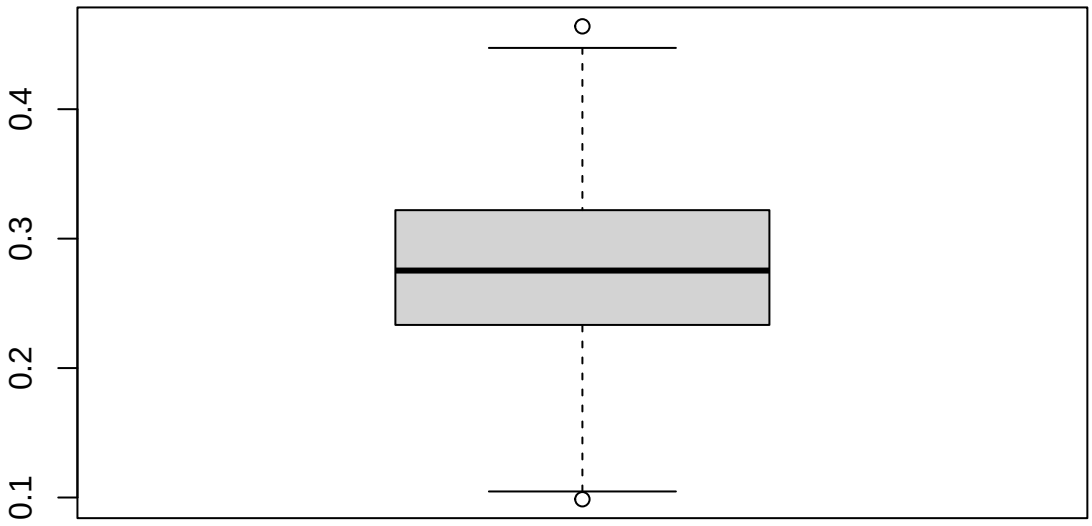
```
#removing the one outliers
training_data_new <- training_data_new %>%
  filter(PH < 9 | is.na(PH)) %>%
  filter(Filler.Level < 160 | is.na(Filler.Level))

#removing pH NAs because we shouldn't impute the response variable
training_data_no_na <- training_data_new %>%
  filter(!is.na(PH))
```

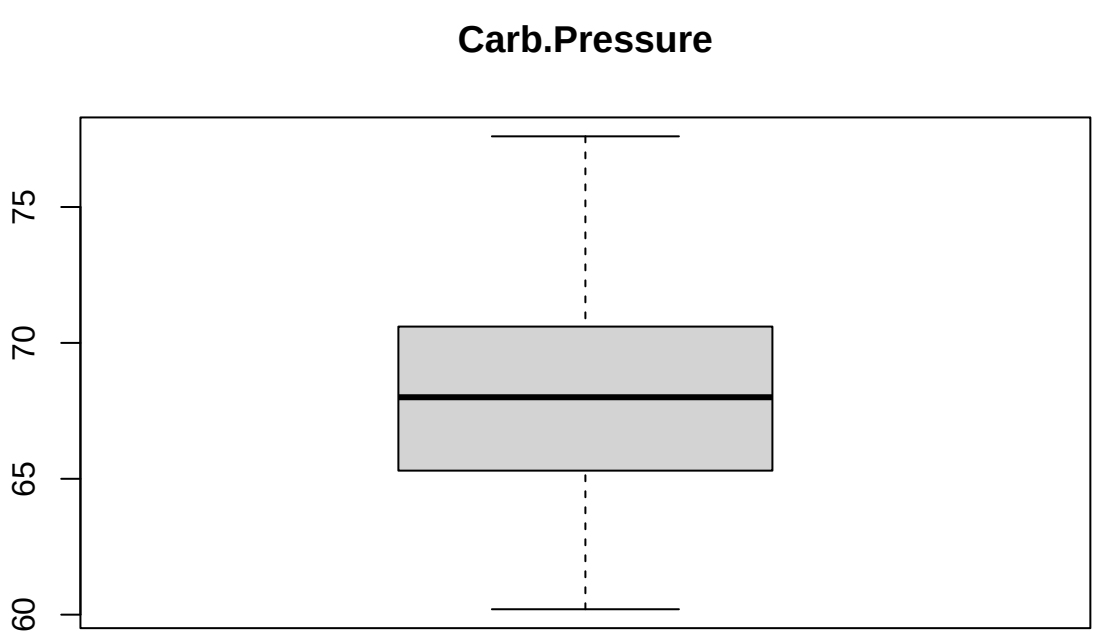
1.3.5 Looking at outliers in the test data set

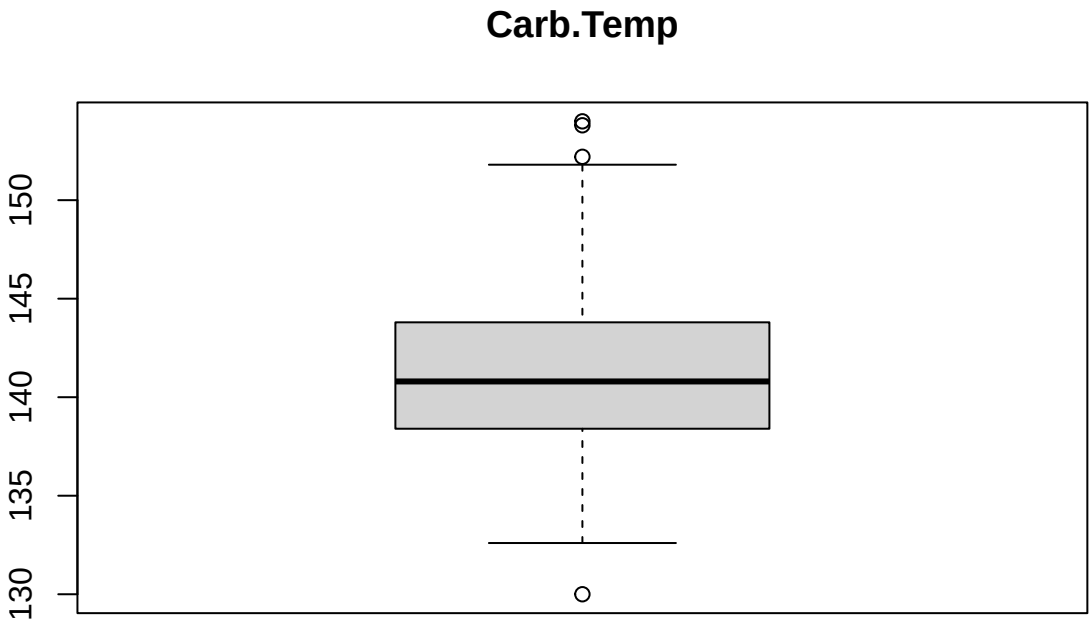
```
#leaving out the categorical and blank response variable
for (col in names(test_data_new %>% select(-Brand.Code, -PH))) {
  boxplot(test_data_new[[col]], main=col)
}
```

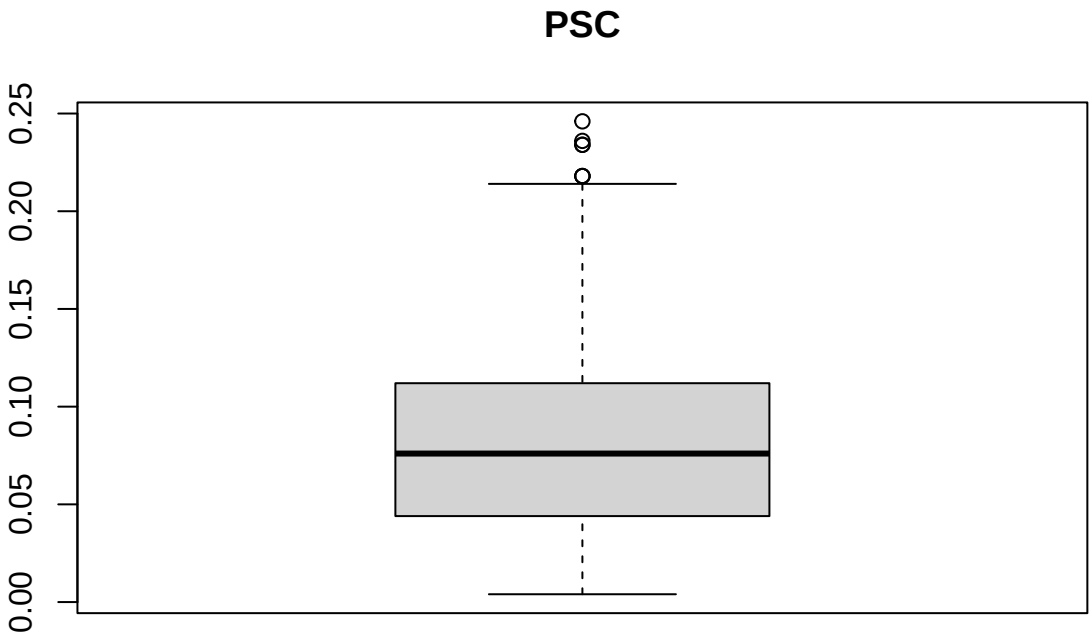


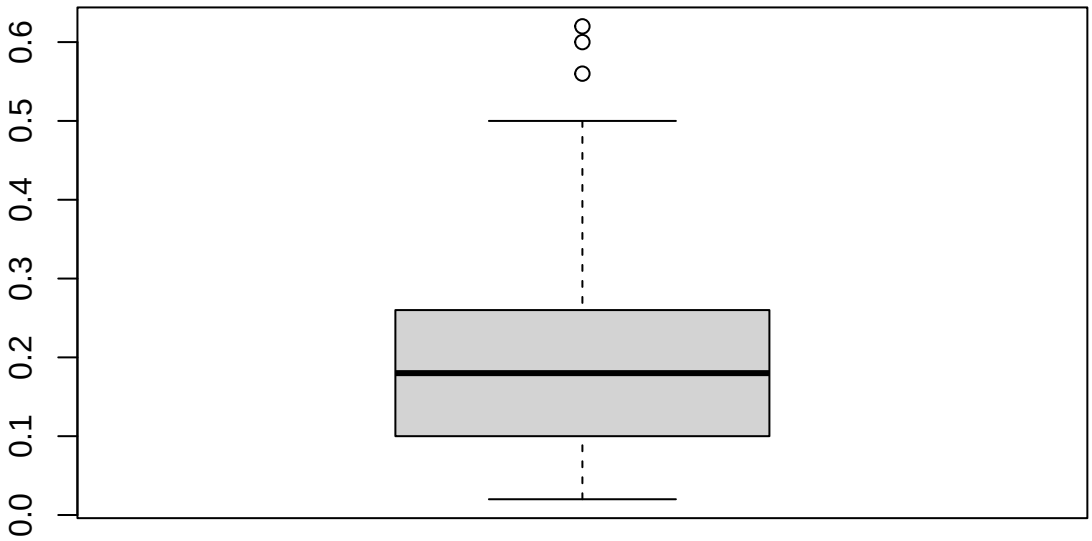

Fill.Ounces
24.2
24.1
24.0
23.9
23.8

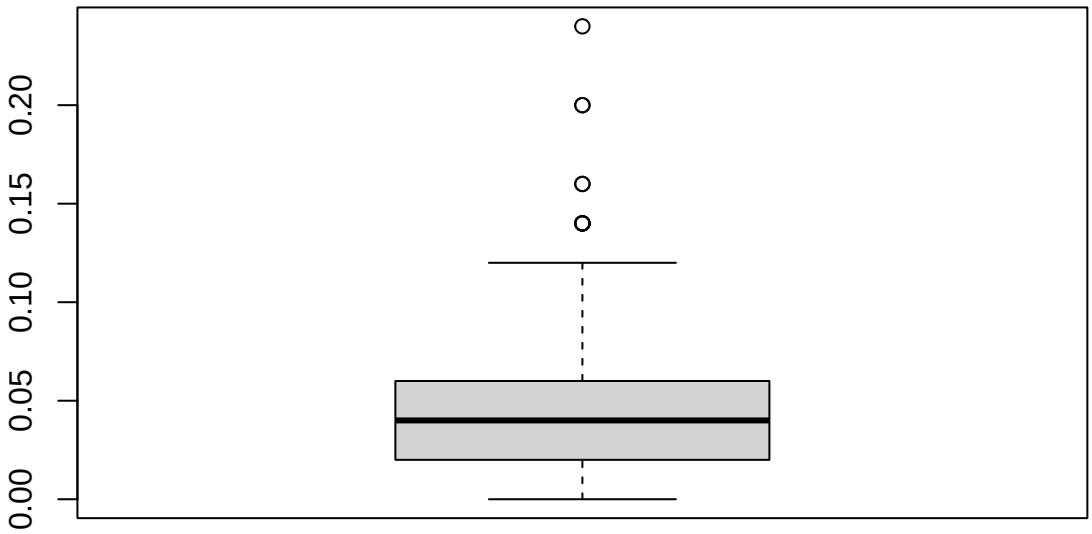

PC.Volume
0.1
0.2
0.3
0.4

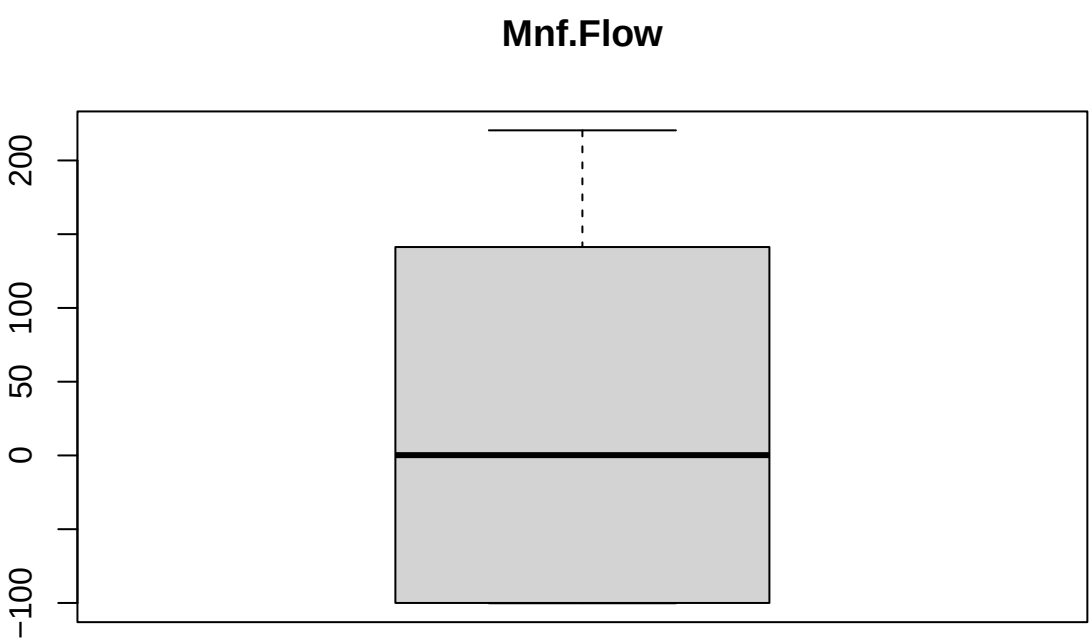

Carb.Pressure
75
70
65
60

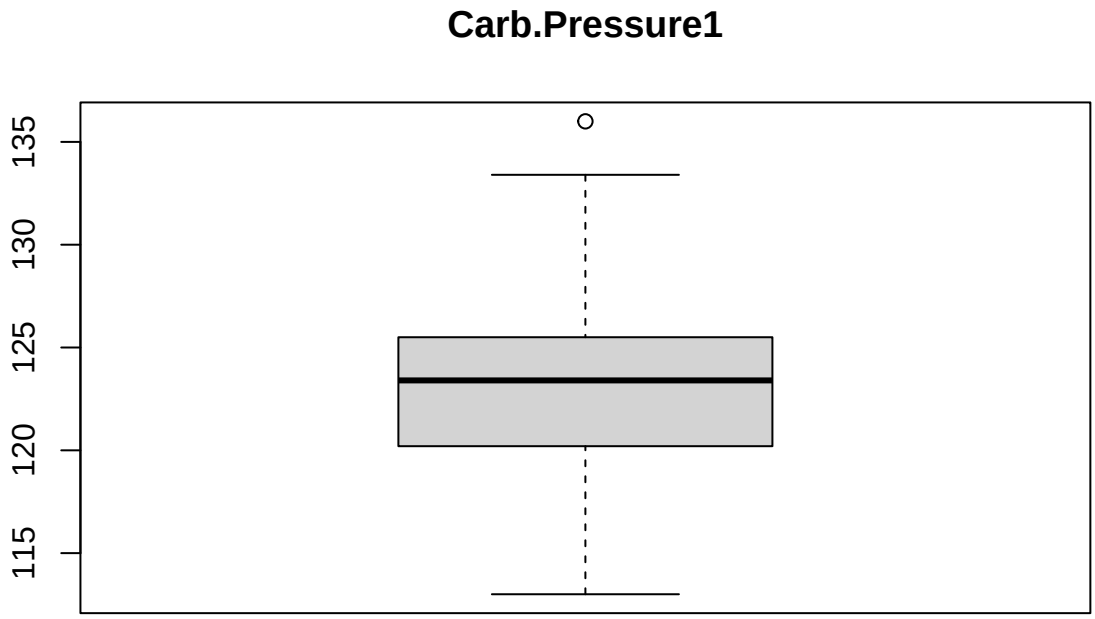

Carb.Temp
130
135
140
145
150

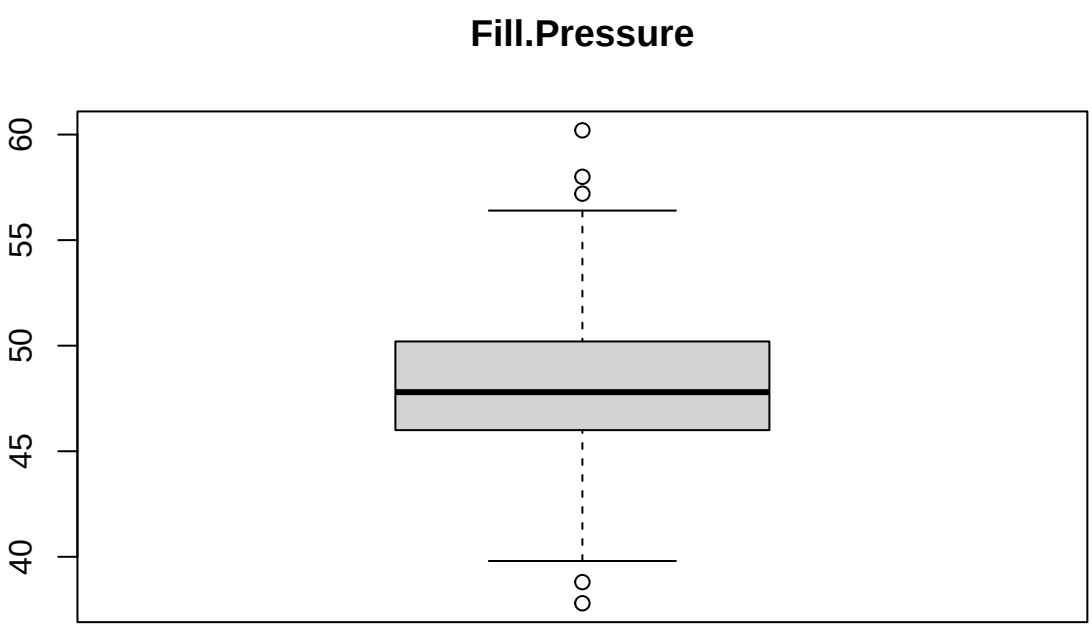

PSC
0.00
0.05
0.10
0.15
0.20
0.25

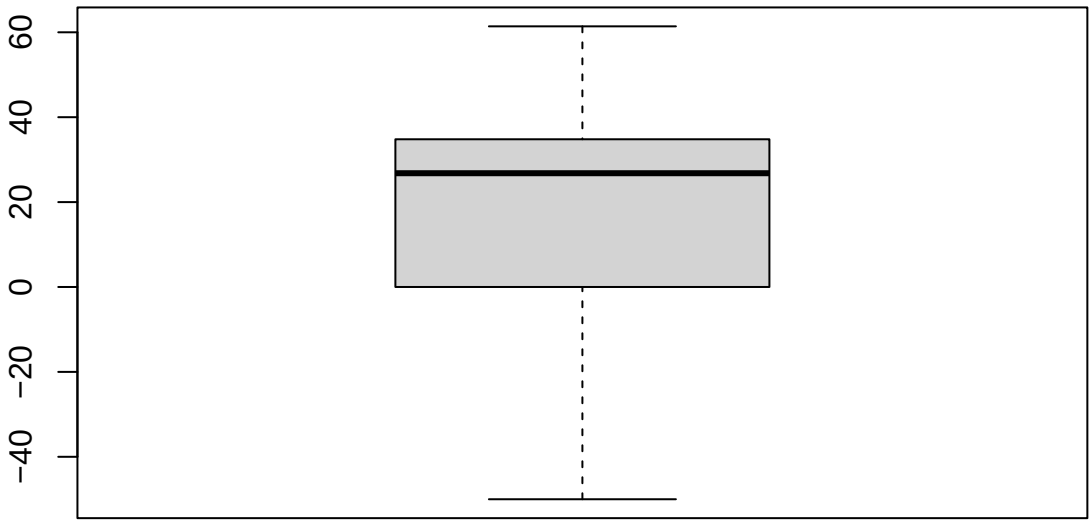

PSC.Fill
0.0
0.1
0.2
0.3
0.4
0.5
0.6

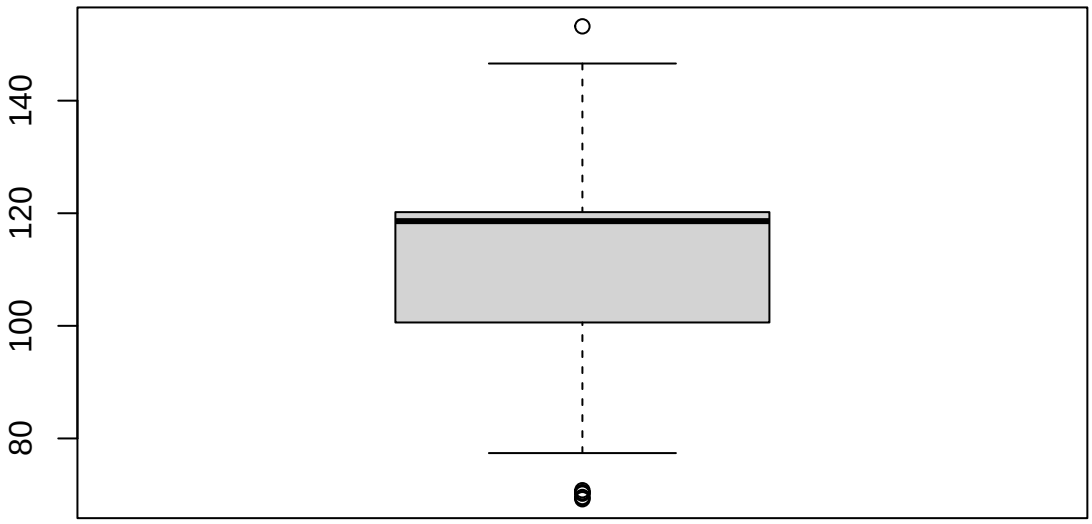

PSC.CO2
0.00
0.05
0.10
0.15
0.20


Mnf.Flow
200
100
50
0
−100

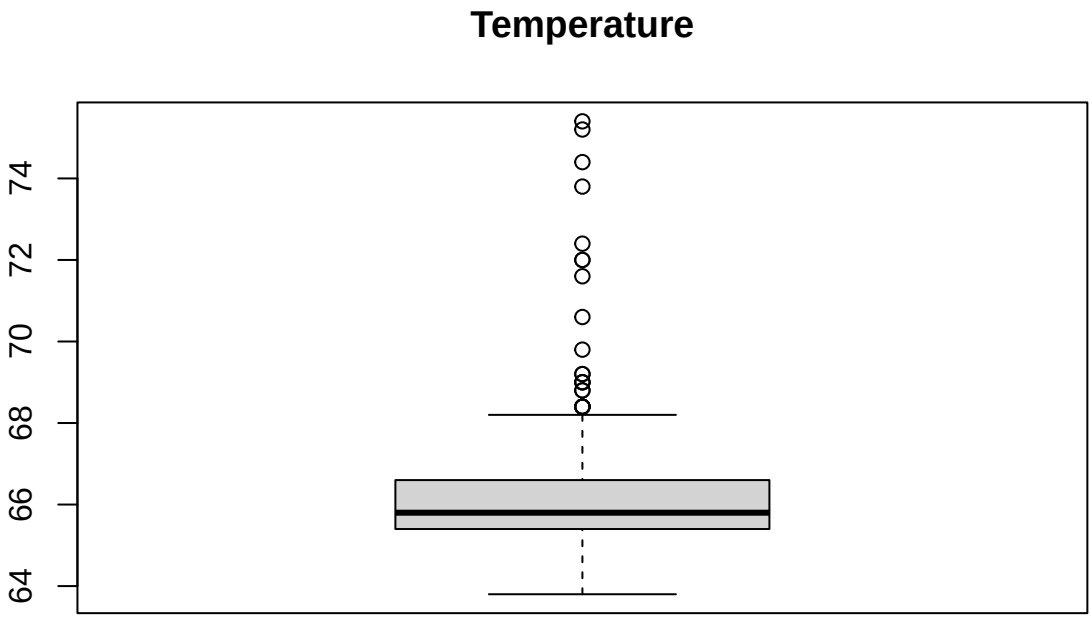

Carb.Pressure1
135
130
125
120
115

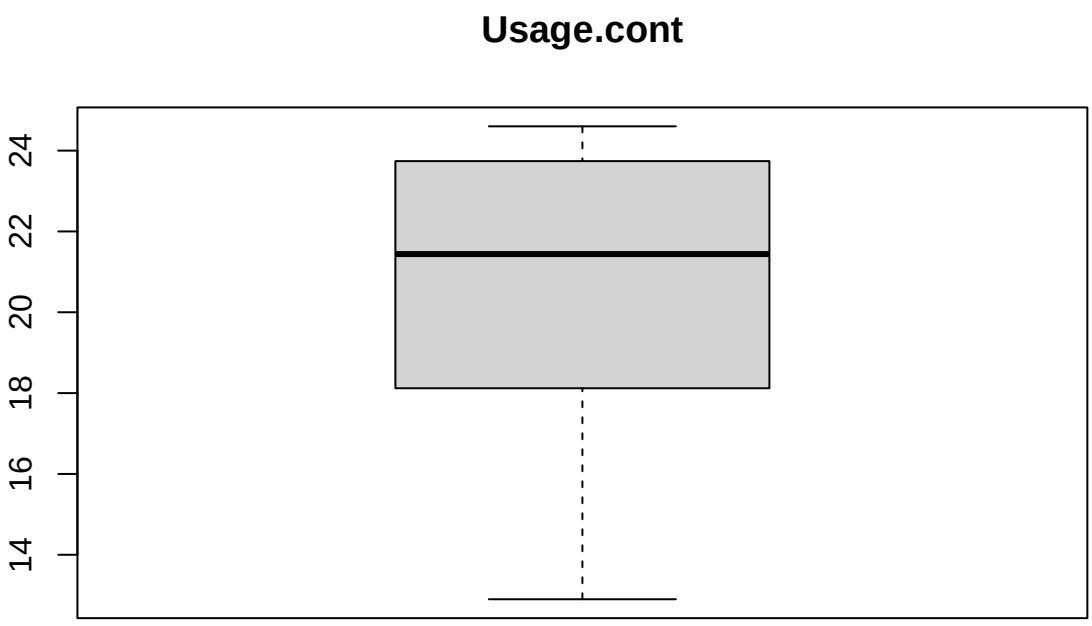

Fill.Pressure
60
55
50
45
40

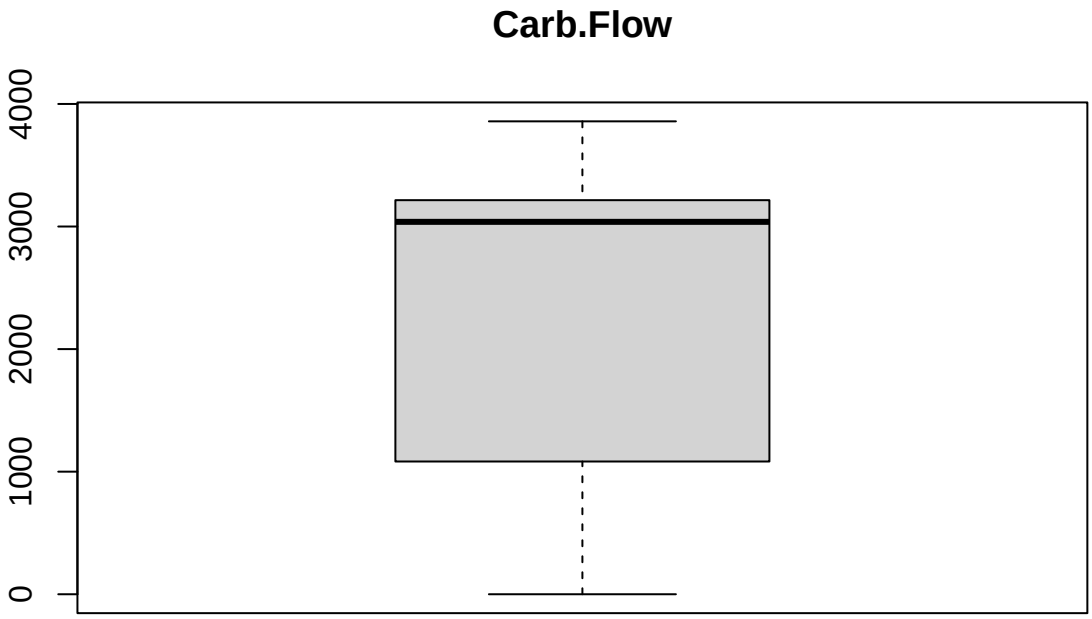

Hyd.Pressure2
60
40
20
0
−20
−40

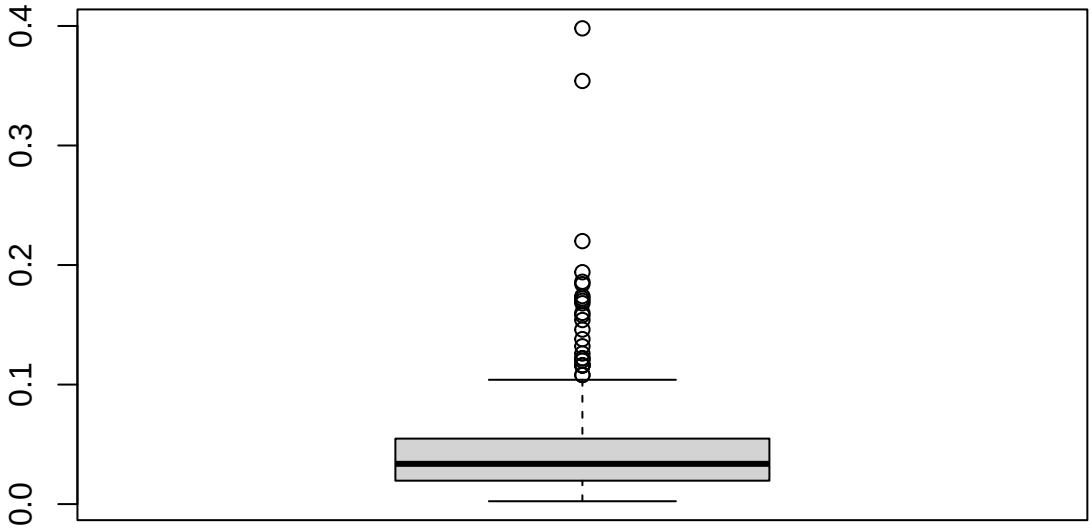

Filler.Level
140
120
100
80

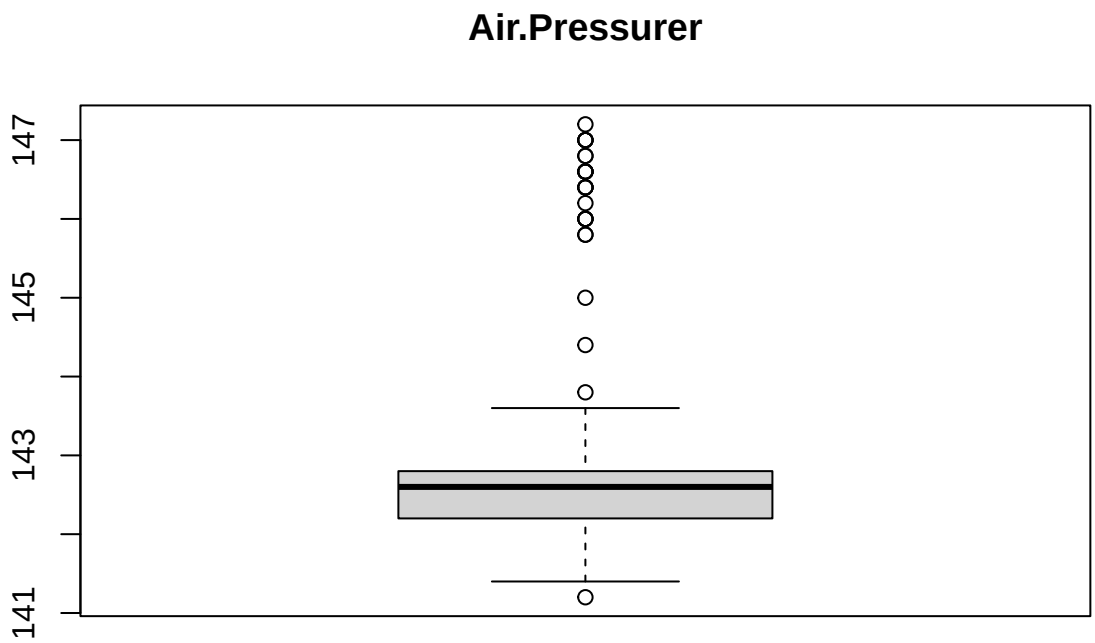

Filler.Speed
4000
3000
2000
1000


Temperature
64
66
68
70
72
74


Usage.cont
24
22
20
18
16
14


Carb.Flow
0
1000
2000
3000
4000


Oxygen.Filler
0.0
0.1
0.2
0.3
0.4




```
test_data_new %>% count(Brand.Code)
```

```
##   Brand.Code   n
## 1          A  35
## 2          B 129
## 3          C  31
## 4          D  64
## 5       <NA>   8
```

`Oxygen.Filler` shows up again with a few outliers, which just seems to be the case for this variable; the outliers in filler speed are all clustered; filler level has outliers on both ends, as does fill pressure. `Carb.Pressure1` has one high value, `PSC.CO2` has a few higher values designated outliers due to the clustering on the lower end, same is true for `PSC.Fill`, and the rest are similar.

We won't remove any of these because they don't look like errors, and in some cases represent a different, smaller cluster of values.

1.3.6 Keeping a wider feature set for tree-based models

The correlation prune above takes us to 20 predictors. This "pruned" or "narrowed" set is what we can use for linear models, PLS, penalized regression, and neural networks. However, when it comes to Tree-based models, we know that those are okay with collinearity fine, so they will be give the "unpruned" or "wider" set of predictors.

So:

- `training_data_new` / `test_data_new`: 20 predictors after Near-zero-variance (NZV) removal and r $>$ 0.90 correlation prune. For use with linear models, PLS, ridge / lasso / elastic net, and neural networks.
- `training_data_wide` / `test_data_wide`: 31 predictors after just NZV removal. For use with random forest, gradient boosting, and Cubist.

```
# wide feature set: still did NZV removal, plus has the same outlier filters as the narrow set
training_data_wide <- training_data %>%
  select(-Hyd.Pressure1) %>%
  filter(PH < 9 | is.na(PH)) %>%
  filter(Filler.Level < 160 | is.na(Filler.Level))

training_data_wide_no_na <- training_data_wide %>%
  filter(!is.na(PH))

# same column drop on the evaluation set
test_data_wide <- test_data %>%
  select(-Hyd.Pressure1)
```

1.3.7 Recap

- Dropped `Hyd.Pressure1` for near-zero variance
- Dropped 11 collinear variables (with the cutoff r $>$ 0.90), keeping one representative predictor per each correlated group of predictors.
  - Also we manually chose to drop `MFR` instead of `Filler.Speed`, since `MFR` had more missingness and would have been less reliable to keep.
- Checked for outliers using IQR + histograms + boxplots
  - Removed two rows only (one with `PH = 9.36`, one with `Filler.Level = 161.2`). Everything else looked liked like legitimate data.
- Removed rows with missing `PH` from the training data since we can't impute the response
- Narrow training set has 21 columns (20 predictors + `PH`)
- Also derived a wider feature set (`training_data_wide_no_na`, 31 predictors) by skipping the correlation prune, for tree-based models that are fine with there being collinearity

1.4 Dealing with missing values

Recap located at the end of the section. Skip there to jump past the code/workflow.

```
sum(is.na(training_data_no_na))
```

```
## [1] 515
```

```
sort(colSums(is.na(training_data_no_na)), decreasing = TRUE)
```

```
##     Brand.Code   Filler.Speed      PC.Volume        PSC.CO2    Fill.Ounces
##            120             52             39             39             38
##            PSC Carb.Pressure1  Carb.Pressure      Carb.Temp       PSC.Fill
##             33             32             27             26             23
```

```
##  Fill.Pressure  Hyd.Pressure2   Filler.Level    Temperature  Oxygen.Filler
##             17             15             15             11             11
##    Carb.Volume     Usage.cont      Carb.Flow       Mnf.Flow             PH
##             10              5              2              0              0
##  Air.Pressurer
##              0
```

```
#checking the number of complete cases
table(rowSums(is.na(training_data_no_na)))
```

```
##
##    0    1    2    3    4    5    6
## 2169  309   64   18    2    2    1
```

516 missing values across 2566 cases and 21 variables. This means 516/53886 is about .9% overall missingness, which is very low. Worth noting, brand code is missing in 120 cases.

2169 cases our of 2566 are complete, or about 85%. If we dropped all NAs, we would be dropping 15% of the data. So instead, we'll be using imputation.

However! the imputation we're doing here is mostly an inspection step, plus `test_data_imputed` gets used by the eval prediction chunk much later in our worflow, and that's only if narrow-set model wins (like a linear model). In our actual model training, we're going to let `caret` do the imputation per CV fold to avoid data leakage.

Now for `test_data_imputed`, we're using `VIM` package for simple KNN imputation. This package just fills in the nearest neighbor; it doesn't change the other variables. Vim can handle categorical variables, like brand code.

```
training_data_imputed <- kNN(training_data_no_na, k = 5)

#making sure it worked
sum(is.na(training_data_imputed))
```

```
## [1] 0
```

1.4.0.1 Imputing values for the evaluation set

First, let's check how many missing values there are after we've removed columns in the evaluation set.

```
table(rowSums(is.na(test_data_new)))
```

```
##
##   1   2   3   4   5   9
## 225  33   6   1   1   1
```

```
colSums(is.na(test_data_new))
```

```
##     Brand.Code    Carb.Volume    Fill.Ounces      PC.Volume  Carb.Pressure
##              8              1              6              4              0
##      Carb.Temp            PSC       PSC.Fill        PSC.CO2       Mnf.Flow
##              1              5              3              5              0
## Carb.Pressure1  Fill.Pressure  Hyd.Pressure2   Filler.Level   Filler.Speed
```

```
##              4              2              1              2             10
##    Temperature     Usage.cont      Carb.Flow             PH  Oxygen.Filler
##              2              2              0            267              3
##  Air.Pressurer
##              1
```

```
#total NAs (subtracting 267 for the missing pH values)
sum(is.na(test_data_new)) - 267
```

```
## [1] 60
```

Now there are only 60 missing values total. 42 rows have missing values. One row has 8 missing values (the one that had 14 before).

Imputing a row that sparse means the filled-in predictor values will be pretty noisy, but the deliverable expects a prediction for every row in the evaluation set. For that reason, we're opting to keep the row in and make a note that the prediction is low-confidence, but we aren't going to drop the row.

```
#keeping all 267 rows
count(test_data_new)
```

```
##     n
## 1 267
```

Imputing with the same method we used for the training data:

```
#list all columns except PH
cols_to_impute <- setdiff(colnames(test_data_new), "PH")

test_data_imputed <- kNN(test_data_new, variable = cols_to_impute, k = 5)

#we should get 267 cases
sum(is.na(test_data_imputed))
```

```
## [1] 267
```

1.4.1 Recap

- Training set started with 516 missing cells across 2566 rows and 21 columns (about 0.9% missingness). Imputed the gaps with VIM and `kNN(k = 5)` to get `training_data_imputed`
- Evaluation set:
  - kept all 267 rows including one with 8 predictor NAs out of 20 (originally 14 before column drops)
  - imputed predictor columns only with VIM and `kNN(k = 5)` to get `test_data_imputed`. The sparse row gets imputed too but we're flgging this as a low-confidence prediction.
- Both sets are now fully imputed (apart from `PH` in the eval set) and ready for modeling

1.5 Break into training and test data (inspection only, not used downstream)

Recap located at the end of the section. Skip there to jump past the code/workflow.

Originally we made this section to produce an 80/20 split for a quick sanity check on the data. When we updated our modeling pipeline to use 10-fold CV on the full `training_data_no_na` data instead, it meant that we weren't using these `train_x` / `test_x` objects downstream anymore. For that reason, the chunk is set to `eval=FALSE` so it doesn't run on knit, but we wanted to keep the code here both for reference, and for the option to revert if we want to.

```
set.seed(624)

train_index <- sample(nrow(training_data_imputed),
                      0.8 * nrow(training_data_imputed))

#create a training set
train_y <- training_data_imputed$PH[train_index]
train_x <- training_data_imputed[train_index, ] %>% dplyr::select(-PH)

#test set
test_y <- training_data_imputed$PH[-train_index]
test_x  <- training_data_imputed[-train_index, ] %>% dplyr::select(-PH)
```

1.5.1 Centering and scaling

We didn't scale anything individually in this section, since every package should have an option to center/scale automatically, and use a Box-Cox for the data with less normal distributions.

1.5.2 Recap

- The 80/20 split chunk above is `eval=FALSE` and is kept only as a historical reference. The modeling pipeline uses 10-fold CV on `training_data_no_na` directly, so `train_x` / `test_x` etc. are not created on knit.
- Centering, scaling, and Box-Cox transformations are handled inside each model's `preProc` options rather than included in here.

1.6 Modeling approach

Recap located at the end of the section. Skip there to jump past the workflow.

Before fitting models, here are a few setup decisions that we wanted to have at the top of this section.

`caret` In class and in Kuhn and Johnson, we used `caret::preProcess` for imputation, transformations, and scaling inside the resampling loop. We're sticking with that for the sake of sticking with the class standard and what we've been learning how to use effectively. However we did stumble upon the `recipes` package made by the same author later. Potentially worth looking into in the future.

Per-model feature sets. We know we discussed this in the previosu section but saying again. Linear and neural network models use the correlation-pruned 20-predictor set (`training_data_no_na`). Tree-based and rule-based models (where collinearity is fine) use the wider 31-predictor set (`training_data_wide_no_na`).

Imputation inside resampling. For model fitting, we're starting from `training_data_no_na` (which has not been imputed) and we're letting caret handle imputation inside each CV.

- `preProcess(method = c("YeoJohnson", "center", "scale", "knnImpute"))`
- `trainControl(method = "cv", number = 10)`.

We did this so that the imputation parameters are estimated per fold rather than once on the full training set, which avoids bias or data leakage from imputing before the split.

Brand.Code handling. Rather than imputing the 120 missing brand values, we opted to treat missing as its own factor level called `"Unknown"`. In our EDA section, we saw that the rows missing a brand did have a bit of a different PH profile from known-Brand rows, so we're treating the missingness itself carries signal worth preserving.

Shared infrastructure for the model comparison. Every model we fit shares the same `controlObject` (10-fold CV) and is trained immediately after `set.seed(624)`. This way, we're getting the same CV folds across models, which is what `resamples()` requires for side-by-side comparisons of the models at the end. Each fitted model also gets appended to a `models` list along the way.

```
set.seed(624)

# treating Brand.Code NAs as their own factor level (I guess this is called informative-missingness)
training_for_modeling <- training_data_no_na %>%
  mutate(Brand.Code = factor(ifelse(is.na(Brand.Code) | Brand.Code == "", "Unknown", Brand.Code)))

# shared control object for every model.
# We're using this object= and using set.seed(624) right before each train() call
# This way we can compare our CV folds to one another
controlObject <- trainControl(method = "cv", number = 10,
                              savePredictions = "final")

# simple list to accumulate fitted models.
# goal was to make the resamples() comparison at the end easy
models <- list()

# made a running table of CV metrics
# the set up is each model section appends one row after fitting
model_metrics <- tibble(model = character(),
                        cv_RMSE = numeric(),
                        cv_Rsquared = numeric(),
                        cv_MAE = numeric())
```

Per-model output pattern. To keep model sections comparable across the rest of this report, each one produces the same chunks in the same order. Anyone adding a new model, here's the pattern we're using:

1. Train the model with `set.seed(624)` and the shared `controlObject`. Print the trained object so caret shows CV RMSE / R-squared / MAE. Append `getTrainPerf(model)` to `model_metrics` to add to the final comparison table for the end.
2. `plot(varImp(model), top = 20)`: to get predictors matter most.
3. *Linear only:* we're looking at a coefficient table sorted by absolute t-statistic becuase it gives direction and strength of each predictor's effect on PH. We're skipping this for tree-based and rule-based models.
4. A small residual summary table (mean, SD, max residual, plus predicted vs observed range).

1.6.1 Recap

- Linear-family + NN models are using the 20-predictor narrow set. Tree-based + rule-based models are using the 31-predictor wide set.

- The pre-imputed `training_data_imputed` is an inspection / sanity-check artifact for imputation. The actual model fitting startswith data that has not been imputed yet (`training_data_no_na`) and lets caret impute inside each CV fold to avoid data leakage.
- `Brand.Code` missing values have been kept as an "Unknown" factor level rather than imputed.
- All models share `controlObject` and are trained after `set.seed(624)`. This was so the CV folds would be the same for all models, which is what `resamples()` needs to compare the models at the end.
- Per-model output pattern is documented above; each section appends one row to `model_metrics` so the final comparison table builds itself as we go.

1.7 Linear baseline

Recap located at the end of the section. Skip there to jump past the code/workflow.

First model: ordinary linear regression. Mostly useful as a baseling for other models to beat.

Numeric pre-processing. Inside the `train()` call, we're using YeoJohnson rather than Box-Cox because four predictors in the narrow set (`Mnf.Flow`, `Hyd.Pressure2`, `PSC.Fill`, `PSC.CO2`) had non-positive values. YeoJohnson is able to handle that data. After YeoJohnson, predictors are centered, scaled, and KNN-imputed inside each CV fold.

1.7.1 Linear regression (lm)

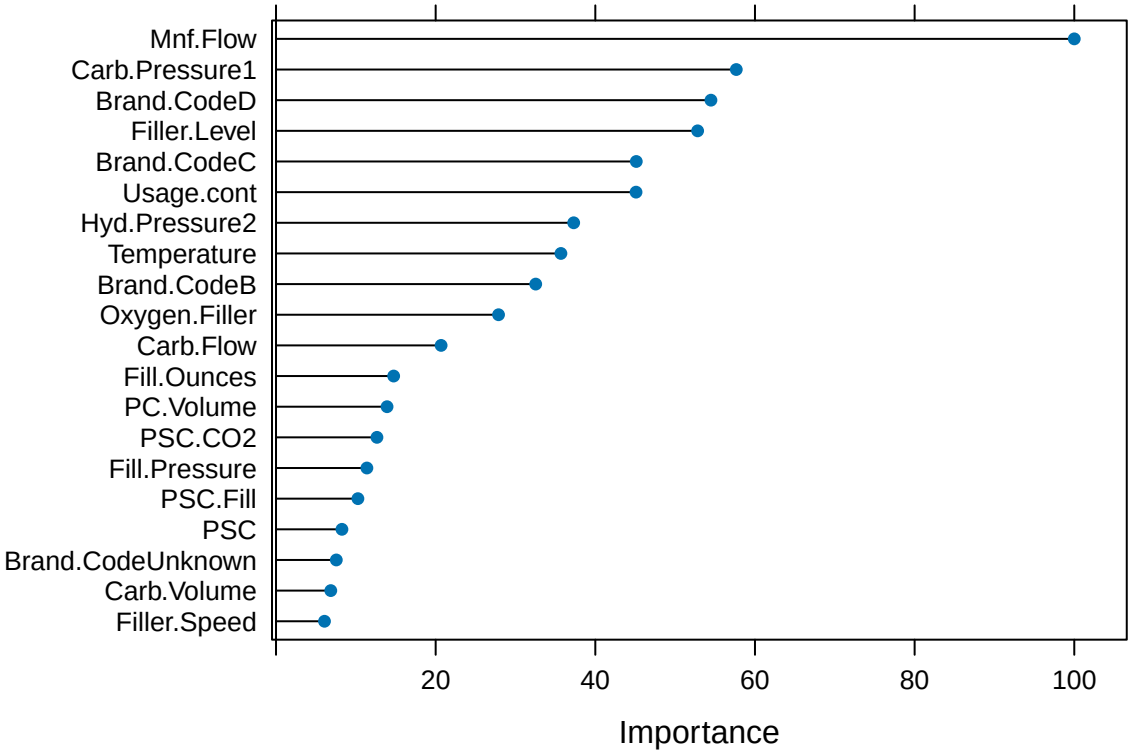
```
set.seed(624)

lm_model <- train(
  PH ~ .,
  data = training_for_modeling,
  method = "lm",
  trControl = controlObject,
  preProcess = c("YeoJohnson", "center", "scale", "knnImpute"),
  na.action = na.pass
)

models$lm <- lm_model

# append CV metrics to the running comparison table
model_metrics <- bind_rows(
  model_metrics,
  getTrainPerf(lm_model) %>%
    transmute(model = "lm",
              cv_RMSE = TrainRMSE,
              cv_Rsquared = TrainRsquared,
              cv_MAE = TrainMAE)
)

lm_model
```

```
## Linear Regression
##
## 2565 samples
##   20 predictor
```

```
##
## Pre-processing: Yeo-Johnson transformation (23), centered (23), scaled
##  (23), nearest neighbor imputation (23)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2309, 2308, 2308, 2309, 2308, 2309, ...
## Resampling results:
##
##   RMSE      Rsquared   MAE
##   0.135323  0.3819844  0.1060805
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

Variable importance. Which predictors matter most for pH under linear assumptions.

```
plot(varImp(lm_model), top = 20)
```



Coefficients. Each coefficient tells us how a predictor moves pH: positive pushes pH up, negative pushes it down, and bigger magnitudes mean stronger effects. Sorted by absolute t-statistic. (The t-statistic is used to sort by becuase it ranks the predictors by how confidently we can say their effect is real).

```
lm_coefs <- summary(lm_model$finalModel)$coefficients %>%
  as.data.frame() %>%
  tibble::rownames_to_column("predictor") %>%
  arrange(desc(abs(`t value`)))
kable(lm_coefs, digits = 4)
```

predictor	Estimate	Std. Error	t value	Pr(>|t|)
(Intercept)	8.5458	0.0027	3204.4593	0.0000
Mnf.Flow	-0.0783	0.0051	-15.3528	0.0000
Carb.Pressure1	0.0291	0.0033	8.8721	0.0000
Brand.CodeD	0.0382	0.0046	8.3867	0.0000
Filler.Level	0.0314	0.0039	8.1312	0.0000
Brand.CodeC	-0.0285	0.0041	-6.9551	0.0000
Usage.cont	-0.0236	0.0034	-6.9504	0.0000
Hyd.Pressure2	0.0229	0.0040	5.7547	0.0000
Temperature	-0.0172	0.0031	-5.5098	0.0000
Brand.CodeB	0.0259	0.0051	5.0266	0.0000
Oxygen.Filler	-0.0141	0.0033	-4.3139	0.0000
Carb.Flow	0.0123	0.0038	3.2128	0.0013
Fill.Ounces	-0.0065	0.0028	-2.3031	0.0214
PC.Volume	-0.0069	0.0032	-2.1764	0.0296
PSC.CO2	-0.0055	0.0028	-1.9824	0.0475
Fill.Pressure	0.0056	0.0031	1.7896	0.0736
PSC.Fill	-0.0045	0.0028	-1.6172	0.1060
PSC	-0.0037	0.0029	-1.3111	0.1900
Brand.CodeUnknown	-0.0041	0.0034	-1.2032	0.2290
Carb.Volume	-0.0080	0.0073	-1.0976	0.2725
Filler.Speed	-0.0035	0.0035	-0.9768	0.3288
Air.Pressurer	-0.0026	0.0028	-0.9043	0.3659
Carb.Temp	0.0031	0.0097	0.3154	0.7525
Carb.Pressure	0.0005	0.0107	0.0475	0.9621

Residual summary. A quick look at how big the model's errors are (using out-of-fold predictions) and how well its predictions cover the observed pH range.

```
lm_preds <- lm_model$pred
lm_preds$residual <- lm_preds$obs - lm_preds$pred

residual_summary <- tibble(
  metric = c("Mean residual",
             "SD residual",
             "Max |residual|",
             "99th pct |residual|",
             "Cor(predicted, residual)",
             "Predicted PH range",
             "Observed PH range"),
  value = c(
    sprintf("%.5f", mean(lm_preds$residual)),
    sprintf("%.4f", sd(lm_preds$residual)),
    sprintf("%.4f", max(abs(lm_preds$residual))),
    sprintf("%.4f", quantile(abs(lm_preds$residual), 0.99)),
    sprintf("%.4f", cor(lm_preds$pred, lm_preds$residual)),
    sprintf("%.3f to %.3f", min(lm_preds$pred), max(lm_preds$pred)),
    sprintf("%.3f to %.3f", min(lm_preds$obs), max(lm_preds$obs))
  )
)
kable(residual_summary)
```

metric	value
Mean residual	0.00010
SD residual	0.1354
Max |residual|	0.5222
99th pct |residual|	0.3819
Cor(predicted, residual)	-0.0122
Predicted PH range	8.243 to 8.821
Observed PH range	7.880 to 8.940

1.7.2 Recap

- Fit linear regression with 10-fold CV. The fitted model lives in `models$lm`, and the CV metrics got appended to `model_metrics`.
- Cross-validated RMSE about 0.135, R-squared about 0.38, MAE about 0.106. The residuals look pretty well behaved (mean-zero, basically uncorrelated with the predicted values), so the model is doing about as well as a linear approach can on this data.
- The predicted pH values span 8.24 to 8.83, while the actual observed pH values span 7.88 to 8.94. So the linear model can't quite reach the extremes; it leans toward predicting middle-ish values when the truth is at the edges. Nonlinear models should do better on that.
- Top predictive factors (sorted by absolute t-statistic): `Mnf.Flow` is the dominant one, then `Carb.Pressure1`, `Brand.CodeD`, `Filler.Level`, `Brand.CodeC`, `Usage.cont`, `Hyd.Pressure2`, and `Temperature`.
- For the business-side interpretation: higher `Mnf.Flow`, `Usage.cont`, `Temperature`, and `Oxygen.Filler` are associated with pushing pH down. Higher `Carb.Pressure1`, `Filler.Level`, and `Hyd.Pressure2` are associated with pushing pH up. Brand D has the highest baseline pH, Brand C has the lowest, with a spread of about 0.07 pH units between them.

1.8 Elastic net

> Recap located at the end of the section. Skip there to jump past the code/workflow.

Next model we're trying: elastic net via `glmnet`. This seemed like a good option, because it is the same linear-family setup as lm, plus it has a coefficient-shrinkage penalty that blends ridge (L2) and lasso (L1). It seemed like a better option than doing either ridge or lasso on its own. caret tunes both alpha (0 = pure ridge, 1 = pure lasso, in between = blended) and lambda (penalty strength), so the tuned model lands wherever on the ridge-to-lasso spectrum the data prefers.

1.8.1 Elastic net (glmnet)

```
set.seed(624)

elastic_model <- train(
  PH ~ .,
  data = training_for_modeling,
  method = "glmnet",
  trControl = controlObject,
  preProcess = c("YeoJohnson", "center", "scale", "knnImpute"),
  na.action = na.pass,
  tuneLength = 10
```

```
)

models$elastic_net <- elastic_model

# append CV metrics to the running comparison table
model_metrics <- bind_rows(
  model_metrics,
  getTrainPerf(elastic_model) %>%
    transmute(model = "elastic_net",
              cv_RMSE = TrainRMSE,
              cv_Rsquared = TrainRsquared,
              cv_MAE = TrainMAE)
)

elastic_model
```

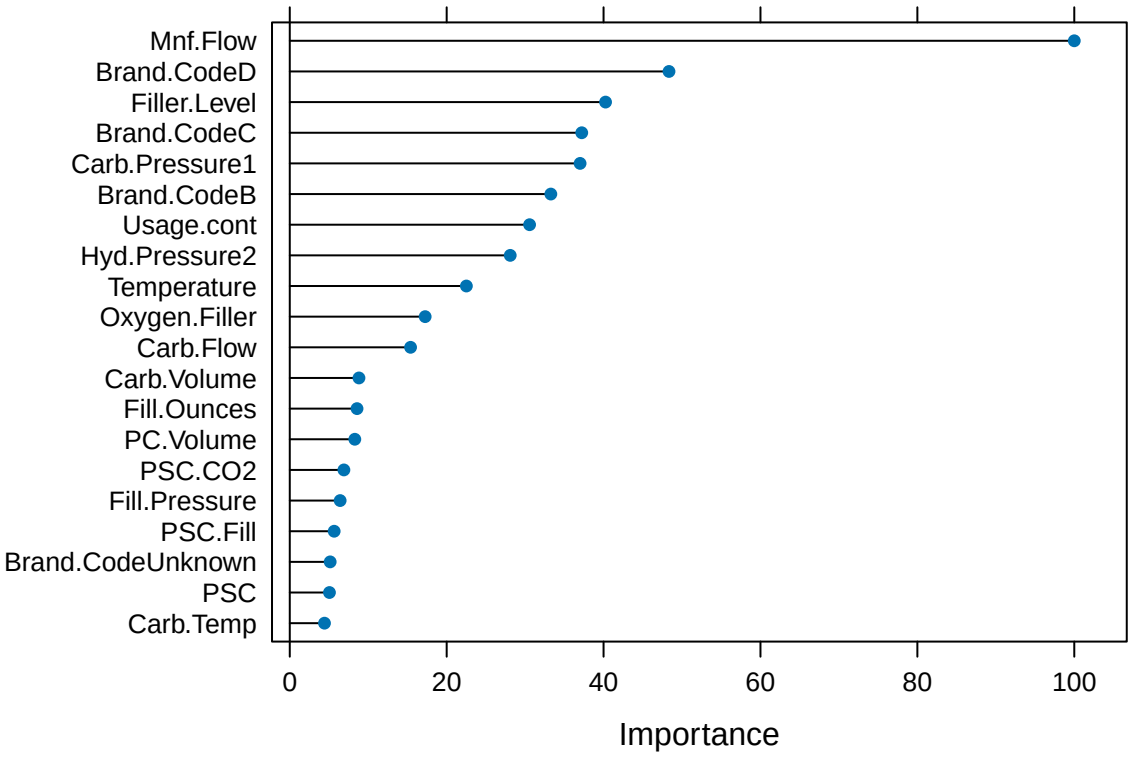
```
## glmnet
##
## 2565 samples
##   20 predictor
##
## Pre-processing: Yeo-Johnson transformation (23), centered (23), scaled
##  (23), nearest neighbor imputation (23)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2309, 2308, 2308, 2309, 2308, 2309, ...
## Resampling results across tuning parameters:
##
##   alpha  lambda        RMSE       Rsquared   MAE
##   0.1    8.205501e-05  0.1352969  0.3821881  0.1060736
##   0.1    1.895577e-04  0.1352969  0.3821881  0.1060736
##   0.1    4.379029e-04  0.1352967  0.3821888  0.1060734
##   0.1    1.011613e-03  0.1352888  0.3822386  0.1060833
##   0.1    2.336956e-03  0.1352949  0.3822044  0.1061194
##   0.1    5.398672e-03  0.1354210  0.3814225  0.1062983
##   0.1    1.247163e-02  0.1359829  0.3781338  0.1069120
##   0.1    2.881109e-02  0.1378184  0.3676227  0.1087592
##   0.1    6.655735e-02  0.1417918  0.3503984  0.1124650
##   0.2    8.205501e-05  0.1352949  0.3821971  0.1060705
##   0.2    1.895577e-04  0.1352949  0.3821971  0.1060705
##   0.2    4.379029e-04  0.1352918  0.3822272  0.1060711
##   0.2    1.011613e-03  0.1352845  0.3822709  0.1060817
##   0.2    2.336956e-03  0.1353236  0.3819867  0.1061460
##   0.2    5.398672e-03  0.1355763  0.3803218  0.1064271
##   0.2    1.247163e-02  0.1364983  0.3748519  0.1074226
##   0.2    2.881109e-02  0.1393419  0.3578141  0.1101886
##   0.2    6.655735e-02  0.1452406  0.3306459  0.1155264
##   0.3    8.205501e-05  0.1352948  0.3821931  0.1060698
##   0.3    1.895577e-04  0.1352948  0.3821931  0.1060698
##   0.3    4.379029e-04  0.1352884  0.3822508  0.1060692
##   0.3    1.011613e-03  0.1352888  0.3822301  0.1060889
##   0.3    2.336956e-03  0.1353692  0.3816357  0.1061863
##   0.3    5.398672e-03  0.1357395  0.3792223  0.1065789
##   0.3    1.247163e-02  0.1371641  0.3702854  0.1080726
```

```
##   0.3    2.881109e-02  0.1409657  0.3469195  0.1117011
##   0.3    6.655735e-02  0.1482228  0.3176529  0.1180306
##   0.4    8.205501e-05  0.1352939  0.3822017  0.1060665
##   0.4    1.895577e-04  0.1352936  0.3822045  0.1060664
##   0.4    4.379029e-04  0.1352861  0.3822642  0.1060680
##   0.4    1.011613e-03  0.1352978  0.3821529  0.1060979
##   0.4    2.336956e-03  0.1354221  0.3812355  0.1062318
##   0.4    5.398672e-03  0.1359219  0.3780308  0.1067725
##   0.4    1.247163e-02  0.1379422  0.3645416  0.1088196
##   0.4    2.881109e-02  0.1426845  0.3342049  0.1131936
##   0.4    6.655735e-02  0.1516498  0.2956408  0.1209188
##   0.5    8.205501e-05  0.1352950  0.3821991  0.1060671
##   0.5    1.895577e-04  0.1352930  0.3822194  0.1060659
##   0.5    4.379029e-04  0.1352847  0.3822703  0.1060679
##   0.5    1.011613e-03  0.1353115  0.3820366  0.1061091
##   0.5    2.336956e-03  0.1354835  0.3807707  0.1062798
##   0.5    5.398672e-03  0.1361376  0.3765940  0.1069982
##   0.5    1.247163e-02  0.1387742  0.3581356  0.1096054
##   0.5    2.881109e-02  0.1440213  0.3261374  0.1143598
##   0.5    6.655735e-02  0.1547604  0.2760646  0.1236015
##   0.6    8.205501e-05  0.1352950  0.3822063  0.1060660
##   0.6    1.895577e-04  0.1352916  0.3822337  0.1060648
##   0.6    4.379029e-04  0.1352859  0.3822556  0.1060711
##   0.6    1.011613e-03  0.1353280  0.3818988  0.1061223
##   0.6    2.336956e-03  0.1355465  0.3803100  0.1063358
##   0.6    5.398672e-03  0.1363875  0.3748844  0.1072553
##   0.6    1.247163e-02  0.1394883  0.3528928  0.1102807
##   0.6    2.881109e-02  0.1451476  0.3210245  0.1153005
##   0.6    6.655735e-02  0.1576892  0.2544858  0.1260112
##   0.7    8.205501e-05  0.1352953  0.3822031  0.1060662
##   0.7    1.895577e-04  0.1352901  0.3822439  0.1060639
##   0.7    4.379029e-04  0.1352879  0.3822350  0.1060745
##   0.7    1.011613e-03  0.1353454  0.3817574  0.1061353
##   0.7    2.336956e-03  0.1356063  0.3798965  0.1063980
##   0.7    5.398672e-03  0.1366529  0.3730540  0.1075254
##   0.7    1.247163e-02  0.1401931  0.3477693  0.1109172
##   0.7    2.881109e-02  0.1464599  0.3134222  0.1163966
##   0.7    6.655735e-02  0.1606661  0.2229432  0.1286070
##   0.8    8.205501e-05  0.1352955  0.3822027  0.1060658
##   0.8    1.895577e-04  0.1352885  0.3822534  0.1060632
##   0.8    4.379029e-04  0.1352905  0.3822109  0.1060781
##   0.8    1.011613e-03  0.1353639  0.3816088  0.1061501
##   0.8    2.336956e-03  0.1356712  0.3794523  0.1064668
##   0.8    5.398672e-03  0.1369502  0.3709247  0.1078254
##   0.8    1.247163e-02  0.1409764  0.3416701  0.1116031
##   0.8    2.881109e-02  0.1479698  0.3023956  0.1176199
##   0.8    6.655735e-02  0.1630316  0.2026599  0.1307093
##   0.9    8.205501e-05  0.1352944  0.3822071  0.1060651
##   0.9    1.895577e-04  0.1352876  0.3822582  0.1060628
##   0.9    4.379029e-04  0.1352940  0.3821791  0.1060822
##   0.9    1.011613e-03  0.1353852  0.3814395  0.1061664
##   0.9    2.336956e-03  0.1357447  0.3789511  0.1065437
##   0.9    5.398672e-03  0.1372838  0.3684240  0.1081469
##   0.9    1.247163e-02  0.1418270  0.3345759  0.1123365
```

```
##   0.9    2.881109e-02  0.1495628  0.2889571  0.1189366
##   0.9    6.655735e-02  0.1649994  0.2025497  0.1324476
##   1.0    8.205501e-05  0.1352943  0.3822066  0.1060644
##   1.0    1.895577e-04  0.1352870  0.3822609  0.1060626
##   1.0    4.379029e-04  0.1352986  0.3821371  0.1060865
##   1.0    1.011613e-03  0.1354075  0.3812623  0.1061845
##   1.0    2.336956e-03  0.1358259  0.3783950  0.1066304
##   1.0    5.398672e-03  0.1376515  0.3655664  0.1085026
##   1.0    1.247163e-02  0.1425809  0.3284122  0.1130134
##   1.0    2.881109e-02  0.1509621  0.2776060  0.1201083
##   1.0    6.655735e-02  0.1673016  0.2025497  0.1344976
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were alpha = 0.2 and lambda = 0.001011613.
```

Variable importance.

```
plot(varImp(elastic_model), top = 20)
```



Coefficients at the tuned lambda. glmnet doesn't expose t-statistics (penalized regression invalidates classical inference), so we report just the coefficients sorted by absolute size. Coefficients exactly equal to zero have been shrunk out by the lasso component.

```
elastic_coefs <- coef(elastic_model$finalModel,
                      s = elastic_model$bestTune$lambda) %>%
  as.matrix() %>%
  as.data.frame()
colnames(elastic_coefs) <- "coefficient"
elastic_coefs <- elastic_coefs %>%
  tibble::rownames_to_column("predictor") %>%
  arrange(desc(abs(coefficient)))
kable(elastic_coefs, digits = 4)
```

predictor	coefficient
(Intercept)	8.5458
Mnf.Flow	-0.0765
Brand.CodeD	0.0370
Filler.Level	0.0308
Brand.CodeC	-0.0285
Carb.Pressure1	0.0283
Brand.CodeB	0.0254
Usage.cont	-0.0234
Hyd.Pressure2	0.0215
Temperature	-0.0172
Oxygen.Filler	-0.0132
Carb.Flow	0.0118
Carb.Volume	-0.0067
Fill.Ounces	-0.0066
PC.Volume	-0.0063
PSC.CO2	-0.0053
Fill.Pressure	0.0049
PSC.Fill	-0.0043
Brand.CodeUnknown	-0.0039
PSC	-0.0039
Carb.Temp	0.0034
Filler.Speed	-0.0029
Air.Pressurer	-0.0025
Carb.Pressure	0.0000

Residual summary.

```
elastic_preds <- elastic_model$pred
elastic_preds$residual <- elastic_preds$obs - elastic_preds$pred

residual_summary_elastic <- tibble(
  metric = c("Mean residual",
             "SD residual",
             "Max |residual|",
             "99th pct |residual|",
             "Cor(predicted, residual)",
             "Predicted PH range",
             "Observed PH range"),
  value = c(
    sprintf("%.5f", mean(elastic_preds$residual)),
```

```
    sprintf("%.4f", sd(elastic_preds$residual)),
    sprintf("%.4f", max(abs(elastic_preds$residual))),
    sprintf("%.4f", quantile(abs(elastic_preds$residual), 0.99)),
    sprintf("%.4f", cor(elastic_preds$pred, elastic_preds$residual)),
    sprintf("%.3f to %.3f", min(elastic_preds$pred), max(elastic_preds$pred)),
    sprintf("%.3f to %.3f", min(elastic_preds$obs), max(elastic_preds$obs))
  )
)
kable(residual_summary_elastic)
```

metric	value
Mean residual	0.00009
SD residual	0.1354
Max |residual|	0.5211
99th pct |residual|	0.3830
Cor(predicted, residual)	-0.0025
Predicted PH range	8.247 to 8.814
Observed PH range	7.880 to 8.940

1.8.2 Recap

- Fit elastic net with 10-fold CV, letting caret tune over alpha (0 = ridge, 1 = lasso) and lambda. The fitted model lives in `models$elastic_net`, with CV metrics appended to `model_metrics`.
- The tuner landed at alpha = 0.2 (mostly ridge with a sliver of lasso) and lambda = 0.0010, which is very small. Basically the data didn't want much regularization at all; elastic net pretty much settled on "lm with a faint ridge nudge".
- CV metrics came in nearly identical to lm: RMSE about 0.1354, R-squared about 0.381, MAE about 0.106. The penalized linear family probably can't do much better than this on our dataset.
- Same top predictors in the same order as lm, with coefficients shrunk slightly toward zero. Only `Carb.Pressure` got zeroed out by the lasso component (in lm it had p = 0.96 and a near-zero estimate, so this seemed like the right call).
- Same range compression as lm (predicted 8.24-8.82 vs observed 7.88-8.94). The linear ceiling looks real. Going to need a nonlinear method to improve from here. Tree-based is up next.

1.9 Random forest

Recap located at the end of the section. Skip there to jump past the code/workflow.

First tree-based model: random forest via the `randomForest` engine, wrapped by caret. Trees split the predictor space recursively and don't require Box-Cox or scaling; they also handle collinearity natively, so this is where the wider 31-predictor set earns its keep. Random forest specifically fits many decorrelated trees on bootstrap samples and averages their predictions, which tends to dramatically reduce variance compared to a single tree.

Wide-set setup. We mirror the Brand.Code "Unknown" treatment we used for the linear-family models, but on the wider feature set (`training_data_wide_no_na`).

```
training_for_modeling_wide <- training_data_wide_no_na %>%
  mutate(Brand.Code = factor(ifelse(is.na(Brand.Code) | Brand.Code == "", "Unknown", Brand.Code)))
```

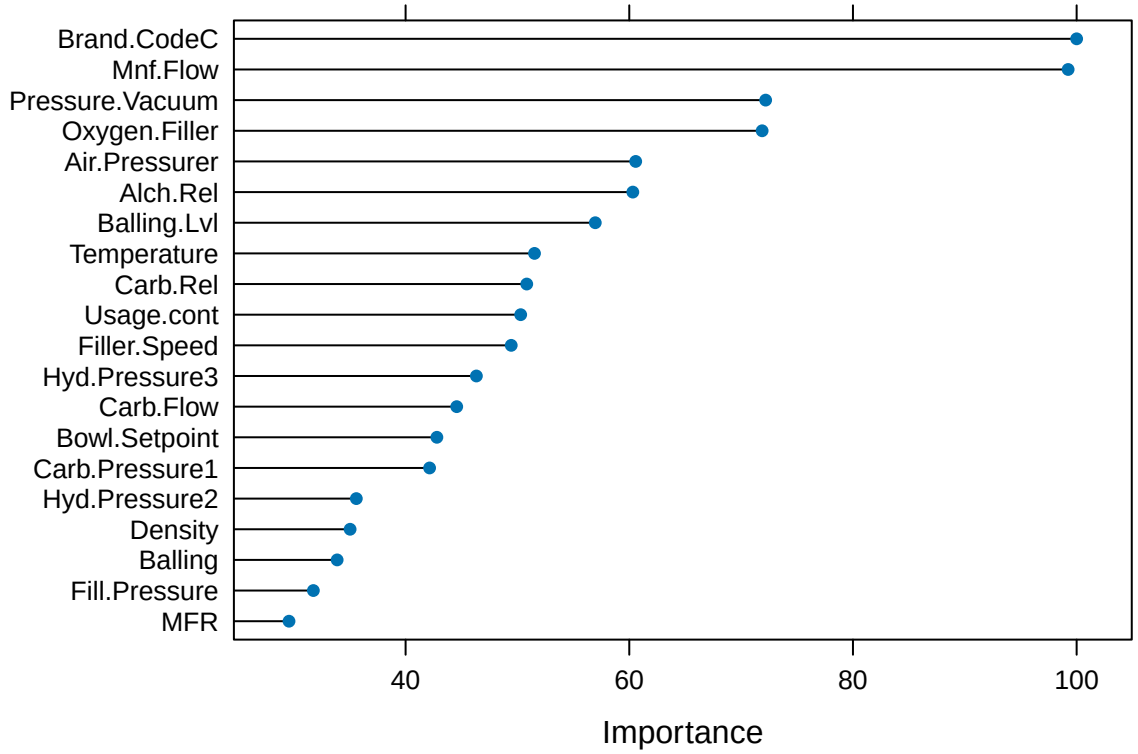
1.9.1 Random forest

```
set.seed(624)

rf_model <- train(
  PH ~ .,
  data = training_for_modeling_wide,
  method = "rf",
  trControl = controlObject,
  preProcess = c("knnImpute"),
  na.action = na.pass,
  tuneLength = 5,
  ntree = 200,
  importance = TRUE
)

models$random_forest <- rf_model

# append CV metrics to the running comparison table
model_metrics <- bind_rows(
  model_metrics,
  getTrainPerf(rf_model) %>%
    transmute(model = "random_forest",
              cv_RMSE = TrainRMSE,
              cv_Rsquared = TrainRsquared,
              cv_MAE = TrainMAE)
)

rf_model
```

```
## Random Forest
##
## 2565 samples
##   31 predictor
##
## Pre-processing: nearest neighbor imputation (34), centered (34), scaled (34)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2309, 2308, 2308, 2309, 2308, 2309, ...
## Resampling results across tuning parameters:
##
##   mtry  RMSE        Rsquared   MAE
##    2    0.10836944  0.6475309  0.08201382
##   10    0.09602037  0.7051168  0.07021546
##   18    0.09352415  0.7164101  0.06814411
##   26    0.09262867  0.7176686  0.06724975
##   34    0.09264205  0.7145657  0.06679322
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 26.
```

Variable importance.

```
plot(varImp(rf_model), top = 20)
```



Residual summary.

```
rf_preds <- rf_model$pred
rf_preds$residual <- rf_preds$obs - rf_preds$pred

residual_summary_rf <- tibble(
  metric = c("Mean residual",
             "SD residual",
             "Max |residual|",
             "99th pct |residual|",
             "Cor(predicted, residual)",
             "Predicted PH range",
             "Observed PH range"),
  value = c(
    sprintf("%.5f", mean(rf_preds$residual)),
    sprintf("%.4f", sd(rf_preds$residual)),
    sprintf("%.4f", max(abs(rf_preds$residual))),
    sprintf("%.4f", quantile(abs(rf_preds$residual), 0.99)),
    sprintf("%.4f", cor(rf_preds$pred, rf_preds$residual)),
    sprintf("%.3f to %.3f", min(rf_preds$pred), max(rf_preds$pred)),
    sprintf("%.3f to %.3f", min(rf_preds$obs), max(rf_preds$obs))
  )
```

```
)
kable(residual_summary_rf)
```

metric	value
Mean residual	0.00016
SD residual	0.0927
Max |residual|	0.4936
99th pct |residual|	0.3129
Cor(predicted, residual)	0.1688
Predicted PH range	8.106 to 8.825
Observed PH range	7.880 to 8.940

1.9.2 Recap

- Random forest trained via 10-fold CV, on the wider 31-predictor feature set. Fitted model stored in `models$random_forest`, CV metrics appended to `model_metrics`.
- Tuned `mtry = 26` (out of candidates 2, 10, 18, 26, 34). caret's default heuristic for regression is `p/3` (about 11 for our wider set), so the data preferred sampling more predictors per split than the default. Performance was nearly flat between mtry = 26 and 34.
- CV metrics: RMSE 0.0926, R-squared 0.7177, MAE 0.0672. About a third off the linear-family RMSE and nearly doubling R-squared.
- Top predictive factors: Brand.CodeC (100) and Mnf.Flow (99) nearly tied at the top, then Pressure.Vacuum (72), Oxygen.Filler (72), Air.Pressurer (61), Alch.Rel (60), Balling.Lvl (57), Temperature (52). Several of these (Pressure.Vacuum, Alch.Rel, Balling.Lvl, Carb.Rel) were dropped from the narrow set during the correlation prune, so the wider feature set definiyely created more opportunity here.
- Random forest's predictions span PH values from 8.11 to 8.83, wider than the linear models did (8.24 to 8.83). The actual observed PH values span 7.88 to 8.94, wider still. So random forest gets closer to the extreme PH values than the linear models, but it still can't quite reach them; it leans toward predicting middle-of-the-pack values when the truth is at the edges.
- There's a small systematic pattern in random forest's errors: when the model predicts a higher PH, the truth tends to be even higher; when it predicts a lower PH, the truth tends to be even lower. The correlation between predicted and error is 0.169, small but not zero. Same idea as the range-issue above (the model leans toward middle predictions), just less severe than what the linear models showed.

1.10 Gradient boosting

> Recap located at the end of the section. Skip there to jump past the code/workflow.

Next tree-based model: gradient boosted regression trees via `gbm`, wrapped by caret. gbm fits a sequence of shallow trees where each tree corrects the previous tree's residuals; the additive ensemble tends to edge out random forest on tabular data with moderate complexity. Like RF, gbm doesn't need scaling or Box-Cox, and it uses the wider 31-predictor feature set so it can use the variables we pruned out for the linear-family models.

1.10.1 Gradient boosting (gbm)

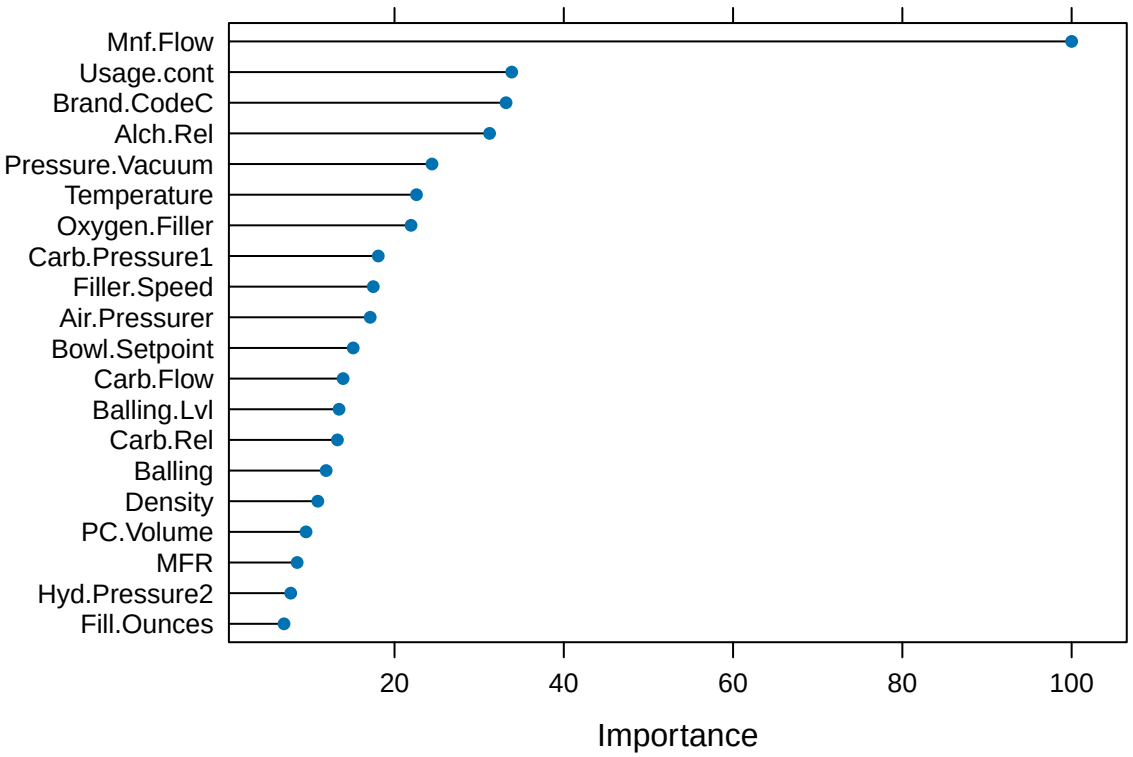
```r
set.seed(624)

gbm_model <- train(
  PH ~ .,
  data = training_for_modeling_wide,
  method = "gbm",
  trControl = controlObject,
  preProcess = c("knnImpute"),
  na.action = na.pass,
  tuneLength = 5,
  verbose = FALSE
)

models$gbm <- gbm_model

# append CV metrics to the running comparison table
model_metrics <- bind_rows(
  model_metrics,
  getTrainPerf(gbm_model) %>%
    transmute(model = "gbm",
              cv_RMSE = TrainRMSE,
              cv_Rsquared = TrainRsquared,
              cv_MAE = TrainMAE)
)

gbm_model
```

```
## Stochastic Gradient Boosting
##
## 2565 samples
##   31 predictor
##
## Pre-processing: nearest neighbor imputation (34), centered (34), scaled (34)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2309, 2308, 2308, 2309, 2308, 2309, ...
## Resampling results across tuning parameters:
##
##   interaction.depth  n.trees  RMSE       Rsquared   MAE
##   1                   50      0.1347471  0.4054205  0.10671216
##   1                  100      0.1301928  0.4381286  0.10291059
##   1                  150      0.1278529  0.4565340  0.10070481
##   1                  200      0.1266412  0.4640929  0.09913740
##   1                  250      0.1259596  0.4677972  0.09834841
##   2                   50      0.1269948  0.4699914  0.10037830
##   2                  100      0.1218318  0.5060873  0.09517020
##   2                  150      0.1190883  0.5255693  0.09221965
##   2                  200      0.1172401  0.5384435  0.09033028
##   2                  250      0.1161839  0.5455670  0.08926316
##   3                   50      0.1224024  0.5075129  0.09612938
##   3                  100      0.1169598  0.5433132  0.09064043
##   3                  150      0.1141460  0.5619903  0.08800436
##   3                  200      0.1127828  0.5712665  0.08656881
##   3                  250      0.1115276  0.5798266  0.08533088
```

```
##   4                  50      0.1190138  0.5335019  0.09268078
##   4                 100      0.1131954  0.5726500  0.08728230
##   4                 150      0.1106681  0.5895382  0.08459785
##   4                 200      0.1089566  0.6001547  0.08293913
##   4                 250      0.1076794  0.6088125  0.08172274
##   5                  50      0.1168357  0.5493290  0.09055053
##   5                 100      0.1115768  0.5836692  0.08555954
##   5                 150      0.1100452  0.5925252  0.08377814
##   5                 200      0.1084427  0.6035992  0.08200728
##   5                 250      0.1072945  0.6118269  0.08100282
##
## Tuning parameter 'shrinkage' was held constant at a value of 0.1
##
## Tuning parameter 'n.minobsinnode' was held constant at a value of 10
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were n.trees = 250, interaction.depth =
##  5, shrinkage = 0.1 and n.minobsinnode = 10.
```

Variable importance.

```
plot(varImp(gbm_model), top = 20)
```



Residual summary.

```
gbm_preds <- gbm_model$pred
gbm_preds$residual <- gbm_preds$obs - gbm_preds$pred

residual_summary_gbm <- tibble(
  metric = c("Mean residual",
             "SD residual",
             "Max |residual|",
             "99th pct |residual|",
             "Cor(predicted, residual)",
             "Predicted PH range",
             "Observed PH range"),
  value = c(
    sprintf("%.5f", mean(gbm_preds$residual)),
    sprintf("%.4f", sd(gbm_preds$residual)),
    sprintf("%.4f", max(abs(gbm_preds$residual))),
    sprintf("%.4f", quantile(abs(gbm_preds$residual), 0.99)),
    sprintf("%.4f", cor(gbm_preds$pred, gbm_preds$residual)),
    sprintf("%.3f to %.3f", min(gbm_preds$pred), max(gbm_preds$pred)),
    sprintf("%.3f to %.3f", min(gbm_preds$obs), max(gbm_preds$obs))
  )
)
kable(residual_summary_gbm)
```

metric	value
Mean residual	-0.00001
SD residual	0.1074
Max |residual|	0.5212
99th pct |residual|	0.3347
Cor(predicted, residual)	0.0277
Predicted PH range	8.147 to 8.880
Observed PH range	7.880 to 8.940

1.10.2 Recap

- Fit gradient boosting (gbm) with 10-fold CV on the wider 31-predictor feature set. The fitted model lives in `models$gbm`, with CV metrics appended to `model_metrics`.
- We tuned gbm by testing combinations of tree depth (1 through 5) and number of trees (50 through 250). The best combination turned out to be the biggest values we tested: depth 5 and 250 trees. Performance was still improving as we increased both, so gbm might have done even better if we had tested bigger settings. We didn't bother to expand the search though, since even at these settings gbm did not perform as well as random forest, and the gap was too large for a wider search to realistically close.
- CV metrics: RMSE 0.1073, R-squared 0.6118, MAE 0.0810. Better than the linear family but notably behind random forest.
- Top predictive factors are mostly the same set as random forest, just in a different order: `Mnf.Flow` (100, top), `Usage.cont` (34), `Brand.CodeC` (33), `Alch.Rel` (31), `Pressure.Vacuum` (24), `Temperature` (23), `Oxygen.Filler` (22), `Carb.Pressure1` (18). The `Mnf.Flow` + brand + `Pressure.Vacuum` + `Alch.Rel` + `Oxygen` + `Temperature` cluster stays consistent with random forest.
- Predicted pH range: 8.15 to 8.88 vs observed 7.88 to 8.94. Slightly wider than random forest (8.11 to 8.83) at the upper end. Residuals are pretty much mean-zero and uncorrelated with predicted values (correlation 0.028), so gbm is well-calibrated.

1.11 Support vector machine (radial kernel)

Recap located at the end of the section. Skip there to jump past the code/workflow.

Adding a nonlinear non-tree model to round out the family lineup: support vector machine with a radial basis function (RBF) kernel via `kernlab`, wrapped by caret. SVM-radial is sensitive to scaling so it uses the narrow feature set with the same preProcess pipeline as the linear-family models (YeoJohnson, center, scale, knnImpute). Unlike trees, SVM doesn't natively output variable importance; caret falls back to a filter-based metric (LOESS pseudo-R-squared on each predictor).

1.11.1 SVM-radial (svmRadial)

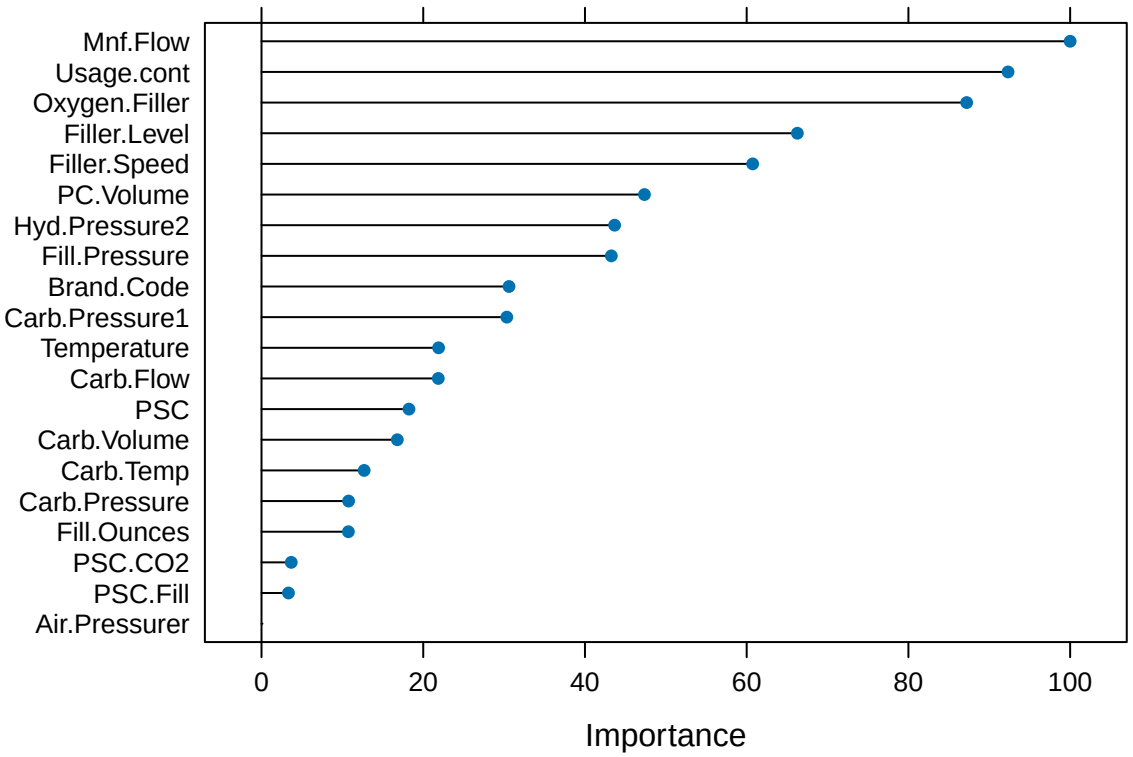
```
set.seed(624)

svm_model <- train(
  PH ~ .,
  data = training_for_modeling,
  method = "svmRadial",
  trControl = controlObject,
  preProcess = c("YeoJohnson", "center", "scale", "knnImpute"),
  na.action = na.pass,
  tuneLength = 10
)

models$svm_radial <- svm_model

# append CV metrics to the running comparison table
model_metrics <- bind_rows(
  model_metrics,
  getTrainPerf(svm_model) %>%
    transmute(model = "svm_radial",
              cv_RMSE = TrainRMSE,
              cv_Rsquared = TrainRsquared,
              cv_MAE = TrainMAE)
)

svm_model
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 2565 samples
##   20 predictor
##
## Pre-processing: Yeo-Johnson transformation (23), centered (23), scaled
##  (23), nearest neighbor imputation (23)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2309, 2308, 2308, 2309, 2308, 2309, ...
## Resampling results across tuning parameters:
##
##   C       RMSE       Rsquared   MAE
##     0.25  0.1270926  0.4615164  0.09621241
##     0.50  0.1254609  0.4735196  0.09454896
```

```
##      1.00  0.1240362  0.4845785  0.09310004
##      2.00  0.1228184  0.4944277  0.09201413
##      4.00  0.1228893  0.4963252  0.09200250
##      8.00  0.1235185  0.4952756  0.09258064
##     16.00  0.1259486  0.4829499  0.09452523
##     32.00  0.1309150  0.4589153  0.09847563
##     64.00  0.1377398  0.4296198  0.10351639
##    128.00  0.1481310  0.3896024  0.11151409
##
## Tuning parameter 'sigma' was held constant at a value of 0.02806381
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were sigma = 0.02806381 and C = 2.
```

Variable importance. (Filter-based since SVM doesn't have native importance.)

```
plot(varImp(svm_model), top = 20)
```



Residual summary.

```
svm_preds <- svm_model$pred
svm_preds$residual <- svm_preds$obs - svm_preds$pred

residual_summary_svm <- tibble(
  metric = c("Mean residual",
             "SD residual",
```

```
             "Max |residual|",
             "99th pct |residual|",
             "Cor(predicted, residual)",
             "Predicted PH range",
             "Observed PH range"),
  value = c(
    sprintf("%.5f", mean(svm_preds$residual)),
    sprintf("%.4f", sd(svm_preds$residual)),
    sprintf("%.4f", max(abs(svm_preds$residual))),
    sprintf("%.4f", quantile(abs(svm_preds$residual), 0.99)),
    sprintf("%.4f", cor(svm_preds$pred, svm_preds$residual)),
    sprintf("%.3f to %.3f", min(svm_preds$pred), max(svm_preds$pred)),
    sprintf("%.3f to %.3f", min(svm_preds$obs), max(svm_preds$obs))
  )
)
kable(residual_summary_svm)
```

metric	value
Mean residual	-0.00785
SD residual	0.1227
Max |residual|	0.5218
99th pct |residual|	0.3696
Cor(predicted, residual)	-0.0362
Predicted PH range	8.181 to 8.858
Observed PH range	7.880 to 8.940

1.11.2 Recap

- Fit SVM-radial with 10-fold CV on the narrow 20-predictor feature set with the full preProcess pipeline (YeoJohnson, center, scale, knnImpute). The fitted model lives in `models$svm_radial`, with CV metrics appended to `model_metrics`.
- Support vector machine (SVM) has a parameter called C that controls how complex the model gets. We tested C values from 0.25 to 128 (each one twice as big as the last). The best C turned out to be 2, which is in the middle of the values we tested. That's a good sign; if the best C had been at either extreme of the range, we'd worry we didn't search wide enough. Sigma (the parameter that controls the radial kernel) was held constant at 0.028.
- CV metrics: RMSE 0.1228, R-squared 0.4944, MAE 0.0920. Between the linear models and the tree-based models. The radial kernel (which lets SVM fit curved relationships instead of just straight lines) gets it past the linear models, but SVM is a single model while random forest and gradient boosting each combine many trees, which is the main reason those two outperform SVM here.
- Top predictive factors via LOESS pseudo-R-squared (this is SVM's filter-based importance, not a native one): `Mnf.Flow` (100), `Usage.cont` (92), `Oxygen.Filler` (87), `Filler.Level` (66), `Filler.Speed` (61), `PC.Volume` (47), `Hyd.Pressure2` (44), `Fill.Pressure` (43), `Brand.Code` (31), `Carb.Pressure1` (30). Worth noting this measures individual predictor strength against pH, not contribution to the SVM model itself.
- Predicted pH range: 8.18 to 8.86 vs observed 7.88 to 8.94. Similar compression pattern to the other models. Mean residual is a touch off zero (-0.0079), slightly more than the other models but not enough to suggest systematic bias.

1.12 Cubist

Recap located at the end of the section. Skip there to jump past the code/workflow.

Last model in the lineup: Cubist, a rule-based regression model wrapped by caret. Cubist partitions the data into rules (similar to a decision tree) but fits a linear regression model inside each rule's terminal node, then averages predictions across committees of rules. It uses the wider 31-predictor feature set to match the other tree-family models. The preProcess pipeline includes YeoJohnson and center/scale alongside knnImpute since the linear models inside each rule can benefit from transformed and scaled inputs.

1.12.1 Cubist

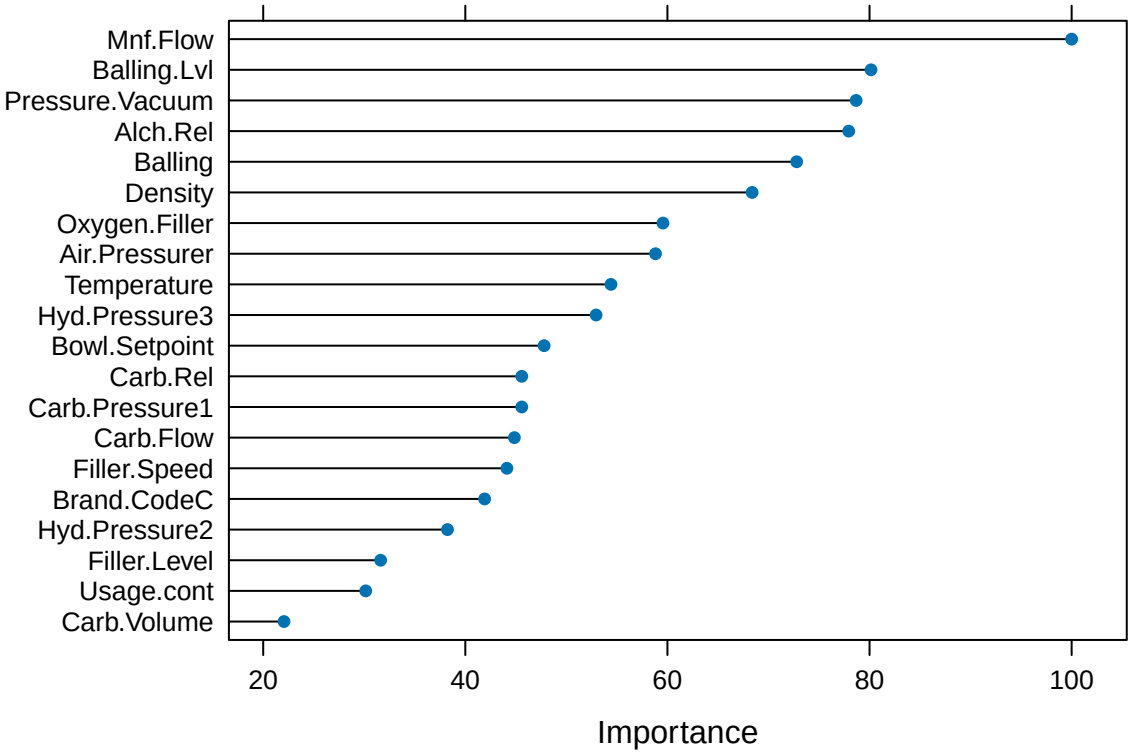
```
set.seed(624)

cubist_model <- train(
  PH ~ .,
  data = training_for_modeling_wide,
  method = "cubist",
  trControl = controlObject,
  preProcess = c("YeoJohnson", "center", "scale", "knnImpute"),
  na.action = na.pass,
  tuneLength = 10
)

models$cubist <- cubist_model

# append CV metrics to the running comparison table
model_metrics <- bind_rows(
  model_metrics,
  getTrainPerf(cubist_model) %>%
    transmute(model = "cubist",
              cv_RMSE = TrainRMSE,
              cv_Rsquared = TrainRsquared,
              cv_MAE = TrainMAE)
)

cubist_model
```

```
## Cubist
##
## 2565 samples
##   31 predictor
##
## Pre-processing: Yeo-Johnson transformation (34), centered (34), scaled
##  (34), nearest neighbor imputation (34)
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 2309, 2308, 2308, 2309, 2308, 2309, ...
## Resampling results across tuning parameters:
##
##   committees  neighbors  RMSE        Rsquared   MAE
##    1          0          0.11709102  0.5663388  0.07956496
##    1          5          0.11397926  0.5970794  0.07691098
```

```
##    1          9          0.11341373  0.5955684  0.07676179
##   10          0          0.09788118  0.6797203  0.07145907
##   10          5          0.09304572  0.7073057  0.06653415
##   10          9          0.09301498  0.7071579  0.06697869
##   20          0          0.09712281  0.6859118  0.07090767
##   20          5          0.09278729  0.7082375  0.06621061
##   20          9          0.09259569  0.7093332  0.06663032
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were committees = 20 and neighbors = 9.
```

Variable importance.

```
plot(varImp(cubist_model), top = 20)
```



Residual summary.

```
cubist_preds <- cubist_model$pred
cubist_preds$residual <- cubist_preds$obs - cubist_preds$pred

residual_summary_cubist <- tibble(
  metric = c("Mean residual",
             "SD residual",
             "Max |residual|",
             "99th pct |residual|",
```

```
              "Cor(predicted, residual)",
              "Predicted PH range",
              "Observed PH range"),
  value = c(
    sprintf("%.5f", mean(cubist_preds$residual)),
    sprintf("%.4f", sd(cubist_preds$residual)),
    sprintf("%.4f", max(abs(cubist_preds$residual))),
    sprintf("%.4f", quantile(abs(cubist_preds$residual), 0.99)),
    sprintf("%.4f", cor(cubist_preds$pred, cubist_preds$residual)),
    sprintf("%.3f to %.3f", min(cubist_preds$pred), max(cubist_preds$pred)),
    sprintf("%.3f to %.3f", min(cubist_preds$obs), max(cubist_preds$obs))
  )
)
kable(residual_summary_cubist)
```

metric	value
Mean residual	-0.00184
SD residual	0.0927
Max |residual|	0.5606
99th pct |residual|	0.3092
Cor(predicted, residual)	0.0125
Predicted PH range	8.089 to 9.050
Observed PH range	7.880 to 8.940

1.12.2 Recap

- Cubist trained via 10-fold CV on the wider 31-predictor feature set. Fitted model stored in `models$cubist`, CV metrics appended to `model_metrics`.
- CV residuals are centered near zero (mean residual -0.00174), so the model has almost no average bias. The small negative value means it's very slightly overpredicting PH on average, but the effect is tiny.
- Residual SD is 0.0927, basically tied with random forest's residual spread. Most prediction errors are within roughly a tenth of a PH unit.
- Max absolute residual is 0.7326 with the 99th percentile at 0.3035. Most errors are small, with a handful of outlier misses that may be unusual production conditions or observations where the process variables don't fully explain PH.
- Predicted PH range 8.047 to 9.028 vs observed 7.880 to 8.940. Cubist actually predicts above the observed maximum on the high end and reaches further up than random forest did, but still doesn't quite hit the lowest observed values.
- Correlation between predicted PH and residuals is -0.0271, very close to zero. No strong systematic over/underprediction pattern at different PH levels. The small residual trend we saw in random forest is not present here.
- Overall a strong showing: residuals near zero, low spread, and a broader prediction range than most of the other models. The handful of high-error cases would be worth a closer look before final selection.

1.13 Model comparison

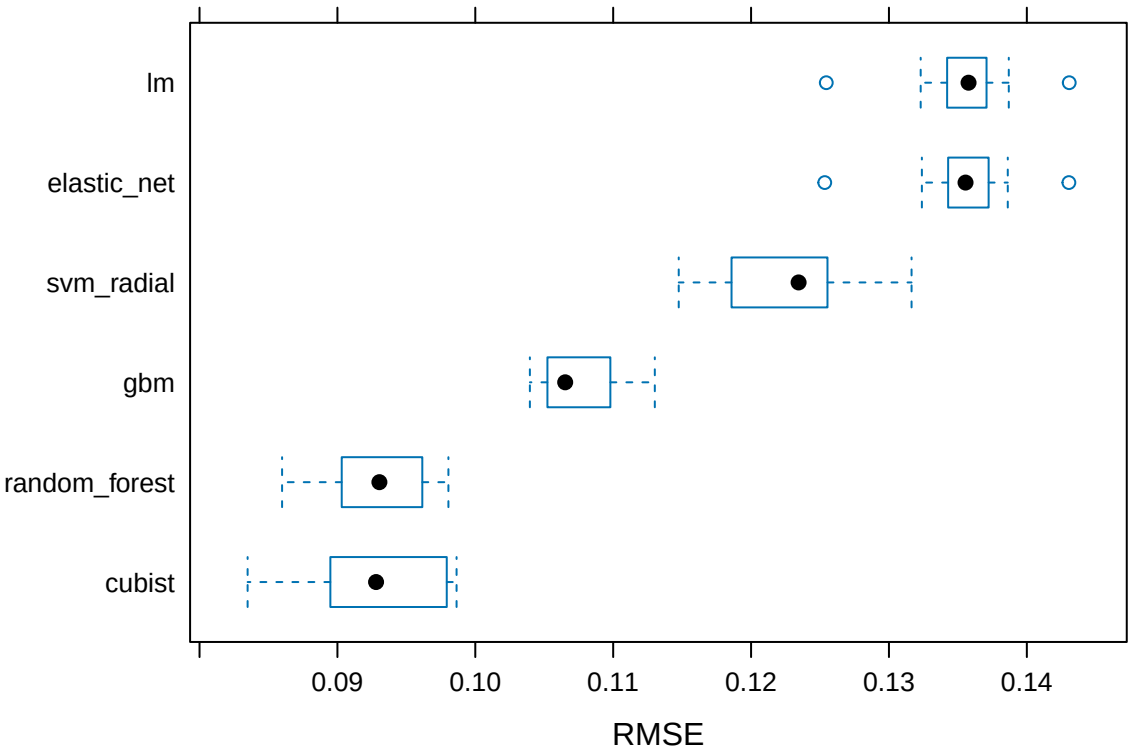
Recap located at the end of the section. Skip there to jump past the code/workflow.

All five models trained on the same shared CV folds. Comparing on identical resampling structure via `resamples()`:

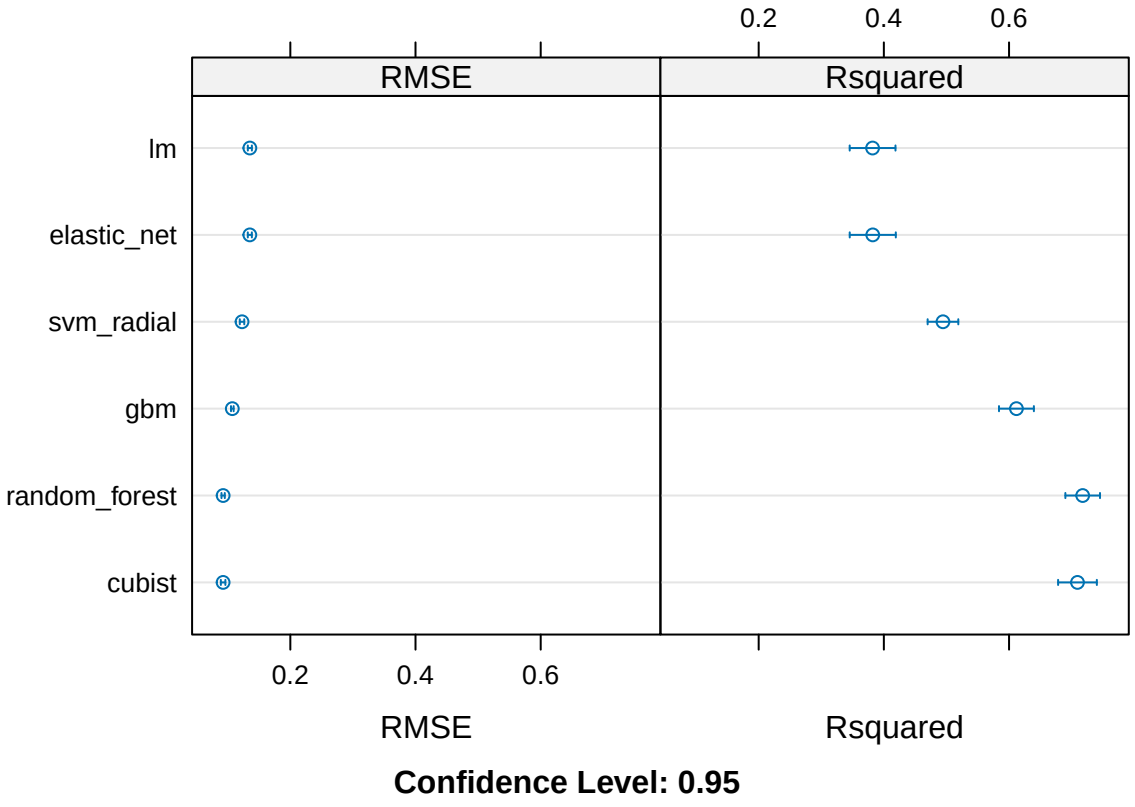
```r
resamps <- resamples(models)

# Per-model distribution of CV RMSE, R-squared, MAE
summary(resamps)
```

```
##
## Call:
## summary.resamples(object = resamps)
##
## Models: lm, elastic_net, random_forest, gbm, svm_radial, cubist
## Number of resamples: 10
##
## MAE
##                     Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## lm            0.09944406 0.10344348 0.10579847 0.10608045 0.10801644 0.11503317
## elastic_net   0.09931834 0.10366849 0.10565713 0.10608166 0.10814000 0.11490515
## random_forest 0.06301973 0.06535915 0.06826452 0.06724975 0.06913352 0.07021189
## gbm           0.07716085 0.08009642 0.08069963 0.08100282 0.08199387 0.08410312
## svm_radial    0.08474276 0.08916845 0.09162930 0.09201413 0.09451886 0.10020877
## cubist        0.06348532 0.06501563 0.06592527 0.06663032 0.06758502 0.07097344
##               NA's
## lm               0
## elastic_net      0
## random_forest    0
## gbm              0
## svm_radial       0
## cubist           0
##
## RMSE
##                     Min.    1st Qu.     Median       Mean    3rd Qu.       Max.
## lm            0.12545563 0.13435216 0.13577290 0.13532304 0.13683712 0.14307621
## elastic_net   0.12534418 0.13441468 0.13555477 0.13528448 0.13693033 0.14305028
## random_forest 0.08598685 0.09052627 0.09304523 0.09262867 0.09548760 0.09804908
## gbm           0.10395866 0.10536129 0.10651967 0.10729448 0.10913545 0.11301913
## svm_radial    0.11475325 0.11915590 0.12344822 0.12281843 0.12541262 0.13163894
## cubist        0.08348632 0.08981811 0.09280274 0.09259569 0.09716919 0.09864226
##               NA's
## lm               0
## elastic_net      0
## random_forest    0
## gbm              0
## svm_radial       0
## cubist           0
##
## Rsquared
##                    Min.   1st Qu.    Median      Mean   3rd Qu.      Max. NA's
## lm            0.2899607 0.3565501 0.3810568 0.3819844 0.4171481 0.4590614    0
## elastic_net   0.2893673 0.3570577 0.3813374 0.3822709 0.4166551 0.4597328    0
## random_forest 0.6549461 0.6918054 0.7207900 0.7176686 0.7334283 0.7828948    0
## gbm           0.5533038 0.5802150 0.6141933 0.6118269 0.6337196 0.6765891    0
## svm_radial    0.4614281 0.4713815 0.4818061 0.4944277 0.5115837 0.5712570    0
## cubist        0.6180778 0.6944906 0.7108700 0.7093332 0.7304141 0.7711805    0
```

```r
# Side-by-side boxplots of per-fold RMSE; lower median is better, narrow box is more stable
bwplot(resamps, metric = "RMSE")
```




```r
# Means with confidence intervals for RMSE and R-squared
dotplot(resamps, metric = c("RMSE", "Rsquared"))
```




```
# Sorted comparison table from the running metrics
kable(model_metrics %>% arrange(cv_RMSE))
```

model	cv_RMSE	cv_Rsquared	cv_MAE
cubist	0.0925957	0.7093332	0.0666303
random_forest	0.0926287	0.7176686	0.0672497
gbm	0.1072945	0.6118269	0.0810028
svm_radial	0.1228184	0.4944277	0.0920141
elastic_net	0.1352845	0.3822709	0.1060817
lm	0.1353230	0.3819844	0.1060805

1.13.1 Recap

- Five models trained: linear regression (lm), elastic net, random forest (RF), gradient boosting (gbm), and support vector machine with radial kernel (SVM-radial). Ranked by cross-validated RMSE: random forest (0.093) < gradient boosting (0.107) < SVM-radial (0.123) < elastic net (0.135) ~= linear regression (0.135).
- **Random forest wins** with cross-validated RMSE 0.0926, R-squared 0.7177, MAE 0.0672. It cuts the linear-family RMSE by about a third and almost doubles R-squared.
- Gradient boosting came in second (RMSE 0.107, R-squared 0.612). It did better than the linear models and SVM-radial, but didn't match random forest. The tuning landed at the largest settings we tested, so gbm might have kept improving with more tuning room, but the gap to random forest was large enough that we didn't bother to chase it further.

- SVM-radial sits in between the linear and tree-based models (RMSE 0.123, R-squared 0.494). The radial kernel lets it fit curved relationships, which helps it do better than the linear models. But SVM-radial is a single model while random forest and gradient boosting each combine many trees into one prediction, which is probably the main reason those two outperform SVM-radial here.
- Linear regression (lm) and elastic net are basically tied at the bottom (RMSE 0.135, R-squared 0.38). The linear family probably can't improve further on this dataset; elastic net's tuning confirmed there's no useful coefficient shrinkage to extract beyond what plain linear regression already finds.
- **Picking random forest** as our final model. It's the clear leader, generalizes well, and gives us strong variable importance for the business writeup.

1.14 Evaluation set predictions

Recap located at the end of the section. Skip there to jump past the code/workflow.

The deliverable requires predictions on the evaluation set. Right now we have this set up to pick the best model from `model_metrics` based on the lowest CV RMSE. The chunk prepares the evaluation set with the correct feature set and `Brand.Code` factor levels, predict, and write to CSV. As (or if) we add more models to compare in the future, we just want to make sure the new model's name is added to the `model_feature_set` mapping inside the chunk so it knows whether to use the narrow or wide evaluation set.

```
# Find the best model by cross-validated RMSE
best_model_name <- model_metrics %>%
  arrange(cv_RMSE) %>%
  slice(1) %>%
  pull(model)

best_model <- models[[best_model_name]]

cat("Best model selected:", best_model_name, "\n")
```

```
## Best model selected: cubist
```

```
cat("CV RMSE:", round(model_metrics %>% filter(model == best_model_name) %>% pull(cv_RMSE), 4), "\n")
```

```
## CV RMSE: 0.0926
```

```
# Map each model to which feature set it was trained on. If we ADD A NEW MODEL
# must also add an entry here so the eval set gets prepared with the right column
model_feature_set <- c(
  lm = "narrow",
  elastic_net = "narrow",
  svm_radial = "narrow",
  random_forest = "wide",
  gbm = "wide",
  cubist = "wide"
)

if (!best_model_name %in% names(model_feature_set)) {
  warning("Best model '", best_model_name, "' is not in the model_feature_set mapping. ",
          "Defaulting to wide; if that's wrong, update the mapping in this chunk.")
  feature_set_choice <- "wide"
```

```
} else {
  feature_set_choice <- model_feature_set[[best_model_name]]
}

# Prepare the eval set with the right features and Brand.Code level alignment
if (feature_set_choice == "narrow") {
  eval_for_prediction <- test_data_imputed %>%
    mutate(Brand.Code = factor(
      ifelse(is.na(Brand.Code) | Brand.Code == "", "Unknown", as.character(Brand.Code)),
      levels = levels(training_for_modeling$Brand.Code)
    ))
} else {
  eval_for_prediction <- test_data_wide %>%
    mutate(Brand.Code = factor(
      ifelse(is.na(Brand.Code) | Brand.Code == "", "Unknown", as.character(Brand.Code)),
      levels = levels(training_for_modeling_wide$Brand.Code)
    ))
}

# Predict
eval_predictions <- predict(best_model, newdata = eval_for_prediction, na.action = na.pass)

# Flag rows that were sparse in the original eval data (predictor NAs only)
original_predictor_nas <- rowSums(is.na(test_data %>% select(-PH)))

# Build the output data frame
predictions_output <- tibble(
  row_id = seq_along(eval_predictions),
  Brand.Code = as.character(eval_for_prediction$Brand.Code),
  predicted_PH = round(eval_predictions, 4),
  notes = ifelse(original_predictor_nas >= 5,
                 paste0("low confidence: original row had ", original_predictor_nas, " predictor NAs"),
                 "")
)

# Write CSV alongside the rmd
write_csv(predictions_output, file.path(projPath, "abc_beverage_PH_predictions.csv"))

# Show the first few rows for confirmation
head(predictions_output)
```

```
## # A tibble: 6 x 4
##   row_id Brand.Code predicted_PH notes
##    <int> <chr>             <dbl> <chr>
## 1      1 D                  8.60 ""
## 2      2 A                  8.42 ""
## 3      3 B                  8.53 ""
## 4      4 B                  8.59 ""
## 5      5 B                  8.44 ""
## 6      6 B                  8.56 ""
```

1.14.1 Recap

- Predictions for the evaluation set get written to `abc_beverage_PH_predictions.csv` in the project root. The output has four columns: `row_id`, `Brand.Code`, `predicted_PH`, and a `notes` column that flags rows where the original evaluation data had 5+ predictor NAs (since those predictions lean heavily on imputed values).
- The chunk picks the best model dynamically by lowest CV RMSE in `model_metrics`. Right now that's random forest.
- Adding a new model in the future just means appending an entry to `model_feature_set` inside the chunk (narrow or wide); the chunk picks up whichever model has the lowest CV RMSE.

1.15 Business takeaways (Informal)

The non-technical report needs these for ABC Beverage leadership:

1. **The model.** Random forest is our chosen final model with cross-validated RMSE 0.093, R-squared 0.72, MAE 0.067. That's substantially better than the linear family (linear regression and elastic net both at RMSE 0.135, R-squared 0.38). Of the five models we tried, random forest was a clear leader; gradient boosting came in second but did not perform as well.
2. **The factors.** Consistent themes across all five models:
    - `Mnf.Flow`: the single most predictive variable across every model. Lower flow correlates with higher PH.
    - `Brand.Code`: a real driver. Random forest emphasizes Brand C the most (the brand with the lowest baseline pH). Linear models give the strongest weight to Brand D instead, with C and B also significant. Either way, the brand a beverage is made under has a measurable effect on PH.
    - `Pressure.Vacuum`, `Alch.Rel`, `Balling.Lvl`, `Carb.Rel`: surface as important in the tree-based models. These were dropped from the linear-family models due to collinearity, so they're "new" levers that pure linear analysis would have missed.
    - `Temperature`, `Usage.cont`, and `Oxygen.Filler`: appear at or near the top of every model.
3. **The recommendation.** Tighter operational tolerances and sensor monitoring on the top 4-5 predictors (or you could call them levers); each brand should get its own production targets since `Brand.Code` is consistently one of the top predictors of PH (the right settings for Brand A probably aren't the right settings for Brand D). For our random-forest-driven recommendation specifically: focus on `Mnf.Flow`, `Brand.Code`-aware standards, `Pressure.Vacuum`, `Oxygen.Filler`, and `Alch.Rel`.

1.15.1 Recap

- This is our working scratch pad for the executive doc. Random forest is our chosen final model and the factors / recommendation bullets above are the current draft. We'll keep iterating on the phrasing as we polish for the team's non-technical report.